기초 파이썬

2026.10.7.일 & 10.14일

담당교수: 박진호

서강대학교 경제대학원

2026년 2학기

◈ 오늘의 학습 목표

▣ 학습 목표: 정상성, 차분, 자기상관관계의 개념에 대해 숙지

참고: Marco Peixeiro(2024), Time Series Forecasting in Python. 번역판

8.1 시계열 예측의 이해와 단순한 예측

8.2 확률 보행

8.3 이동평균과정 모델링: MA

8.4 자기회귀과정 모델링: AR

8.5 복잡한 시계열 모델링: ARMA

8.6 비정상적 시계열 예측: ARIMA

8.7 계절성을 반영한 시계열 예측: SARIMA

8.8 모형에 외생변수 추가: SARIMAX

제8장 시계열분석 (1)

8.1 시계열 예측의 이해와 단순한 예측

8.1.1 시계열 예측의 이해

◆ 시계열(time series)

□ 시간에 따라 정렬된 데이터 요소들의 집합

o 데이터는 일정한 시간 간격으로 관측가능

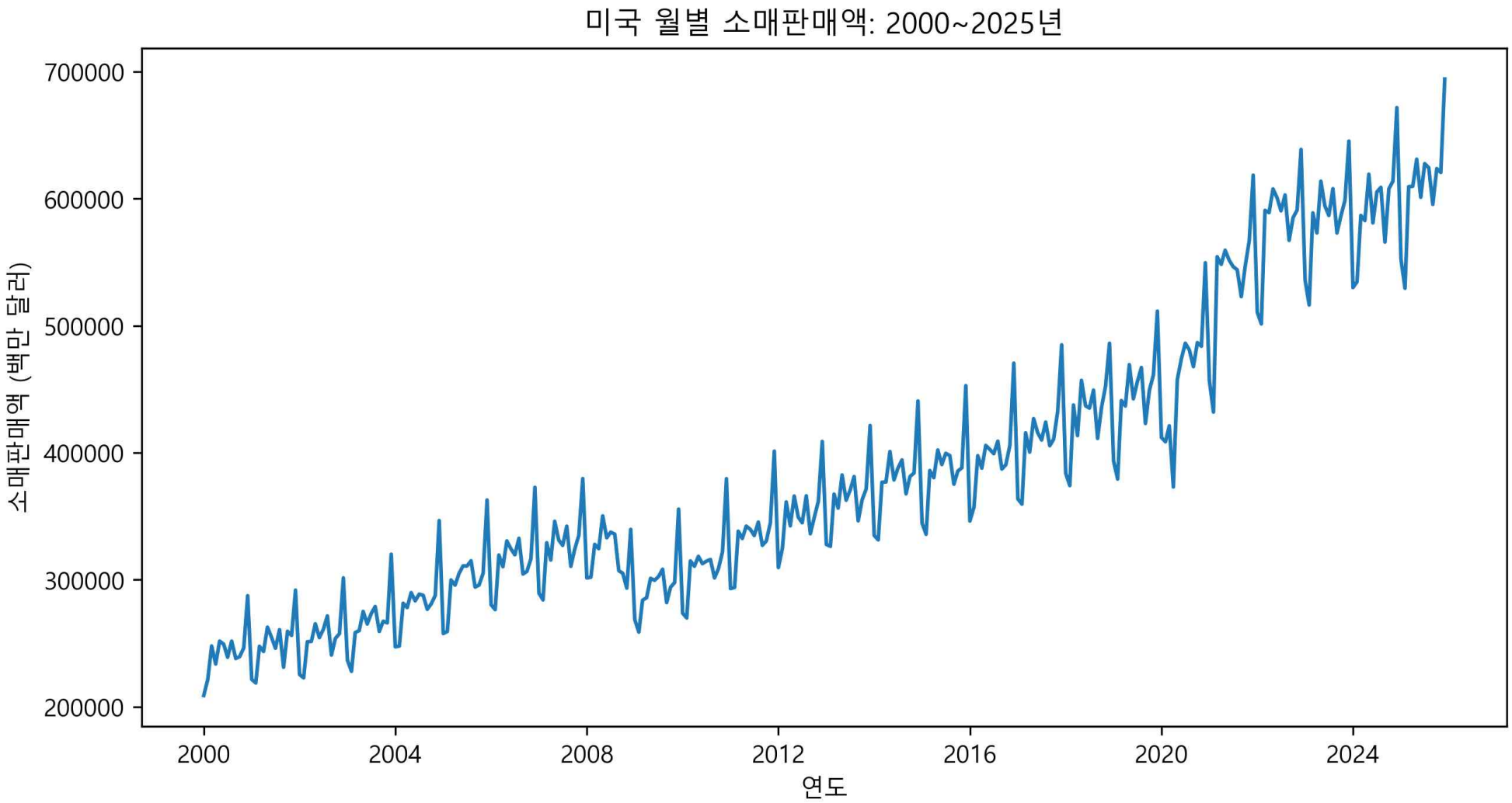
▸ 예: 일별 주가, 월별 소매판매액, 분기별 GDP

□ 아래 그림은 2000~2025년 미국의 월별 소매판매액을 나타냄

o 미국 Census Bureau가 작성하고 FRED에서 제공하는 자료(단위: 백만 달러)

o 계절적 패턴을 관찰하기 위해 비계절조정(Not Seasonally Adjusted) 자료를 사용

o 장기적으로 판매액이 증가하는 추세(trend)와 일정한 주기로 반복되는 계절적 변동(seasonality)을 관찰할 수 있음



◆ 시계열의 구성요소

□ 모든 시계열은 다음과 같은 세 가지 주요 구성요소를 관찰할 수 있음

❶ 추세(trend) :

ㅁ 장기간에 걸쳐 나타나는 시계열의 전반적인 상승 또는 하락 움직임

❷ 계절성(seasonality)

ㅁ 일정한 시간 간격을 두고 반복적으로 나타나는 패턴

❸ 잔차(residual)

ㅁ 추세와 계절성으로 설명되지 않는 불규칙한 변동

□ 시계열을 이렇게 **추세, 계절성, 잔차** 등의 구성요소로 분리해 살펴보는 것을 시계열 분해 (time series decomposition)라고 함

◇ Python을 이용한 시계열 분해 그래프 작성

□ statsmodels 라이브러리에서 시계열을 추세·계절성·잔차로 분해하는 함수를 불러옴

o tsa : time series analysis

① 방법 1

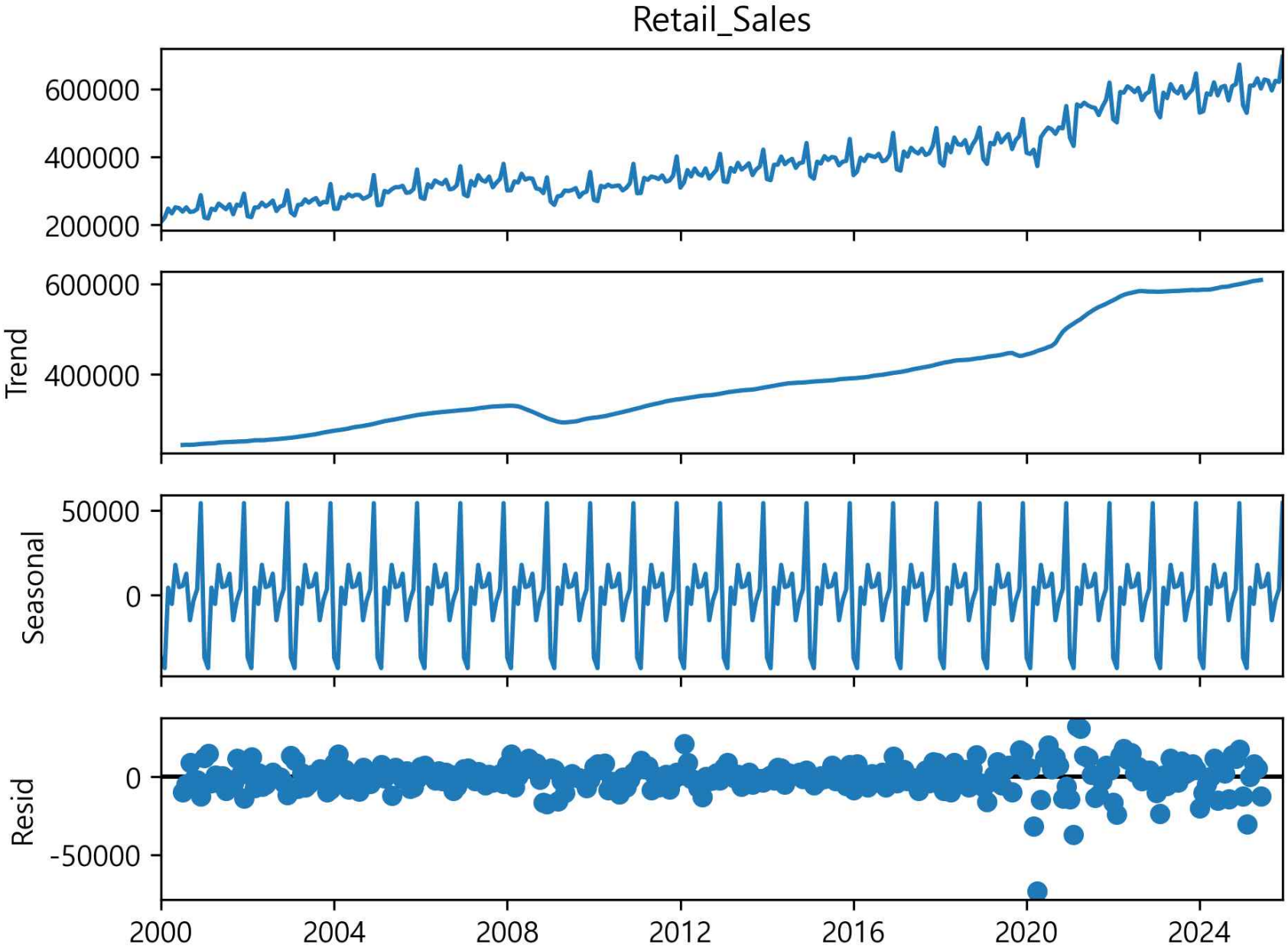
```
from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(retail["Retail_Sales"], model="additive", period=12)
decomposition.plot()
plt.show()
```

② 방법 2

```
import statsmodels.tsa.seasonal as seasonal

decomposition = seasonal.seasonal_decompose(retail["Retail_Sales"], model="additive", period=12)
decomposition.plot()
plt.show()
```


Retail_Sales
600000
400000
200000
Trend
600000
400000
Seasonal
50000
0
Resid
0
-50000
2000
2004
2008
2012
2016
2020
2024

8.1.2 시계열 예측

□ 예측(forecasting)은 과거에 관측된 시계열 정보를 이용하여 미래의 값을 예상하는 것

□ 시계열 예측에서는 시간의 순서가 중요

o 시계열 예측에서 미래의 값을 예측하기 위해 과거의 정보를 이용하므로
데이터를 임의로 섞지 않고 시간의 순서를 유지해야 함

o 예: 미국 월별 소매판매액을 y_t라고 하면,

$y_{t+1} = f(y_t, y_{t-1}, y_{t-2}, ...)$와 같이 과거의 관측값을 이용하여 미래의 값을 예측

◆ 일반적인 예측과 시계열 예측의 차이

□ Prediction은 시간 구조가 없는 일반적인 예측까지 포함하는 넓은 개념인 반면, forecasting은 주로 시간의 흐름에 따른 미래 값의 예측을 의미

구분	일반적인 예측(Prediction)	시계열 예측(Forecasting)
주요 목적	X를 이용하여 Y를 예측	과거 정보를 이용하여 미래 값 예측
시간 순서	꼭 필요하지는 않음	**중요함**
데이터 분할	무작위 분할 가능	**과거 → 미래 순서 유지**
예	소득으로 소비 예측	과거 소매판매로 다음 달 판매 예측

8.2 확률보행

8.2.1 확률보행 프로세스

◆ 확률보행이란?

□ 확률보행(random walk)은 현재 값이 바로 이전 시점의 값과 무작위적인 충격에 의해 결정되는 시계열

$$y_t = y_{t-1} + \epsilon_t \qquad (8.2)$$

- o y_t : t 시점의 값, y_{t-1} : t-1 시점의 값
- o ϵ_t : t 시점에서 새롭게 발생하는 예측하기 어려운 무작위 충격(random shock)
- o random walk : 취객의 걸음걸이
- o 충격이 양(+)이면 값이 상승하고, 음(-)이면 하락
- o 한 번 발생한 충격은 이후의 y_t에 누적되어 영향을 미침

◆ 백색잡음

□ 확률보행에서 ϵ_t는 매 시점에서 무작위로 발생하는 무작위 충격

□ 이러한 무작위 충격 ϵ_t이 다음과 같은 성질을 가지면, 백색잡음(White Noise)이라 함

$$\epsilon_t \sim WN(0, \sigma^{2)}$$

o ∼(tilde)는 확률·통계에서 "어떤 확률분포 또는 확률과정을 따른다" 는 의미로 사용

o 0 : 충격의 평균, σ^2 : 충격의 분산, WN : white nose

□ 일반적으로 백색잡음의 조건은

① $E(\epsilon_t) = 0$: 양의 충격과 음의 충격이 장기적으로 한쪽 방향에 치우치지 않는다는 의미

② $Var(\epsilon_t) = \sigma^2$: 충격의 변동 정도가 시간에 따라 일정하다는 의미

③ $Corr(\epsilon_{t,}\epsilon_{t-k}) = 0,\ k \neq 0$: 서로 다른 시점의 충격 간에는 자기상관관계가 없음

<참고> 백색잡음(White Noise)의 개념과 백색잡음의 조건

◆ 백색잡음 ϵ_t은 다음과 같은 성질을 갖는 확률변수로 정의

$$\epsilon_t \sim WN(0,\sigma^2)$$

❶ 평균이 0

□ 백색잡음은 양(+)과 음(-)의 충격이 특정 방향으로 치우치지 않는다고 가정

$$E(\epsilon_t) = 0$$

o 충격이 항상 0이라는 뜻이 아니라, 양(+)의 충격과 음(-)의 충격이 장기적으로 서로 상쇄되어 평균적인 충격이 0이라는 의미

□ 즉, 장기적으로 무작위 충격의 평균은 0

$$\frac{1}{T}\sum_{t=1}^{T}\epsilon_t \rightarrow 0$$

□ 따라서 백색잡음 자체에는 지속적인 상승 또는 하락 방향이 존재하지 않음

❷ 분산이 일정

□ 각 시점의 충격은 동일한 분산을 가진다고 가정

□ 분산의 정의는 $Var(\epsilon_t) = E[(\epsilon_t - E(\epsilon_t))^2]$

□ 앞에서 $E(\epsilon_t) = 0$이므로 $Var(\epsilon_t) = E[(\epsilon_t - 0)^2] = E[(\epsilon)^2] = \sigma^2$

❸ 서로 다른 시점의 충격은 상관관계가 없음

□ 시차가 k인 두 충격의 상관계수는

$$Corr(\epsilon_t, \epsilon_{t-k}) = \frac{Cov(\epsilon_t, \epsilon_{t-k})}{\sqrt{Var(\epsilon_t)\,Var(\epsilon_{t-k})}}$$

□ 백색잡음에서는 서로 다른 시점의 충격이 상관되지 않는다고 가정하므로

① $Cov(\epsilon_t, \epsilon_{t-k}) = 0,\ k \neq 0,$ 그리고 ② $Var(\epsilon_t) = Var(\epsilon_{t-k}) = \sigma^2$

□ 따라서 $Corr(\epsilon_t, \epsilon_{t-k}) = 0,\ k \neq 0$ ⇒ 과거 충격으로 현재 충격을 예측할 수 없음

◆ 표류항이 있는 확률보행

□ 기본적인 확률보행에서는 현재 값이 이전 값과 무작위 충격에 의해 결정

$$y_t = y_{t-1} + \epsilon_t$$

o 백색잡음의 평균이 $E(\epsilon_t) = 0$이므로 변화가 특정 방향으로 지속되는 것을 가정하지 않음

□ 여기에 일정한 상수 μ를 추가하면 표류항이 있는 확률보행(random walk with drift)이 됨

$$y_t = \mu + y_{t-1} + \epsilon_t \qquad (8.2)$$

o μ : 표류항(drift), 즉 시간에 따라 변하지 않는 상수(일정한 값)

o ϵ_t : 평균이 0인 무작위 충격

o 따라서 현재 값은 이전 값에 일정한 표류항과 새로운 무작위 충격이 더해져 결정됨

□ μ의 의미는 확률보행의 변화량을 이용하면 쉽게 이해할 수 있음

양변에서 y_{t-1}을 빼면 $y_t - y_{t-1} = \mu + \epsilon_t$

따라서 $\Delta y_t = \mu + \epsilon_t$

양변에 기대값을 취하면 $E(\Delta y_t) = E(\mu + \epsilon_t)$

μ는 상수이고 $E(\epsilon_t) = 0$이므로 $E(\Delta y_t) = \mu + 0$

따라서 $E(\Delta y_t) = \mu$

□ 그러므로 μ는 확률보행의 기간별 변화량 Δy_t의 기댓값으로 다음과 같은 의미를 지님

- o $\mu > 0$: 평균적으로 증가하는 방향으로 표류
- o $\mu < 0$: 평균적으로 감소하는 방향으로 표류
- o $\mu = 0$: 기본적인 확률보행

◆ 표류항은 시간이 지나면서 누적

□ μ는 시간에 따라 변하지 않는 상수이지만, 매 기간 반복해 더해지므로 그 효과는 시간에 따라 누적

초기값을 y_0라고 하면, $y_1 = y_0 + \mu + \epsilon_1$

$$y_2 = y_1 + \mu + \epsilon_2$$

$$y_3 = y_2 + \mu + \epsilon_3$$

두 번째 식 y_2에 첫 번째 식 y_1을 대입하면

$$y_2 = y_0 + 2\mu + \epsilon_1 + \epsilon_2$$

세 번째 식 y_3에 첫 번째 식 y_2을 대입하면

$$y_3 = (y_0 + 2\mu + \epsilon_1 + \epsilon_2) + \mu + \epsilon_3 = y_0 + 3\mu + \epsilon_1 + \epsilon_2 + \epsilon_3$$

같은 과정을 반복하면 $y_t = y_0 + \mu t + \sum_{i=1}^{t} \epsilon_i$

양변에 기대값을 취하면 $E(y_t) = E(y_0 + \mu t + \sum_{i=1}^{t} \epsilon_i)$

기대값의 선형성에 의해 $E(y_t) = E(y_0) + E(\mu t) + E(\sum_{i=1}^{t} \epsilon_i)$

y_0와 μ를 상수로 두면 $E(y_t) = y_0 + \mu t + E(\sum_{i=1}^{t} \epsilon_i)$

백색잡음은 각 시점에서 $E(\epsilon_i) = 0$ 이므로

$$\sum_{i=1}^{t} E(\epsilon_i) = \underbrace{0 + 0 + \cdots + 0}_{t\text{개}} = 0$$

따라서 기댓값은 $E(y_t) = y_0 + \mu t$

◇ 의미

□ 표류항 μ는 시간에 따라 변하지 않는 상수이며, 기간별 변화량의 기대값을 나타냄

□ 표류항이 매 기간 누적되면서 y_t의 기대값은 시간에 따라 증가하거나 감소

8.2.2 확률보행 모형의 시뮬레이션

□ 기본적인 확률보행 모형을 이용하여 시계열을 직접 생성

기본적인 확률보행 모형은 $y_t = y_{t-1} + \epsilon_t$

초기값을 $y_0 = 0$ 으로 설정

t = 1이면 $y_1 = y_0 + \epsilon_1 = 0 + \epsilon_1 = \epsilon_1$

t = 2이면 $y_2 = y_1 + \epsilon_2 = \epsilon_1 + \epsilon_2$

t = 3이면 $y_3 = y_2 + \epsilon_3 = \epsilon_1 + \epsilon_2 + \epsilon_3$

$\vdots$

t = T이면 $y_T = \epsilon_1 + \epsilon_2 + \epsilon_3 + \cdots + \epsilon_T$

같은 과정을 반복하면 $y_T = \epsilon_1 + \epsilon_2 + \cdots + \epsilon_T = \sum_{i=1}^{T} \epsilon_i$

⇒ 확률보행은 시점마다 발생하는 무작위 충격이 누적되어 만들어지는 시계열

◆ Python을 이용한 확률보행 생성

```
import numpy as np

np.random.seed(42)

steps = np.random.standard_normal(1000)
steps[0] = 0

random_walk = np.cumsum(steps)
```

◇ 코드 설명

□ np.random.seed(42)

- o 난수 생성 결과를 재현하기 위해 seed 값을 42로 설정
- o seed는 어떤 난수가 나올지를 결정하는 난수 생성 알고리즘의 초기 설정값
- o 동일한 seed 번호를 사용하면 코드를 여러 번 실행해도 똑같은 결과를 얻음

□ np.random.standard_normal(1000)

o 평균이 0이고 분산이 1인 표준정규분포에서 1,000개의 난수를 생성: $\epsilon_t \sim N(0,1)$

o 이 난수들을 확률보행에 들어가는 무작위 충격 ϵ_t로 사용

□ steps[0] = 0

o 첫 번째 값을 0으로 설정하여 확률보행의 출발점을 0으로 설정

□ np.cumsum(steps)

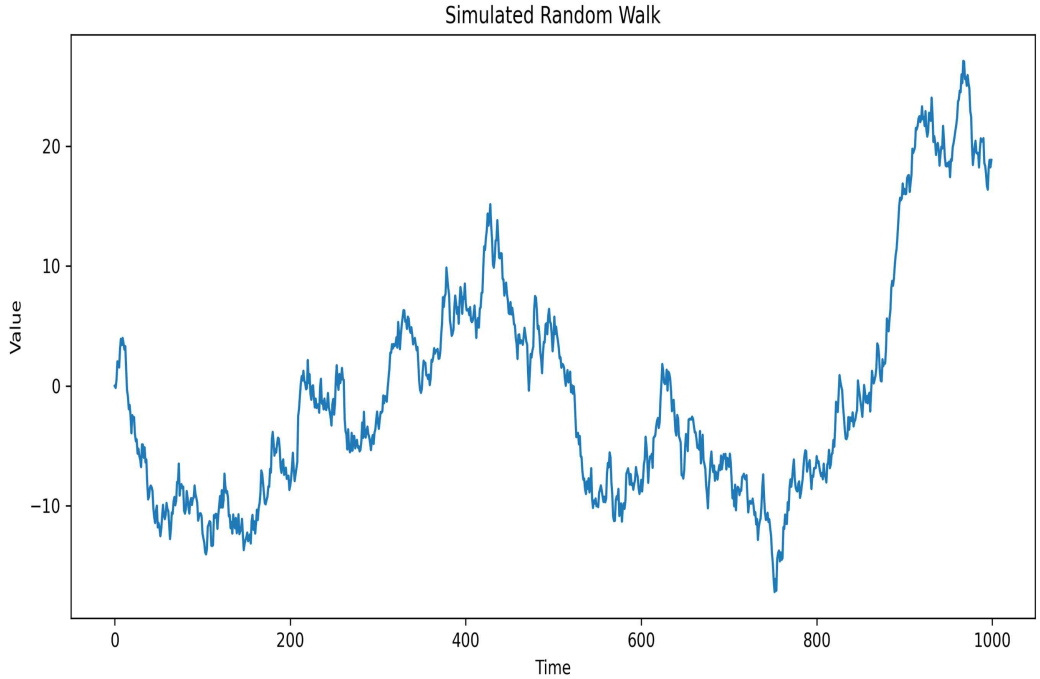
o cumsum은 cumulative sum(누적합)을 의미

o 예 : $\epsilon_1 = 1$, $\epsilon_2 = -0.5$, $\epsilon_3 = 2$인 경우

$$y_1 = 1, \qquad y_2 = 1 - 0.5 = 0.5, \qquad y_3 = 1 - 0.5 + 2 = 2.5$$

◆ 시뮬레이션 결과의 시각화

```
import matplotlib.pyplot as plt
fig, ax = plt.subplots(figsize=(10, 5))
ax.plot(random_walk)
ax.set_xlabel("Time")
ax.set_ylabel("Value")
ax.set_title("Simulated Random Walk")
plt.show()
```



□ 확률 보행의 특성을 확인할 수 있음

- o **단기간의 급격한 변화**(양의 추세, 음의 추세) + **장기 추세**
- o 데이터를 생성할 때 별도의 추세를 넣은 것은 아니고, 백색잡음이 우연히 같은 방향으로 누적되면서 추세처럼 보이는 구간이 만들어진 것

8.2.3 정상성과 변환

(1) 정상성

◆ 정상적 시계열(stationary time series)

□ 시계열의 주요 통계적 특성이 시간에 따라 변하지 않고 일정하게 유지되는 시계열로서 다음 세 가지 조건을 모두 만족해야 정상성이 성립

❶ 평균(mean)이 시간에 따라 일정 : $E(y_t) = \mu$

❷ 분산(variance)이 시간에 따라 일정: $Var(y_t) = \sigma^2$

❸ 서로 다른 시점 간의 관계인 자기 공분산 또는 자기상관관계가 특정 시점 자체가 아니라 두 시점 사이의 간격(시차, lag)에 의해 결정됨

$$Cov(y_t, y_{t-k}) = \gamma_k$$

예: $Cov(y_{10}, y_9) = Cov(y_{50}, y_{49}) = \gamma_1$, $Cov(y_{10}, y_8) = Cov(y_{50}, y_{48}) = \gamma_2$

◆ 왜 정상성이 중요한가?

□ 많은 시계열 모형은 과거에 관찰된 시계열의 통계적 관계가 미래에도 유지된다는 것을 전제로 예측

- o 이동평균(MA), 자기회귀(AR), 자기회귀이동평균(ARMA) 모형 등에서는 정상성이 중요한 가정

□ 그러나 시계열의 평균이나 분산 등이 시간에 따라 계속 변하면 과거 자료에서 추정한 관계를 미래에도 그대로 적용하기 어려움

□ 따라서 비정상 시계열은 적절한 변환을 통해 정상성에 가깝게 만든 후 분석

◆ 그렇다면 확률보행은 정상적인가?

□ 확률보행 $y_t = y_{t-1} + \epsilon_t$에서 ϵ_t는 I.i.d 라고 가정

$$\epsilon_t \overset{i.i.d}{\sim} (0, \sigma^2)$$

□ i.i.d. (independent and identically distributed)

- independent(독립적): 한 시점의 충격이 다른 시점의 충격에 영향을 주지 않음
- identically distributed(동일한 분포): 모든 시점의 충격이 동일한 확률분포를 따름

→ 평균과 분산이 동일

□ 초기값을 y_0 = 0으로 설정하고, 시계열을 직접 생성

앞에서 보듯이 $y_t = y_{t-1} + \epsilon_t$를 반복 대입하면

$$y_T = \epsilon_1 + \epsilon_2 + \cdots + \epsilon_T = \sum_{i=1}^{T} \epsilon_i \quad \Rightarrow \quad y_t = \sum_{i=1}^{t} \epsilon_i$$

□ 평균과 분산

- 평균 : $E(y_t) = E\left(\sum_{i=1}^{t} \epsilon_t\right) = 0$ $(y_0 = 0)$
- 분산 : $Var(y_t) = Var\left(\sum_{i=1}^{t} \epsilon_i\right) = \sum_{i=1}^{t} Var(\epsilon_i) = t\sigma^2$

  → t가 증가하면 $\sigma^2, 2\sigma^2, 3\sigma^2, \cdots$ 처럼 분산도 계속 증가
- 자기공분산 : $Cov(\epsilon_t, \epsilon_s) = 0$ $(t \neq s)$

⇒ 시간이 지날수록 새로운 충격이 계속 누적

→ 확률보행은 평균은 일정하지만, 분산($t\sigma^2$)이 시간에 따라 일정하지 않으므로 비정상 (non-stationary) 시계열

(2) 변환

◆ 비정상 시계열에 적절한 수학적 변환(transformation)을 적용하여 평균이나 분산을 안정화하면 정상성에 가까워질 수 있음

❶ 차분(differencing)

□ 서로 다른 시점의 값 사이의 변화량을 계산

□ 확률보행을 1차 차분하면, $\triangle y_t = y_t - y_{t-1} = \epsilon_t$가 되고, ϵ_t는 백색잡음이므로

- 평균: $E(\Delta y_t) = E(\epsilon_t) = 0$
- 분산: $Var(\Delta y_t) = Var(\epsilon_t) = \sigma^2$
- 자기공분산: $Cov(\triangle y_t, \triangle y_{t-k}) = Cov(\epsilon_t, \epsilon_{t-k}) = 0 \qquad (k \neq 0)$

⇒ 평균과 분산이 시간에 따라 일정하고, 서로 다른 시점의 값 사이의 관계(자기공분산)도 변하지 않으므로($Cov(y_t, y_{t-k}) = \gamma_k$) 정상적 시계열

□ 따라서 비정상적인 확률보행은 1차 차분을 통해 정상 시계열로 변환할 수 있음

◆ 표류항이 있는 확률보행은?

□ 앞에서 배운 $y_t = \mu + y_{t-1} + \epsilon_t$ 를 1차 차분하면

$$\Delta y_t = \mu + \epsilon_t$$

따라서

$$E(\Delta y_t) = \mu$$

$$Var(\Delta y_t) = \sigma^2$$

⇒ 표류항이 있어도 차분된 시계열은 평균 μ, 분산 σ^2가 시간에 따라 일정

<참고> **로그 변환은 왜 사용하는가?**

◆ 시계열의 수준이 높아질수록 변동 폭도 커지는 경우, 로그 변환은 절대적 변화보다 비율적 변화를 반영하여 수준 증가에 따른 변동 폭의 확대를 완화

◇ 예: 매출액 변화(원자료, 단위: 만원)

o 절대적 변화의 폭 : 100 → 110은 10만원 증가, 1,000 → 1,100은 100만원 증가

o 비율적 변화의 폭 : 모두 10% 증가

■ 초기 값이 90, 100, 110일 때 평균은 100이고

분산은 $\frac{(90-100)^2+(100-100)^2+(110-100)^2}{3} = \frac{100+0+100}{3} \approx 66.7$

■ 후기 값이 900, 1000, 1100일 때 평균은 1,000이고

분산은 $\frac{(900-1000)^2+(1000-1000)^2+(1100-1000)^2}{3} = \frac{10,000+0+10,00}{3} \approx 6,666.7$

⇒ 시계열의 수준이 커질수록 값의 변동 폭도 커져 분산이 일정하지 않을 수 있다

◇ 로그를 취하면 절대적인 변화량보다 비율적인 변화에 가깝게 표현할 수 있음

log(110) - log(100) = log(1,100) - log(1,000) = log(1.1)

❷ 로그 변환(Log Transformation)

◆ 시간이 지나면서 시계열의 수준이 증가하고 변동 폭도 함께 커지는 경우, 로그 변환을 이용하여 변동 폭을 완화

□ 로그 변환은 큰 값을 상대적으로 더 많이 압축하기 때문에 데이터의 수준이 높아질수록 변동 폭도 커지는 시계열에 유용 → 분산을 안정화하는 데 도움

◆ 로그 차분(Log Difference)

□ $y_t = \ln(Y_t)$로 정의하면,

$$\Delta y_t = y_t - y_{t-1}$$

$$= \ln(Y_t) - \ln(Y_{t-1})$$

$$= \ln\left(\frac{Y_t}{Y_{t-1}}\right)$$

□ $\Delta Y_t = Y_t - Y_{t-1}$라고 하면 $Y_t = Y_{t-1} + \Delta Y_t$이므로,

$$\Delta y_t = \ln\left(\frac{Y_t}{Y_{t-1}}\right) = \ln\left(\frac{Y_{t-1} + \Delta Y_t}{Y_{t-1}}\right) = \ln\left(1 + \frac{\Delta Y_t}{Y_{t-1}}\right)$$

□ Taylor 전개에 따라 $\ln(1+x) \approx x$ (x가 충분히 작은 경우)

□ 따라서, 변화율이 작을 경우 $\ln\left(1 + \frac{\Delta Y_t}{Y_{t-1}}\right) \approx \frac{\Delta Y_t}{Y_{t-1}}$

⇒ 로그 차분은 대략적인 증가율로 해석 가능

◆ 차분과 로그 변환의 역할

대상	문제 상황	주요 변환	효과
평균	시간에 따라 상승·하락하는 추세	**차분**	추세를 완화·제거하여 평균을 안정화
분산	수준이 커지면서 변동 폭도 증가	**로그 변환**	자료 스케일을 압축해 변동 폭을 안정화

(3) 역변환

◆ 시계열을 정상적으로 만들기 위해 로그 변환이나 차분을 적용할 수 있음

□ 그러나 변환된 자료를 이용하여 모형을 추정하고 예측한 경우, 예측 결과 역시 변환된 값으로 나타남

□ 따라서 예측 결과를 해석하거나 실제 값과 비교하기 위해서는 변환된 값을 원래의 단위와 수준(scale)으로 되돌리는 역변환(inverse transformation)이 필요

❶ 로그 변환의 역변환

□ 원래 시계열 y_t를 로그 변환하여 $z_t = \ln(y_t)$ 라 하면,

로그와 지수함수는 서로 역함수 관계이므로 로그 변환된 값을 지수함수를 이용하여 원래 수준으로 되돌릴 수 있음.

양변에 지수함수 $e^{(\bullet)}$를 적용하면 $e^{z_t} = e^{\ln(y_t)}$

따라서 원래 값은 $y_t = e^{z_t} = \exp(z_t)$으로 복원됨

o 예 : $y_t = 100$ 이면 $z_t = \log(100) \approx 4.605$이고, 다시 $\exp(4.605) \approx 100$ 으로 복원됨

❷ 차분의 역변환

□ 1차 차분은 현재 값과 직전 값의 차이를 계산: $\Delta y_t = y_t - y_{t-1}$

위 식을 y_t에 대해 정리하면 $y_t = y_{t-1} + \Delta y_t$

따라서 직전 시점의 값에 차분값을 더하면 원래 수준을 복원할 수 있음

예 : $y_{t-1} = 100, \Delta y_t = 5$ 이면, y_t = 100 + 5 = 105

□ 여러 기간의 차분값을 원래 수준으로 복원하려면 차분값을 순서대로 누적하여 더함 (cumulative sum)

예 : $y_t = 100$ 이고, 예측된 차분값이 $\Delta\hat{y}_{t+1} = 5, \Delta\hat{y}_{t+2} = 3$ 이면

$$\hat{y}_{t+1} = 100 + 5 = 105$$

$$\hat{y}_{t+2} = 105 + 3 = 108$$

❸ 로그 변환과 차분을 모두 적용한 경우

□ 예를 들어 원자료에 먼저 로그를 취하고 다시 차분하였다면

$$z_t = \Delta log(y_t) = \log(y_t) - \log(y_{t-1})$$

예측 결과 $\hat{z}_{t+1}$ 은 원래 y의 수준이 아니라 로그값의 변화량

□ 따라서 역변환은 변환했던 순서의 반대 순서로 수행

- 1단계: 차분을 되돌림 $\widehat{\log(y_{t+1})} = \log(y_t) + \hat{z}_{t+1}$
- 2단계: 로그를 되돌림 $\hat{y}_{t+1} = \exp\left[\widehat{\log(y_{t+1})}\right]$

◆ 정리

원자료 $\xrightarrow{\log}$ 로그값 $\xrightarrow{\text{차분}}$ 변환자료 에 대해 역변환은

예측값 $\xrightarrow{\text{차분 역변환}}$ 로그 수준 $\xrightarrow{\exp}$ 원래 수준

<참고> 로그 차분을 대략적인 증가율로 해석하는 이유

1. 로그 차분 정의

- $y_t = \ln(Y_t)$ 라고 정의하면,

$$\Delta y_t = y_t - y_{t-1}$$
$$= \ln(Y_t) - \ln(Y_{t-1})$$

- 로그의 성질 $\ln A - \ln B = \ln\left(\frac{A}{B}\right)$을 이용하면

$$\Delta y_t = \ln\left(\frac{Y_t}{Y_{t-1}}\right)$$

2. Y_t를 변화량으로 표현

- 변화량을 $\Delta Y_t = Y_t - Y_{t-1}$라고 하면, $Y_t = Y_{t-1} + \Delta Y_t$ 이므로,

$$\Delta y_t = \ln\left(\frac{Y_t}{Y_{t-1}}\right) = \ln\left(\frac{Y_{t-1} + \Delta Y_t}{Y_{t-1}}\right) = \ln\left(1 + \frac{\Delta Y_t}{Y_{t-1}}\right)$$

3. Taylor 근사식 적용

▶ x가 0에 가까울 때, $\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \ldots\cdots$

여기서 $x = \frac{\Delta Y_t}{Y_{t-1}}$라고 놓으면,

$$\ln\left(1 + \frac{\Delta Y_t}{Y_{t-1}}\right) = \frac{\Delta Y_t}{Y_{t-1}} - \frac{1}{2}\left(\frac{\Delta Y_t}{Y_{t-1}}\right)^2 + \frac{1}{3}\left(\frac{\Delta Y_t}{Y_{t-1}}\right)^3 - \ldots$$

4. 그런데 왜 보통 첫 번째 항만 사용하는가?

▶ 변화율 $\frac{\Delta Y_t}{Y_{t-1}}$이 충분히 작으면 제곱항, 세제곱항 등은 훨씬 작아짐

▶ 예: 변화율이 2%라면 x=0.02, x^2=0.0004, x^3=0.000008이므로 고차항의 영향이 매우 작음

▶ 따라서 $\ln(1+x) \approx x$ 라고 근사하여, $\Delta \ln(Y_t) \approx \frac{\Delta Y_t}{Y_{t-1}}$를 얻음[

▶ $\ln(Y_t) - \ln(Y_{t-1}) \approx \frac{Y_t - Y_{t-1}}{Y_{t-1}}$

8.2.4 정상성 검정

◆ 시계열 분석에서는 그래프만으로 정상성을 판단하는 데에는 한계가 있으므로 다음과 같은 과정을 거쳐 정상성을 확인

① 필요한 경우 차분, 로그 변환 등의 변환을 통해 시계열을 정상성에 가깝게 만듦

② 정상성 검정을 수행하여 정상성 여부를 확인 → 여전히 비정상적이면 추가 변환 검토

□ 대표적인 정상성 검정으로는 ADF(Augmented Dickey-Fuller) 검정을 들 수 있음

◆ ADF 검정의 아이디어

□ ADF 검정은 시계열에 단위근(unit root)이 존재하는지를 검정하여 정상성 여부를 판단

H_0 : 단위근이 존재한다 ⇒ 비정상, H_1 : 단위근이 존재하지 않는다 ⇒ 정상

□ 따라서 ADF 검정에서 중요한 것은 단위근이 존재한다는 귀무가설을 기각할 수 있는가임

(1) 단위근

◆ 다음과 같은 현재 값이 이전 시점의 값에 영향을 받는 간단한 시계열 모형을 생각

$$y_t = \mu + \phi y_{t-1} + \epsilon_t$$

- o μ : 상수항
- o ϕ: 이전 시점의 값 y_{t-1}이 현재 값 y_t에 미치는 정도
- o ϵ_t: 시점 t에서 새롭게 발생하는 무작위 충격

◆ $|\phi| < 1$인 경우

□ 예를 들어 μ = 0이고, $y_t = 0.5y_{t-1} + \epsilon_t$라고 하자.

□ 어떤 시점에 충격이 10만큼 발생했다고 가정하면 이 충격의 영향은 시간이 지나면서 아래와 같이 점차 작아짐

10

10(0.5) = 5

$10(0.5)^2$ = 2.5

$10(0.5)^3$ = 1.25

□ 일반적으로 충격의 영향은 k 기간 후에는 $\phi^k \epsilon_t$의 형태로 나타남

□ 따라서 $|\phi| < 1$ 이면, $\phi^k \rightarrow 0$ ($k \rightarrow \infty$)

□ (의미) 한 번 발생한 충격의 영향이 시간이 지나면서 사라지므로 시계열이 원래의 평균 수준으로 되돌아가려는 성질이 있음

□ 따라서 $|\phi| < 1$ 이면, 정상적 시계열

◆ ϕ = 1 인 경우

□ 이번에는 μ = 0이고, $y_t = y_{t-1} + \epsilon_t$라고 하자. 이는 확률보행과 같음

□ 시점 t + 1에서 새로운 충격 ϵ_{t+1}이 발생하면, $y_{t+1} = y_t + \epsilon_{t+1}$

시점 t + 2에서는 새로운 충격 ϵ_{t+2}이 발생하므로, $y_{t+2} = y_{t+1} + \epsilon_{t+2}$

$y_{t+2} = y_{t+1} + \epsilon_{t+2}$에 $y_{t+1} = y_t + \epsilon_{t+1}$을 대입하면, $y_{t+2} = y_t + \epsilon_{t+1} + \epsilon_{t+2}$

□ 같은 과정을 반복하면, k 기간 후에는 $y_{t+k} = y_t + \epsilon_{t+1} + \epsilon_{t+2} + \cdots + \epsilon_{t+k}$

□ 따라서 t + 1에 발생한 충격 ϵ_{t+1}는 y_{t+1}에 반영된 뒤에도 사라지지 않고 y_{t+2}, y_{t+3},$\cdots$, y_{t+k}에 그대로 전달됨

□ (의미) ϕ = 1 인 확률보행에서는 한 번 발생한 충격이 시계열의 수준에 반영된 이후에도 그 효과가 계속 남아 있음

□ 따라서 ϕ = 1인 경우 단위근이 존재하며, 이는 비정상 시계열을 의미

◆ ϕ = -1인 경우

□ μ = 0이고, $y_t = -y_{t-1} + \epsilon_t$라고 하자.

□ 어떤 시점에 10만큼의 충격이 발생했다고 가정하면, 이 충격의 영향은 시간이 지나면서 아래와 같이 나타남

10

10(-1) = -10

$10(-1)^2$ = 10

$10(-1)^3$ = -10

⋮

□ 일반적으로 k 기간 후 충격의 영향은 $(-1)^k\epsilon_t$이며,

$$(-1)^k = \begin{cases} 1 & k\text{가 짝수} \\ -1 & k\text{가 홀수} \end{cases} \text{ 이므로 } \quad |(-1)^k| = 1$$

□ 따라서 시간이 지나도 충격의 방향(부호)은 반복해서 바뀌지만 크기는 감소하지 않음

□ (의미) 한 번 발생한 충격의 영향이 양(+)과 음(-)의 방향으로 번갈아 나타나지만 그 크기가 소멸하지 않으므로 정상성 조건을 만족하지 않음

◆ 정리

▸ $|\phi|<1 \Rightarrow \phi^k \to 0 \Rightarrow$ 충격 소멸 : 정상적 시계열

▸ $\phi=1 \Rightarrow \phi^k=1 \Rightarrow$ 충격 지속 : 비정상적 시계열

▸ $\phi=-1 \Rightarrow \phi^k=(-1)^k \Rightarrow$ 충격의 부호만 반복 : 비정상적 시계열

(2) ADF 검정

□ 앞에서 살펴본 간단한 시계열 모형에서 출발

$$y_t = \mu + \phi y_{t-1} + \epsilon_t$$

◆ ADF 검정을 위한 식의 변환

□ ADF 검정에서는 $\phi = 1$ 인지를 검정하기 쉽도록 위 식을 차분 형태로 변환

양변에서 y_{t-1}을 빼면 $y_t - y_{t-1} = \mu + \phi y_{t-1} - y_{t-1} + \epsilon_t$

따라서 $\Delta y_t = \mu + (\phi - 1) y_{t-1} + \epsilon_t$

여기서 $\gamma = \phi - 1$ 이라고 정의하면 $\Delta y_t = \mu + \gamma y_{t-1} + \epsilon_t$

◆ 단위근과 γ의 관계

□ 만약 단위근이 존재한다면 $\phi = 1$ 이므로

$\gamma = \phi - 1$ = 1- 1 = 0

따라서 $\gamma = 0$ 인지를 검정하면 $\phi = 1$ 인지를 검정할 수 있음

$\phi = 1 \Leftrightarrow \gamma = 0$

<참고> ◆ 왜 이름이 단위근인가?

□ AR(1) 모형을 $y_t = \phi y_{t-1} + \epsilon_t$ 라 하면 시차 연산자 L을 이용하여 아래와 같이 표현가능

$$(1-\phi L)y_t = \epsilon_t$$

□ $1-\phi L$은 AR(1) 모형의 AR 다항식이며, 이 다항식을 0으로 만드는 값, 즉 근(root)을 구하기 위해 다음과 같이 놓음

$$(1-\phi L) = 0$$

따라서 $L = \frac{1}{\phi}$

□ 확률보행에서는 $\phi = 1$이므로 L = 1

o 근(root)이 1(unit)이기 때문에 단위근(unit root)이라고 부름

◆ ADF 검정의 가설

□ 귀무가설은 $H_0 : \gamma = 0$

그런데 $\gamma = \phi - 1$ 이므로 $\gamma = 0$ 이면 $\phi - 1 = 0$

따라서 $\phi = 1$을 의미

H_0 : 단위근이 존재한다 ⇒ 비정상적 시계열

□ 대립가설은 $H_1 : \gamma < 0$

이는 단위근이 존재하지 않는 방향을 의미하며 정상성을 지지

◆ p-값을 이용한 판단

□ 일반적으로 유의수준을 5%로 설정하면 p-값 < 0.05

→ “귀무가설 H_0 : 단위근이 존재한다” 를 기각

→ 정상 시계열로 판단

□ 반대로 p-값 ≥ 0.05

→ “귀무가설 H_0 : 단위근이 존재한다” 를 기각할 수 없음

→ 비정상 시계열일 가능성을 배제할 수 없음

◆ ADF 통계량을 이용한 판단

□ DF 검정 결과에서는 주로 다음 두 가지가 제시됨: ① ADF 통계량, ② p-값

❶ ADF 통계량은 단위근이 존재한다는 귀무가설을 평가하기 위해 계산되는 검정통계량

o 일반적으로 음수(-)의 값으로 나타남

o 값이 더 작을수록(더 큰 음수일수록) 귀무가설을 기각하는 방향

o ADF 통계량을 임계값(critical value)과 비교하여 판단할 수 있음

❷ p-값은 귀무가설을 기각할 것인지를 보다 간편하게 판단하는 데 사용

o $p < 0.05$ 이면, 귀무가설(단위근 존재 가설) 기각 → 정상성지지

o $p \geq 0.05$ 이면, 귀무가설을 기각하지 못함 → 단위근의 존재를 배제할 수 없음

◆ <참고> ADF 통계량과 임계값의 해석

□ ADF 통계량은 t-검정의 임계값이 아니라, ADF 검정에 해당하는 임계값과 비교

□ ADF 통계량이 임계값보다 더 작은 값, 즉 더 음수이면 귀무가설을 기각

o 예를 들어 다음과 같은 임계값이 제시되었다고 하자.

유의수준	임계값	의미
1%	-3.43	ADF < -3.43이면 H_0 기각
5%	-2.86	ADF < -2.86이면 H_0 기각
10%	-2.57	ADF < -2.57이면 H_0 기각

주의: 임계값은 모형의 설정과 표본 등에 따라 달라질 수 있음

□ 예) ADF 통계값: -3.10

▸ 5% 수준에서 -3.10 < -2.86이므로 H_0 기각 → 단위근 존재 가설 기각 → 5% 유의수준에서 정상성 지지

▸ 1% 수준에서 -3.10 > -3.43이므로 H_0 기각 못함 → 1% 유의수준에서는 정상성을 지지할 만큼 증거가 강하지 않음

□ 실무에서는 p-값을 이용하면 간단

▸ ADF 검정 결과에서는 ADF 통계량, p-value 및 각 유의수준의 임계값이 함께 제시됨

8.2.5 실습: 실제 데이터의 정상성 확인

◆ 데이터 불러오기

```
# 모듈 설치: 한 번만 실행
%pip install pandas-datareader
```

1. 필요한 라이브러리 불러오기

```
import pandas as pd
import pandas_datareader.data as web
import datetime
```

2. FRED에서 직접 불러오기

```
start = datetime.datetime(2000, 1, 1)
end = datetime.datetime(2025, 12, 31)
```

```
retail = web.DataReader( “RSXFSN“, “fred“, start, end )
retail.head()
```

□ web.DataReader()는 외부 온라인 데이터 소스(Web)에서 데이터를 불러와 Pandas DataFrame 형태로 변환해 주는 함수

순서와 매개변수	입력값	역할
① name	“RSXFSN“	- RSXFSN은 가져올 데이터의 티커(Ticker) ID - 미국 상무부(Census Bureau)에서 발표하는 월간 소매 및 식품 서비스 판매 데이터(Advance Retail Sales: Retail Trade) - 월별 · 비계절조정. 단위: 백만 달러
② data_source	“fred“	- 데이터를 제공하는 웹 서비스 이름 (“fred“, “yahoo“, “stooq“ 등) - 미국 세인트루이스 연방준비은행의 경제 데이터베이스인 FRED (Federal Reserve Economic Data) 서버에 API 연결을 요청
③ start	start	데이터 수집 시작일 (datetime 객체 또는 “YYYY-MM-DD“ 문자열)
④ end	end	데이터 수집 종료일 (datetime 객체 또는 “YYYY-MM-DD“ 문자열)

3. 열 이름을 이해하기 쉽게 변경

```
retail.columns = ["Retail_Sales"]
retail.head()
```

4. PC에 저장

```
retail.to_csv( r'E:\실습데이터\US_Retail_Sales_2000_2025.csv' )
```

5. 저장한 CSV 파일 다시 불러오기

```
retail = pd.read_csv(
    r'E:\실습데이터\US_Retail_Sales_2000_2025.csv', index_col="DATE", parse_dates=True
)
retail.head()
```

6. 시계열 분해

```
from statsmodels.tsa.seasonal import seasonal_decompose

decomposition = seasonal_decompose(
    retail[“Retail_Sales“],
    model=“additive“,
    period=12
)decomposition.plot()

plt.show()
```

◇ 미국 월별 소매판매액 retail을 사용하여 원자료, 로그 변환, 1차 차분, 로그 차분을 생성

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# 원자료
retail["Original"] = retail["Retail_Sales"]

# 로그 변환
retail["Log"] = np.log(retail["Retail_Sales"])

# 1차 차분
retail["Diff"] = retail["Retail_Sales"].diff()

# 로그 + 1차 차분
retail["Log_Diff"] = np.log(retail["Retail_Sales"]).diff()
```

◇ 그림으로 비교 : 평균과 변동 폭이 가장 일정해 보이는 그래프는?

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(2, 2, figsize=(12, 7))

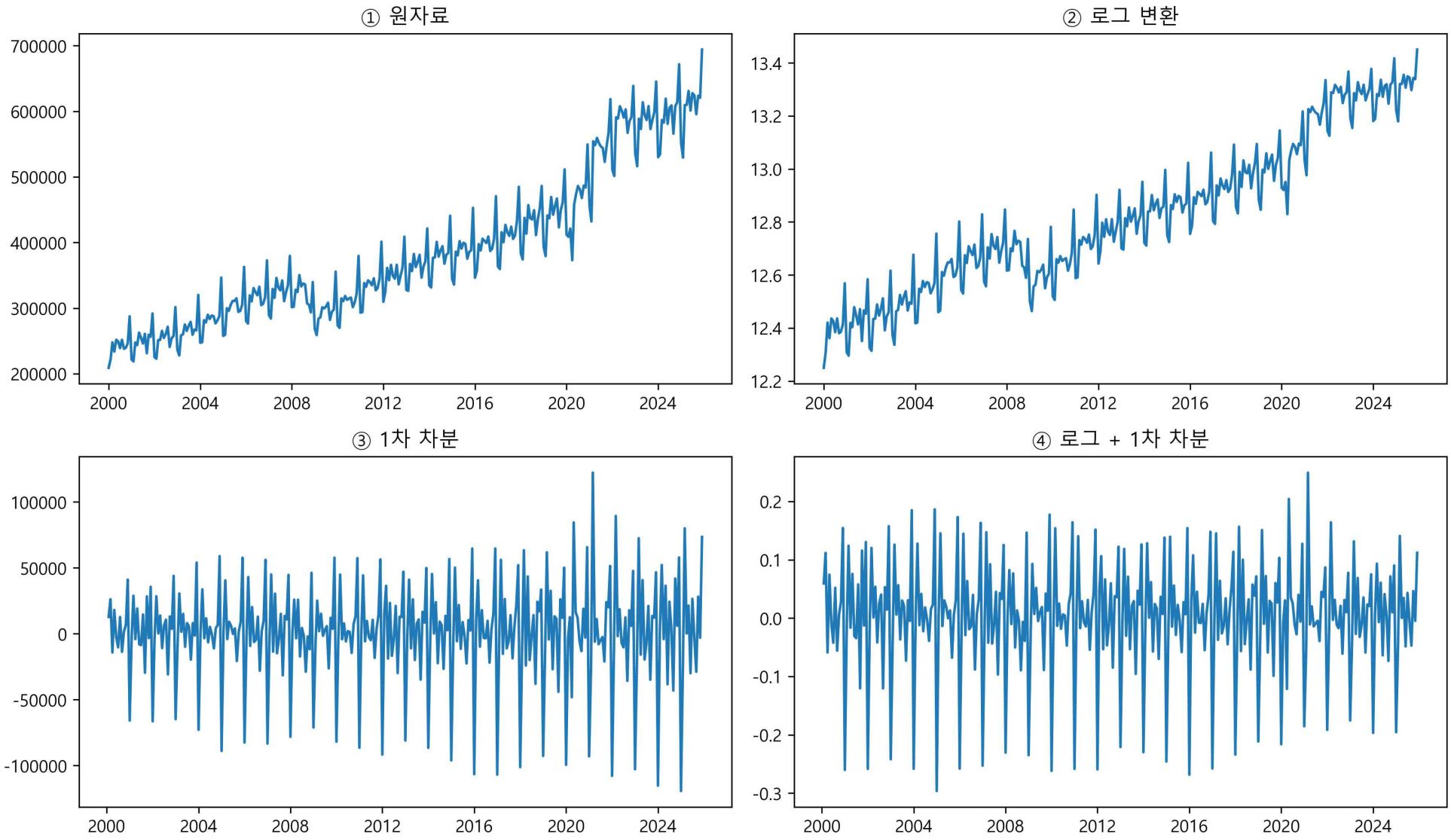
ax[0, 0].plot(retail["Original"])
ax[0, 0].set_title("① 원자료")

ax[0, 1].plot(retail["Log"])
ax[0, 1].set_title("② 로그 변환")

ax[1, 0].plot(retail["Diff"])
ax[1, 0].set_title("③ 1차 차분")

ax[1, 1].plot(retail["Log_Diff"])
ax[1, 1].set_title("④ 로그 + 1차 차분")

plt.tight_layout()
plt.show()
```


① 원자료
② 로그 변환
③ 1차 차분
④ 로그 + 1차 차분
700000
600000
500000
400000
300000
200000
13.4
13.2
13.0
12.8
12.6
12.4
12.2
100000
50000
0
-50000
-100000
0.2
0.1
0.0
-0.1
-0.2
-0.3
2000
2004
2008
2012
2016
2020
2024

◇ ADF 검정

```
from statsmodels.tsa.stattools import adfuller

# 원자료
result = adfuller(retail["Original"].dropna())
print("ADF Statistic:", result[0])
print("p-value:", result[1])

# 로그 변환
retail["Log"] = np.log(retail["Retail_Sales"])

# 1차 차분
retail["Diff"] = retail["Retail_Sales"].diff()

# 로그 + 1차 차분
retail["Log_Diff"] = np.log(retail["Retail_Sales"]).diff()
```

□ 4개 시계열을 비교하려면 간단한 함수를 사용

```
def adf_test(series, name):
    result = adfuller(series.dropna())

    print(name)
    print("ADF Statistic:", round(result[0], 3))
    print("p-value:", round(result[1], 4))
    print()

adf_test(retail["Original"], "원자료")
adf_test(retail["Log"], "로그 변환")
adf_test(retail["Diff"], "1차 차분")
adf_test(retail["Log_Diff"], "로그 + 1차 차분")
```

자료	ADF 통계량	p-value	판단
원자료	0.534	0.9859	비정상성을 배제할 수 없음
로그 변환	-0.118	0.9476	비정상성을 배제할 수 없음
1차 차분	-3.442	0.0096	정상성 지지
로그 + 1차 차분	‘-4.054	0.0012	정상성 지지

□ 원자료

▸ 그래프: 소매판매액이 장기적으로 상승

▸ ADF 결과: 단위근이 존재한다는 귀무가설을 기각하지 못함 → 비정상성을 배제할 수 없음

⇒ 그래프에서 관찰한 비정상적인 모습과 ADF 결과가 일치

□ 로그 변환

▸ 그래프: 로그를 취하면 원자료보다 값의 변동 폭은 완화되지만, 상승 추세는 남아 있음

▸ ADF 결과: 귀무가설을 전혀 기각하지 못함

⇒ 로그 변환은 변동 폭을 완화하는 데 도움을 주지만 추세를 제거하는 것은 아님

□ 1차 차분

▸ 그래프: 차분 후에는 원자료에서 보이던 지속적인 상승 추세가 사라지고 값들이 대체로 0을 중심으로 움직임

▸ ADF 결과: 단위근 귀무가설 기각 → 정상성 지지

□ 로그 + 1차 차분

▸ 그래프: 값들이 대체로 0을 중심으로 움직이며, 원자료에서 보이던 상승 추세가 사라짐

▸ ADF 결과: 단위근 귀무가설을 강하게 기각 → 정상성 지지

▸ 로그차분은 대략 소매판매액의 증가율로 해석: $\Delta \log(y_t) \approx \frac{y_t - y_{t-1}}{y_{t-1}}$

◇ ③과 ④를 보면, 일정한 간격으로 큰 음(-)의 값과 양(+)의 값이 반복

□ 원인: 계절조정하지 않은(NSA) Retail Sales 자료를 사용했기 때문

o 1차 차분으로 추세에 따른 비정상성은 크게 제거되었지만, 계절적 패턴은 남아 있음

◇ 계절 차분:

① 월별 자료이므로 $\Delta_{12} y_t = y_t - y_{t-12}$

retail["Seasonal_Diff"] = retail["Retail_Sales"].diff(12)

② 또는 로그를 취한 후 계절 차분:

retail["Log_Seasonal_Diff"] = (np.log(retail["Retail_Sales"]).diff(12))

⇒ 계절 차분과 계절성을 고려한 모형은 뒤의 SARIMA에서 다룰 예정

8.2.6 자기상관함수 (autocorrelation function, ACF)

◆ 자기상관이란?

□ 상관계수는 두 변수 X와 Y 사이의 선형적 관계의 방향과 강도를 나타냄

$$\rho_{X,Y} = \frac{Cov(X,Y)}{\sqrt{Var(X)\,Var(Y)}}$$

- o 상관계수($\rho_{X,Y}$)의 값은 -1과 1 사이에 있음
- o 1에 가까울수록 강한 양(+)의 선형관계, -1에 가까울수록 강한 음(-)의 선형관계
- o 0에 가까울수록 선형관계가 약함

□ 시계열에서는 서로 다른 두 변수가 아니라 같은 변수의 현재 값과 과거 값 사이의 상관관계에 관심

- o 예: $Corr(y_t, y_{t-2})$는 현재 값과 2기간 전 값의 관계를 의미 → 자기상관(autocorrelation)

◆ 자기상관함수(ACF)

□ 자기상관함수는 시차가 달라짐에 따라 자기상관계수가 어떻게 변하는지를 나타내는 함수

□ 정상적인 시계열에서 시차 k의 자기상관계수는

$$\rho_k = Corr(y_t, y_{t-k}) = \frac{Cov(y_t, y_{t-k})}{\sqrt{Var(y_t)\,Var(y_t)}} = \frac{Cov(y_t, y_{t-k})}{Var(y_t)} = \frac{\gamma_k}{\gamma_0}$$

- $\gamma_k = Cov(y_t, y_{t-k})$는 시차 k 에서의 자기공분산
- $\gamma_0 = Var(y_t)$는 시계열의 분산

□ 예: 자기상관계수는 시차가 1인 경우 $\rho_1 = Corr(y_t, y_{t-1})$,

시차가 2인 경우 $\rho_2 = Corr(y_t, y_{t-2})$

◆ ACF 그래프는 어떻게 읽는가?

□ ACF 그래프(correlogram)는 여러 시차에서 계산한 자기상관계수를 한꺼번에 보여줌

- o 가로축 : 시차(lag)
- o 세로축 : 자기상관계수
- o 막대: 각 시차에서의 자기상관계수
- o 음영 영역: 자기상관계수가 0인지 판단하기 위한 신뢰구간(confidence interval)

□ 시차가 0이면 현재 값과 자기 자신 사이의 상관관계이므로

$$\rho_0 = Corr(y_t, y_t) = 1$$

따라서 lag = 0의 자기상관계수는 항상 1

◆ 음영 영역은 어떻게 해석하는가?

□ ACF 그래프의 음영 영역은 자기상관이 0이라는 가정 아래에서 각 시차의 자기상관계수가 통계적으로 유의한지를 판단하기 위한 기준

□ 백색잡음이고 표본크기 N이 충분히 큰 경우, 95% 신뢰구간은 대략 아래와 같은 형태로 이해할 수 있음

$$-\frac{1.96}{\sqrt{N}} \leq \hat{\rho}_k \leq \frac{1.96}{\sqrt{N}}$$

o 막대가 음영 영역 안 → 자기상관계수가 0과 유의하게 다르다는 충분한 증거가 없음

o 막대가 음영 영역 밖 → 해당 시차에서 통계적으로 유의한 자기상관이 존재함을 시사

⇒ ACF 그래프에서는 막대의 높이뿐 아니라 음영 영역을 벗어나는지도 함께 확인할 필요

◆ ACF 그래프의 대표적인 형태

① 백색잡음

□ 서로 다른 시점의 충격 사이에 상관관계가 없으므로 $\rho_k = 0,\ k \geq 1$

→ lag = 0을 제외하면 자기상관계수가 대부분 0에 나타나며, 대체로 신뢰구간 안에 위치

② 정상 시계열

□ 자기상관이 존재하더라도 일반적으로 시차가 증가하면서 비교적 빠르게 감소

→ 과거 값의 영향이 시간이 지나면서 약화

③ 비정상 시계열

□ 여러 시차에서 높은 자기상관이 나타나고 천천히 감소

→ 시계열의 높은 지속성(persistence)을 시사

④ 계절성이 있는 시계열

□ 일정 시차마다 높은 자기상관이 반복적으로 나타남

o 예: 월별 자료에 계절성이 있다면, $lag = 12, 24, 36, \cdots$에서 반복적인 패턴이 나타남

8.2.7 실습(Lab): 확률보행과 ACF

(1) 확률보행의 ACF

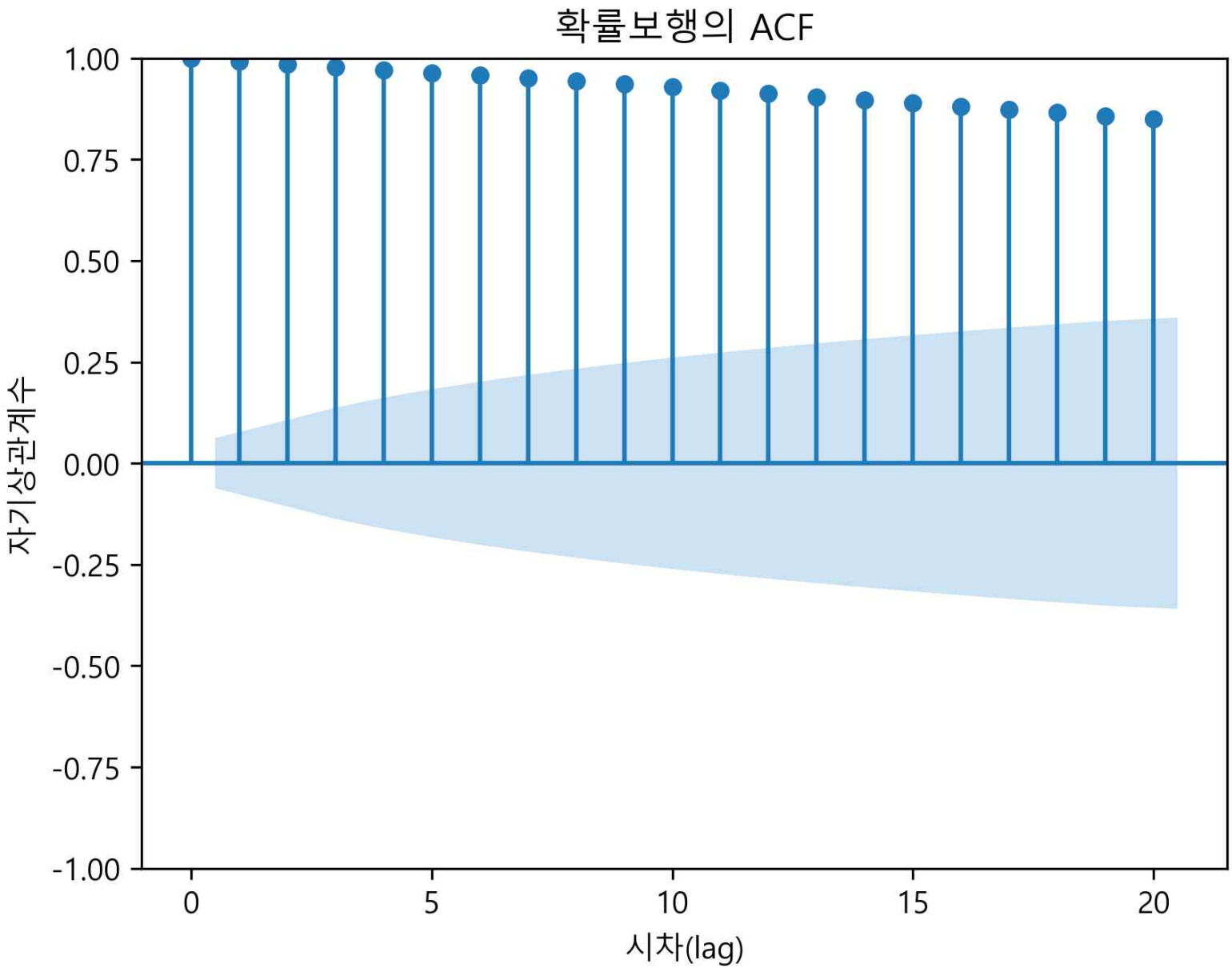
□ ACF를 그래프로 표현하는 함수가 있는 statsmodels 라이브러리를 사용

- o x축의 범위로서 lag=20을 선택
- o 데이터는 앞에서 생성한 random_walk을 사용

```
from statsmodels.graphics.tsaplots import plot_acf
import matplotlib.pyplot as plt

plot_acf(random_walk, lags=20)
plt.title("확률보행의 ACF")
plt.xlabel("시차(lag)")

plt.show()
```


확률보행의 ACF
1.00
0.75
0.50
0.25
0.00
-0.25
-0.50
-0.75
-1.00
자기상관계수
0
5
10
15
20
시차(lag)

◆ 확률보행의 ACF 결과

□ 확률보행의 ACF에서는 lag=0에서 자기 자신과의 상관관계이므로 자기상관계수는 항상 1

□ lag가 증가해도 자기상관계수가 높은 수준을 유지하면서 매우 천천히 감소

- o lag 1에서는 거의 1에 가까운 강한 양(+)의 자기상관
- o lag 20에서도 약 0.85 수준의 높은 자기상관이 유지
- o 대부분의 자기상관계수가 신뢰구간을 크게 벗어남

□ 이는 확률보행에서 한 번 발생한 충격의 효과가 소멸하지 않고 이후의 시계열 수준에 계속 남아 있기 때문

⇒ ACF가 여러 시차에서 높게 나타나고 매우 천천히 감소하는 것은 비정상 시계열 가능성을 시사

◆ <참고> ACF 그래프의 신뢰구간

□ 백색잡음이라는 가정 아래 표본크기 N이 충분히 크면, 자기상관계수의 약 95% 신뢰구간은 대략 다음과 같이 나타낼 수 있음

$$-\frac{1.96}{\sqrt{N}} \leq \hat{\rho}_k \leq \frac{1.96}{\sqrt{N}}$$

□ 따라서

▸ 막대가 음영 영역 안 → 자기상관이 0과 유의하게 다르다는 충분한 증거 없음

▸ 막대가 음영 영역 밖 → 해당 시차에서 통계적으로 유의한 자기상관이 존재할 가능성

⇒ ACF 그래프에서 음영 영역 밖으로 나온 막대에 주목

<참고> ACF 신뢰구간의 도출

□ ACF 그래프의 신뢰구간은 시계열이 백색잡음이라는 가정 아래에서 도출할 수 있음

□ 백색잡음에서는 서로 다른 시점 사이에 자기상관이 존재하지 않으므로, 시차 k > 0에 대해 이론적인 자기상관계수는 $\rho_k = 0$

□ 이는 ACF 분석에서 자기상관이 0이라는 귀무가설에 해당

$$H_0 : \rho_k = 0 \qquad (k > 0)$$

❶ 표본 자기상관계수의 분포

□ 실제 표본에서 계산한 자기상관계수 $\hat{\rho}_k$는 표본에 따라 0에서 조금씩 벗어날 수 있음

□ 백색잡음을 가정하고 표본크기 N이 충분히 크면, 표본 자기상관계수는 근사적으로 다음의 분포를 따름

$$\hat{\rho}_k \approx \mathbb{N}\left(0, \frac{1}{N}\right)$$

o $\mathbb{N}$: 정규분포, 첫 번째 값 0: 평균, 두 번째 값 1/N: 분산

□ 따라서 정규분포 $X \sim \mathbb{N}(\mu, \sigma^2)$에서 μ는 평균, σ^2은 분산을 의미하므로

$$E(\hat{\rho}_k) \approx 0, \quad Var(\hat{\rho}_k) \approx \frac{1}{N}$$

□ 따라서 표본 자기상관계수의 표본분포의 표준편차인 표준오차(standard error)는

$$SE(\hat{\rho}_k) = \sqrt{Var(\hat{\rho}_k)} = \sqrt{\frac{1}{N}} = \frac{1}{\sqrt{N}}$$

❷ 표준정규분포로 변환

□ 평균이 μ, 표준편차가 σ인 확률변수 X를 표준정규분포로 바꾸려면 다음과 같이 표준화

$$Z = \frac{X - \mu}{\sigma}$$

□ 여기서 X = $\hat{\rho}_k$, 평균 $\mu \approx 0$, 표준편차 $\sigma \approx 1/\sqrt{N}$ 이므로

$$Z \approx \frac{\hat{\rho}_k - 0}{1/\sqrt{N}} \approx (\hat{\rho}_k - 0)\sqrt{N} \approx \sqrt{N}\,\hat{\rho}_k$$

□ 따라서 $\sqrt{N}\hat{\rho}_k \overset{a}{\sim} \mathbb{N}(0,1)$

⇒ 표준화한 표본 자기상관계수는 근사적으로 표준정규분포를 따름을 뜻함

❸ 왜 분산이 1이 되는가?

□ 확률변수 X에 상수 a를 곱하면 $Var(aX) = a^2\, Var(X)$ 이므로

$$Var(\sqrt{N}\hat{\rho}_k) = (\sqrt{N})^2\, Var(\hat{\rho}_k) \approx N\, Var(\hat{\rho}_k)$$

□ 그런데 큰 표본에서 $Var(\hat{\rho}_k) \approx \frac{1}{N}$ 이므로

$$Var(\sqrt{N}\hat{\rho}_k) \approx N\left(\frac{1}{N}\right) = 1$$

□ 따라서 표준화된 표본 자기상관계수는 근사적으로 평균 0, 분산 1인 표준정규분포를 따름

$$\sqrt{N}\hat{\rho}_k \overset{a}{\sim} \mathbb{N}(0,1)$$

❹ 95% 범위의 도출

□ 표준정규분포에서는 약 95%의 값이 -1.96 ≤ Z ≤ 1.96 사이에 존재

□ $Z \approx \sqrt{N}\,\hat{\rho}_k$ 이므로 $-1.96 \le \sqrt{N}\hat{\rho}_k \le 1.96$

□ 각 항을 $\sqrt{N}$으로 나누면 $-\frac{1.96}{\sqrt{N}} \leq \hat{\rho}_k \leq \frac{1.96}{\sqrt{N}}$

⇒ 백색잡음을 가정했을 때 ACF에서 자기상관의 통계적 유의성을 판단하기 위한 대략적인 95% 범위

❺ ACF 그래프에서의 의미

□ 특정 시차의 $\hat{\rho}_k$가 95% 신뢰구간 안에 위치하면, $-\frac{1.96}{\sqrt{N}} \leq \hat{\rho}_k \leq \frac{1.96}{\sqrt{N}}$

→ 자기상관이 0이라는 귀무가설($H_0 : \rho_k = 0$)을 기각하지 못함

→ 해당 시차에서 자기상관계수가 0과 통계적으로 유의하게 다르다는 충분한 증거가 없음

□ 특정 시차의 $\hat{\rho}_k$가 95% 신뢰구간 밖에 위치하면, $H_0 : \rho_k = 0$을 5% 유의수준에서 기각

→ 해당 시차에서 통계적으로 유의한 자기상관이 존재한다고 판단

(2) 정상화를 위한 변환: 차분

□ 확률 보행이 비정상 시계열이므로, 정상성을 확보하기 위해 1차 차분을 적용

□ $y_t = y_{t-1} + \epsilon_t$의 양변에서 y_{t-1}을 빼면, $\triangle y_t = y_t - y_{t-1} = \epsilon_t$

o 확률보행을 1차 차분하면 백색잡음이 됨을 뜻함

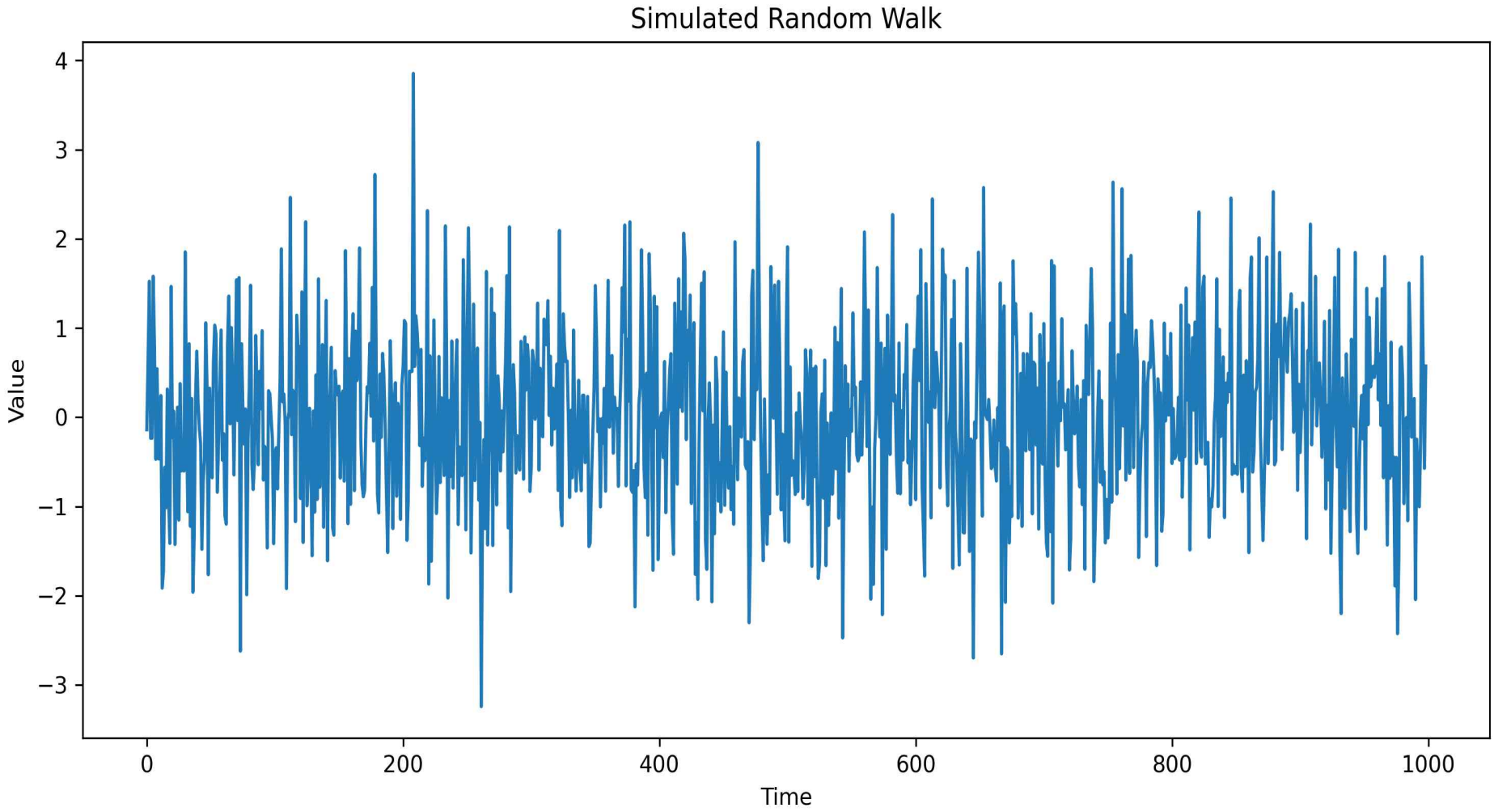
□ Python에서는 numpy 라이브러리의 메소드인 np.diff()를 이용하여 차분

```
diff_random_walk = np.diff(random_walk, n=1)        # 1차 차분
```

o n=1 : 1차 차분

o 1차 차분에서는 인접한 두 관측값의 차이를 계산하므로 관측값의 개수가 하나 감소

□ 1차 차분 결과에서는 확률보행에서 나타났던 비정상적인 움직임이 사라지고, 평균과 분산이 시간에 따라 안정적인 형태를 보임



(3) 차분 후 정상성 재검정: ADF 테스트

□ 차분을 통해 정상성이 확보되었는지를 확인하기 위해 차분된 시계열에 대해 다시 ADF 검정을 수행

```
from statsmodels.tsa.stattools import adfuller

ADF_result = adfuller(diff_random_walk)

print(f'ADF Statistic: {ADF_result[0]}')
print(f'p-value: {ADF_result[1]}')
```

```
ADF Statistic: -31.789310857560622
p-value: 0.0
```

□ p-값(0.0)이 0.05보다 작으므로 단위근이 존재한다는 귀무가설을 기각

→ 차분된 시계열은 정상성이 있는 것으로 판단

(4) 차분 후 시계열의 ACF

□ 확률보행은 단위근을 갖는 비정상 시계열 : $y_t = y_{t-1} + \epsilon_t$

□ 확률보행을 1차 차분하면 $\triangle y_t = y_t - y_{t-1} = \epsilon_t$가 되므로, ϵ_t가 백색잡음이면

차분된 시계열은 평균과 분산이 일정하고, 시차 $k > 0$에서 자기상관이 0. 즉,

$$\rho_k = 0 \qquad\qquad (k > 0)$$

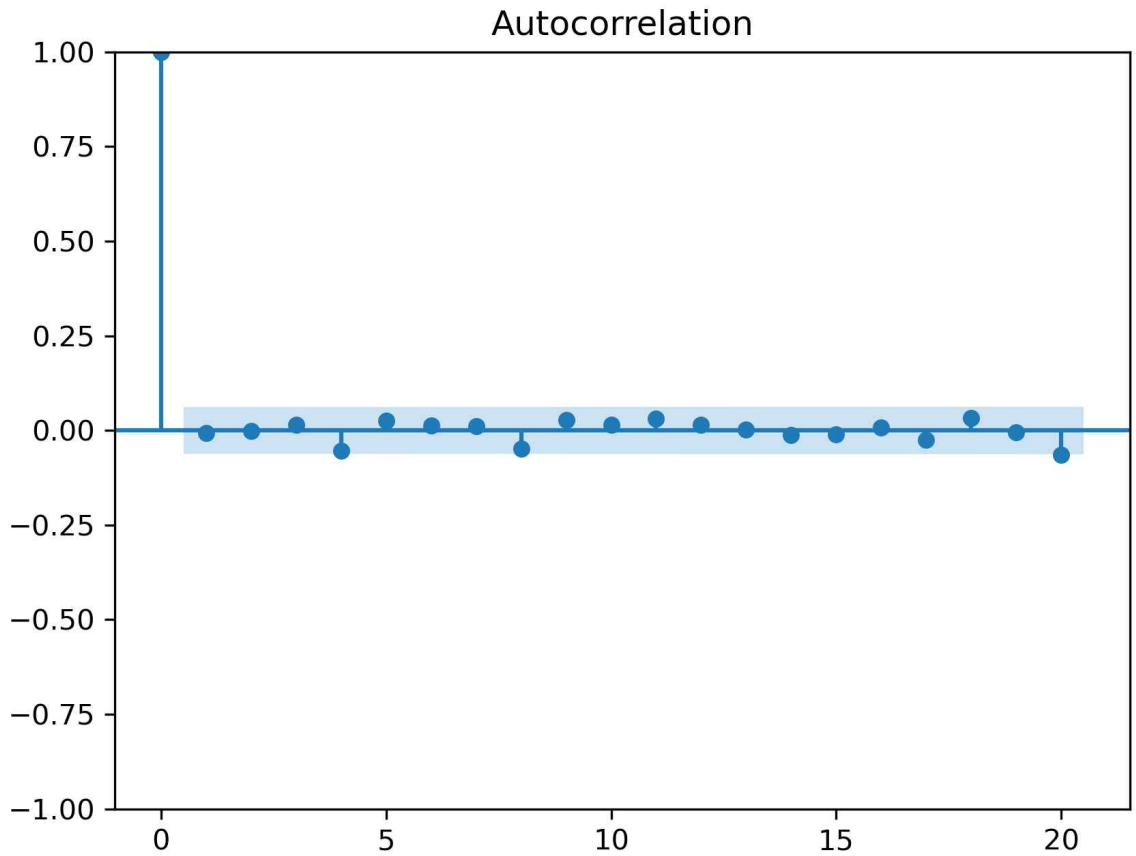
□ 따라서 ACF 그래프에서는

- 시차 0에서 자기상관계수는 1
- 시차 0 이후에는 자기상관계수가 대부분 0에 근접
- 대부분의 자기상관계수가 95% 신뢰구간 안에 위치

→ 차분된 시계열이 백색잡음의 특성을 보이는 것을 확인할 수 있음

◇ 정상성이 확보된 차분 시계열에 대해 ACF를 확인

```
plot_acf(diff_random_walk, lags=20)
plt.show()
```



8.2.8 Apple 주가는 확률보행일까?

◆ 실제 주가 데이터의 활용

□ 앞에서는 모의실험을 통해 확률보행의 특징을 확인

□ 이번에는 실제 주가 데이터로서 Apple 주가가 확률보행의 특징을 보이는지 분석

(1) Apple 주가 데이터 불러오기

□ yfinance를 사용하기 위해 주피터 노트북에 다음과 같이 설치

```
%pip install yfinance
```

□ Apple의 주식 티커(ticker) 확인

- o Yahoo Finance에서 Apple 검색 → Apple Inc. (AAPL)
- o AAPL은 Nasdaq에서 거래되는 Apple 주식의 종목 기호

□ Apple의 일별 주가를 불러옴

```
import yfinance as yf
import pandas as pd
import matplotlib.pyplot as plt

apple = yf.download( “AAPL“, start=“2016-01-01“, end=“2026-01-01“, auto_adjust=False )

apple.head()
```

- start, end: 분석 기간
- auto_adjust=False: 원래 종가(Close)와 수정주가(Adj Close)를 구분하여 확인하기 위한 설정
  - ▸ Close: 해당 거래일의 장 마감 시점에 형성된 종가(closing price)
  - ▸ Adj Close: 주식분할, 배당 등의 영향을 반영하여 조정한 가격
  - ▸ Volume: 해당 거래일의 거래량
- 본 분석에서는 Apple의 일별 종가를 이용하여 주가가 확률보행의 특성을 보이는지 확인

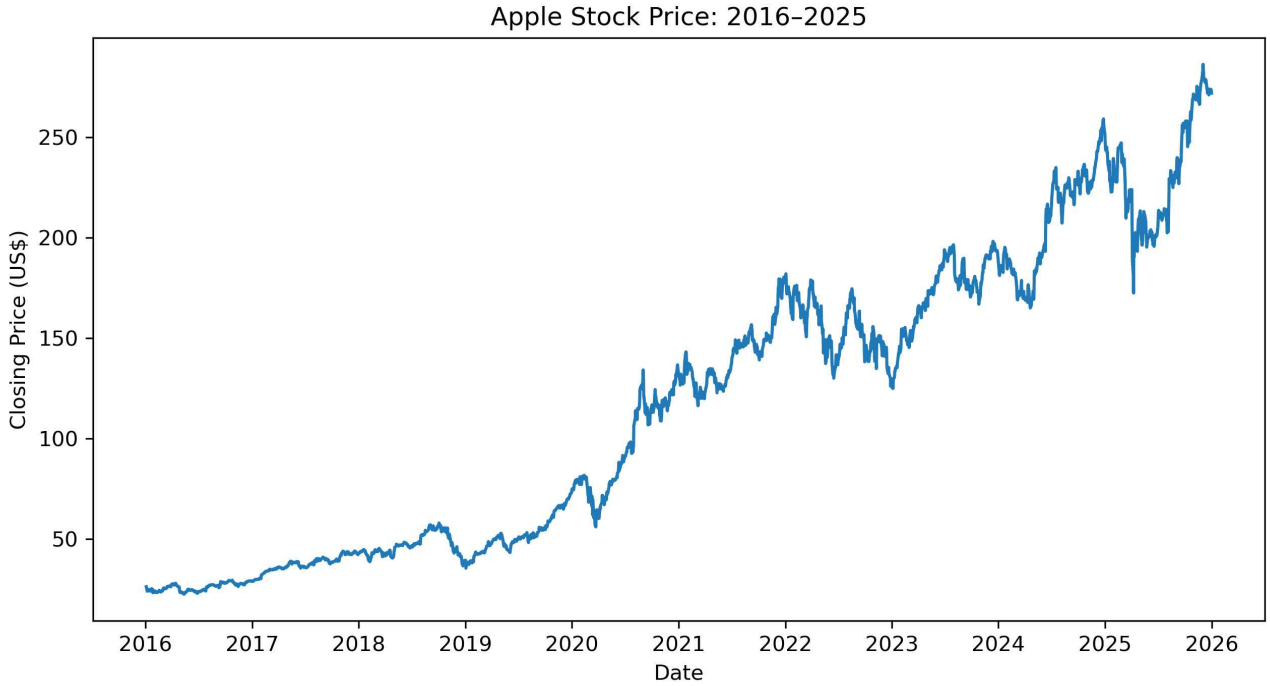
(2) Apple 종가 시각화

```
apple_close = apple["Close"]

plt.figure(figsize=(10, 5))
plt.plot(apple_close)

plt.xlabel("Date")
plt.ylabel("Closing Price (US$)")
plt.title("Apple Stock Price: 2016-2025")

plt.show()
```



◆ Apple 종가의 시계열적 특징

□ 2016~2025년 Apple의 일별 종가 수준이 장기적으로 크게 상승하는 가운데 시간에 따라 큰 폭의 상승과 하락을 반복

□ 이러한 모습은 Apple 종가가 비정상 시계열일 가능성을 시사

→ 그러나 그래프만으로 정상성 여부를 확정할 수 없으므로 ADF 검정을 통해 확인

(3) Apple 종가의 정상성 검정: ADF

□ Apple 종가가 정상적인 시계열인지 확인하기 위해 ADF 검정을 실시

```
from statsmodels.tsa.stattools import adfuller

ADF_result = adfuller(apple_close.dropna())

print(f“ADF Statistic: {ADF_result[0]:.4f}“)
print(f“p-value: {ADF_result[1]:.4f}“)
```

Q. ADF 검정 결과를 이용하여 Apple 종가가 정상 시계열인지 판단해 보세요.

◇ 검정 결과

ADF Statistic: 0.3194

p-value: 0.9782

◇ 결과의 해석

□ ADF 검정의 귀무가설은 단위근이 존재한다 → H_0: 단위근이 존재

□ p-값이 0.9782로 0.05를 크게 상회하므로 귀무가설을 기각하지 못함

→ Apple 종가에서 단위근의 존재를 배제할 수 없음

→ 따라서 Apple 종가는 비정상 시계열로 판단

(4) Apple 종가의 1차 차분

□ Apple 종가가 비정상적이라면, 정상성 확보를 위해 1차 차분을 적용

```
import numpy as np

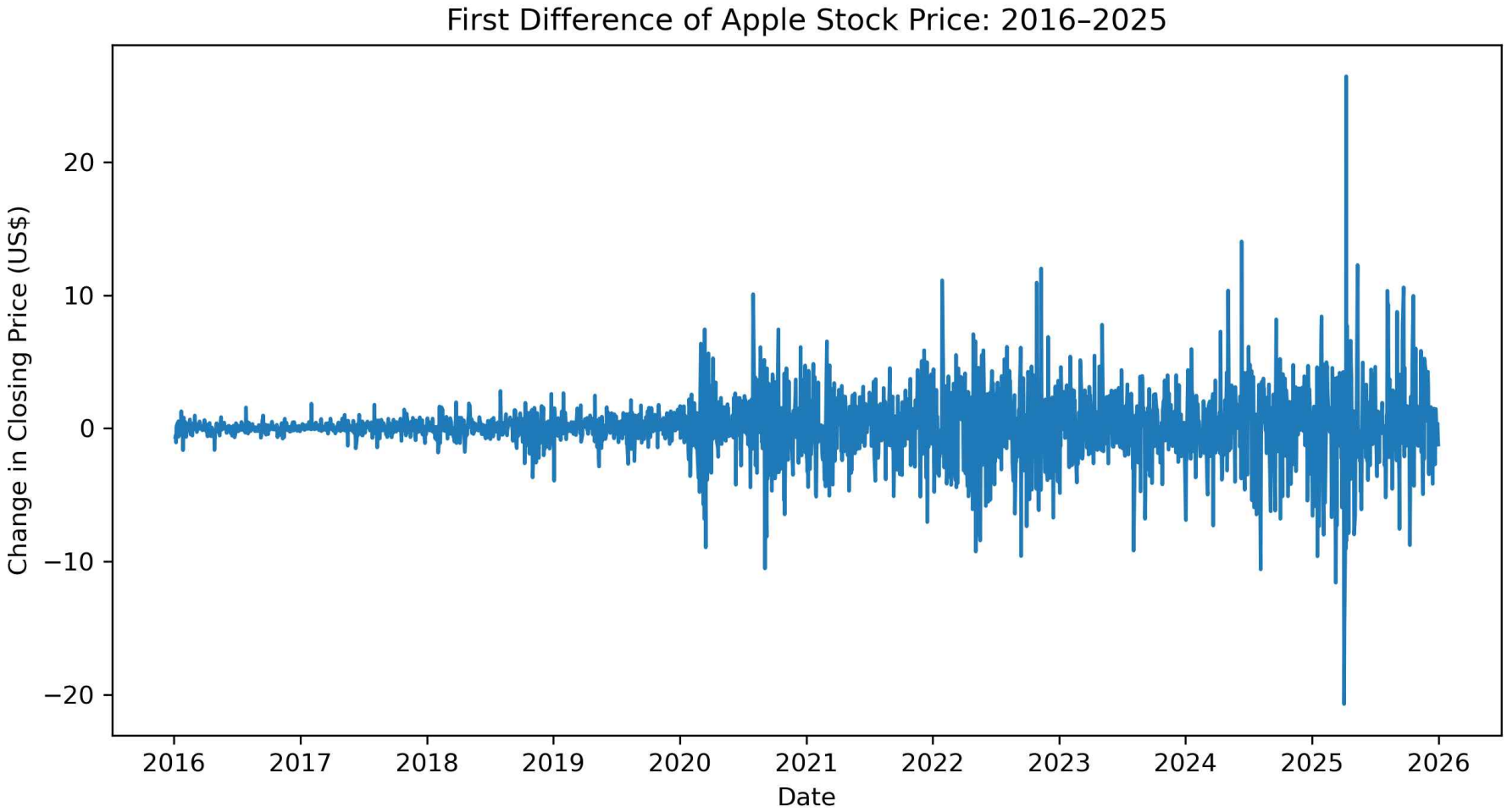
diff_apple = np.diff(apple_close.squeeze(), n=1)
```

□ 차분된 자료의 시각화

```
plt.figure(figsize=(10, 5))
plt.plot(apple_close.index[1:], diff_apple)

plt.xlabel("Date")
plt.ylabel("Change in Closing Price (US$)")
plt.title("First Difference of Apple Stock Price: 2016-2025")

plt.show()
```



- squeeze()를 넣은 이유는 yfinance 버전에 따라 apple["Close"]가 (N, 1) 형태의 DataFrame으로 반환될 수 있기 때문. squeeze()는 이를 1차원 Series로 바꾸어 줌

Q. 그래프에서 차분 후에 정상 시계열에 가까워진 것으로 나타나는지?

□ Apple 종가에 1차 차분을 적용하여 일별 주가 변화량을 계산: $\Delta P_t = P_t - P_{t-1}$

□ 차분 전 종가는 뚜렷하게 장기적인 상승 움직임을 보였으나, 1차 차분 후에는 0을 중심으로 양(+)과 음(-)의 값이 반복

→ 종가 수준에서 나타났던 장기적인 상승 움직임이 제거됨

□ 그러나 그래프를 보면 최근으로 갈수록 주가 변화의 폭이 커지고, 일부 기간에는 큰 폭의 변동이 집중되어 있음

→ 평균은 비교적 안정되어 보이지만 분산이 일정하다고 그래프만으로 판단하기 어려움

◆ 차분 후 정상성 재검정: ADF 검정

```
ADF_diff = adfuller(diff_apple)

print(f"ADF Statistic: {ADF_diff[0]:.4f}")
print(f"p-value: {ADF_diff[1]:.4f}")
```

```
ADF Statistic: -17.4056
p-value: 0.0000
```

□ p-value가 0.05보다 매우 작으므로 H_0: 단위근이 존재 라는 귀무가설을 기각

→ Apple 종가의 1차 차분값은 단위근을 갖지 않는 것으로 판단

□ 따라서 1차 차분을 통해 종가 수준의 비정상성이 제거된 것으로 판단

Q. ADF 검정에서는 정상성이 확인되었는데, 그래프를 보면 변동 폭이 시간에 따라 달라지는 것처럼 보임. 무슨 문제가 있는 것일까?

□ ADF 검정은 단위근의 존재 여부를 검정.

ADF 귀무가설을 기각한 것이 시계열의 분산도 일정하다는 것을 보장하지는 않음

□ 확률보행이라면 $P_t = P_{t-1} + \epsilon_t$ 이므로 $\Delta P_t = \epsilon_t$

□ 따라서 Apple 종가가 확률보행에 가깝다면 차분된 Apple 종가에는 뚜렷한 자기상관이 없어야 함

◆ ACF 그래프

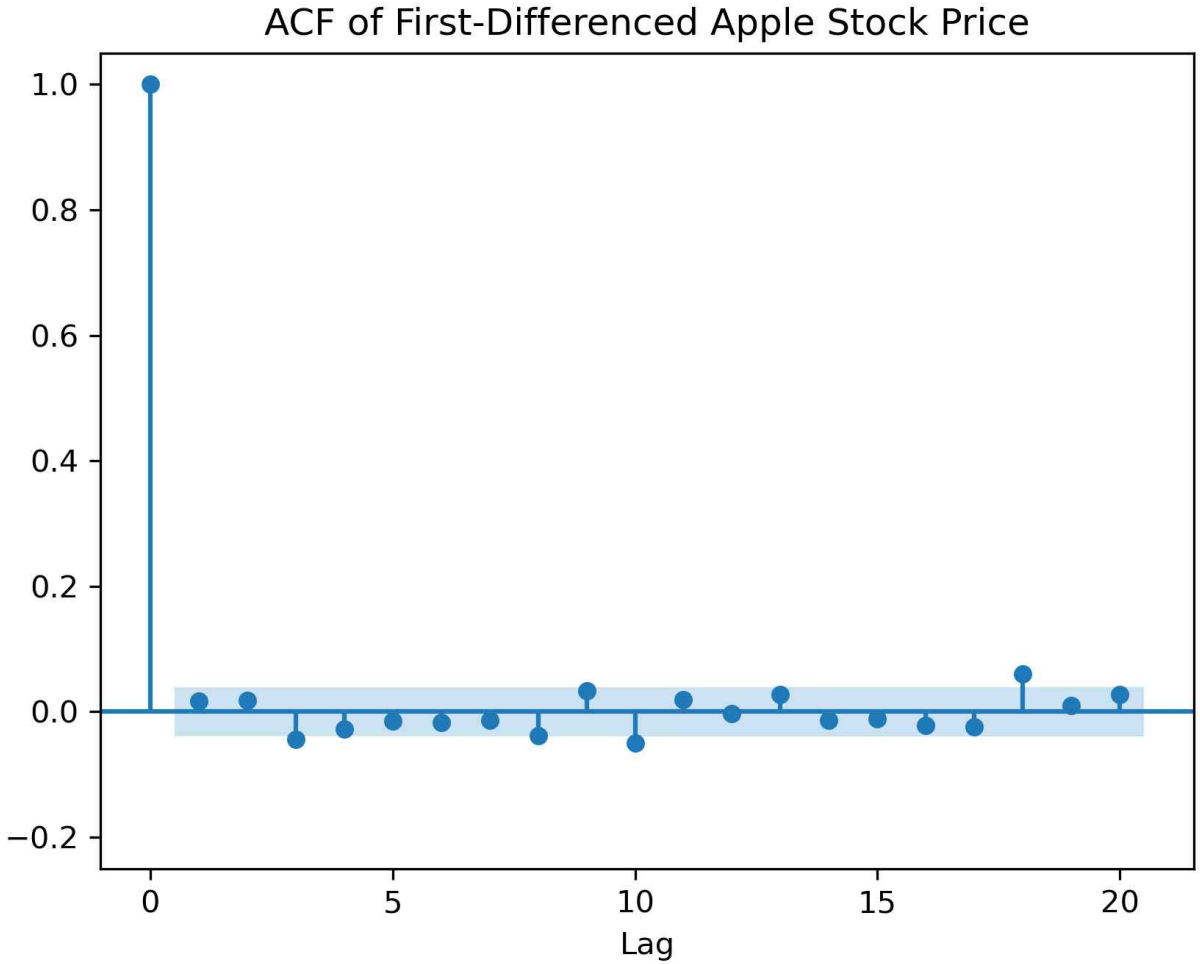
```
from statsmodels.graphics.tsaplots import plot_acf
import matplotlib.pyplot as plt

plot_acf(diff_apple, lags=20)

plt.xlabel("Lag")
plt.title("ACF of First-Differenced Apple Stock Price")

# y축 범위 조정
plt.ylim(-0.25, 1.05)

plt.show()
```


ACF of First-Differenced Apple Stock Price
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
0
5
10
15
20
Lag

◆ ACF 결과의 해석

□ Lag 0에서는 자기 자신과의 상관관계이므로 $\rho_0 = 1$

□ Lag 1 이후의 자기상관계수는 대부분 0에 매우 가까운 값을 보임

□ 대부분의 시차에서 자기상관계수가 95% 신뢰구간 안에 위치

□ 다만 일부 시차에서는 신뢰구간을 벗어나는 작은 자기상관이 관찰됨

→ 따라서 Apple 종가의 일별 변화량에는 전반적으로 강한 선형 자기상관 구조가 나타나지 않음

◆ Apple 주가는 확률보행일까?

□ 분석 결과

- Apple 종가 : 비정상적
- 1차 차분 → 종가 변화량 : 단위근이 없는 것으로 판단
- ACF : 대부분의 시차에서 자기상관이 매우 작음

□ 이는 $P_t = P_{t-1} + \epsilon_t$ 와 같은 확률보행 모형에서 예상되는 주요 특징과 대체로 부합

⇒ Apple 종가는 확률보행에 가까운 특성을 보임

Q. 과거 10년간 Apple 주가가 크게 상승했다는 사실은, 내일의 주가 상승도 예측할 수 있다는 의미일까?

A. 그렇지 않다.

□ Apple 주가가 장기적으로 상승했다는 것은 주가 수준의 장기적인 움직임에 관한 것

□ 반면 ACF는 $Corr(\Delta P_t, \Delta P_{t-k})$를 통해 과거의 주가 변화와 현재의 주가 변화 사이에 선형적인 관계가 있는지를 확인

□ 분석 결과 대부분의 시차에서 자기상관이 0에 가까움

→ 과거의 일별 주가 변화만으로 다음 주가 변화를 예측할 수 있는 뚜렷한 선형적 관계가 거의 나타나지 않음

□ 따라서 장기적인 주가 상승 $\neq$ 일별 주가 변화의 예측 가능성

8.3 확률보행 예측

◆ 학습 목표

□ 확률보행의 특성을 이용하여 미래값을 예측하는 방법을 이해

□ 단순한 예측방법을 비교하여 장기 예측과 단기 예측의 차이를 이해

8.3.1 장기간의 확률보행 예측

◆ 확률보행의 장기 예측

□ 확률보행은 $y_t = y_{t-1} + \epsilon_t$로 표현되며 미래의 무작위 충격 ϵ_t은 사전에 예측할 수 없음

□ 따라서 예측기간이 길어질수록 미래 충격이 누적되어 예측의 불확실성이 증가

$$y_{T+h} = y_T + \epsilon_{T+1} + \epsilon_{T+2} + \cdots + \epsilon_{T+h}$$

□ 이러한 특성을 확인하고자 앞서 생성한 확률보행 자료에 세 가지 단순 예측방법을 적용

(1) 데이터 분할

□ 앞 절에서 생성한 1,000개의 모의 확률보행 데이터를 다시 이용하여 예측 실험을 수행

□ 확률보행이라는 것을 이미 알고 있는 자료를 이용함으로써, 확률보행의 예측 특성과 한계를 명확하게 살펴봄

□ 전체 1,000개 관측값 중 처음 800개를 훈련집합, 나머지 200개를 테스트집합으로 사용

□ 시계열에서는 시간의 순서가 중요하므로 과거 자료를 훈련에 사용하고 이후 자료를 테스트에 사용

```
import pandas as pd

df = pd.DataFrame({"value": random_walk})

train = df.iloc[:800]

test = df.iloc[800:].copy()
```

□ 코드 설명

■ train = df.iloc[:800]

▸ 전체 데이터 df에서 처음 800개 관측값은 훈련 데이터로 사용

▸ iloc은 행의 위치를 기준으로 데이터를 선택

■ test = df.iloc[800:].copy()

▸ 800번째 인덱스부터 마지막까지 관측값은 테스트 데이터 test로 저장

→ 전체 1,000개 관측값의 위치가 0 ~ 999라면, train : 0～799, test : 800～999

■ .copy()

▸ 선택한 데이터를 별도의 DataFrame으로 복사하여 이후 예측값 열을 안전하게 추가

예: test[“pred_mean“] = mean_value

(2) 세 가지 단순 예측방법

1 평균 예측 모형(mean forecast)

□ 훈련 데이터의 평균을 계산해 테스트 구간의 모든 미래값을 동일한 값으로 예측

$$\hat{y}_{T+h} = \bar{y}_{train}$$

```
mean_value = train["value"].mean()
test["pred_mean"] = mean_value
test.head()
```

□ 모든 미래 시점에 같은 값을 사용하므로 예측선은 수평선

□ 다른 예측모형의 성능을 비교하기 위한 단순한 기준모형(baseline model)으로 활용

2 마지막 값 예측(naive forecast)

□ 훈련 데이터의 마지막 관측값을 모든 미래 시점의 예측값으로 사용

$$\hat{y}_{t+h} = y_t$$

```
last_value = train["value"].iloc[-1]
test["pred_last"] = last_value
test.head()
```

□ 확률보행에서는 미래값은 현재값에 새로운 무작위 충격이 더해져 결정

$$y_{t+1} = y_t + \epsilon_{t+1}$$

□ 미래 충격 ϵ_{t+1}은 미리 알 수 없고, 양수나 음수가 될 수 있으므로 평균이 0이라고 가정

$$E(\epsilon_{t+1}) = 0$$

□ 따라서 다음 시점의 기대값은 현재값과 같음

$$
\begin{aligned}
E(y_{t+1}|y_t) &= E(y_t + \epsilon_{t+1}|y_t) \\
&= E(y_t|y_t) + E(\epsilon_{t+1}|y_t) \\
&= y_t + 0 \\
&= y_t
\end{aligned}
$$

여기서 $\underbrace{E(y_t|y_t)}_{\text{현재값은 알고 있음}} = y_t, \quad \underbrace{E(\epsilon_{t+1}|y_t)}_{\text{미래 충격은 평균적으로 0}} = 0$

□ 그러므로 $\hat{y}_{t+1}|t = y_t$ → 확률보행에서는 현재 관측값이 다음 시점의 최선 예측값

□ 그러므로 확률보행에서는 최근 관측값을 다음 시점의 예측값으로 사용

<참고>

▸ $E(y_{t+1}|y_t)$는 t 시점의 y_t를 알고 있을 때 t+1 시점의 y_{t+1}에 대해 평균적으로 예상되는 값

▸ E(A + B) = E(A) + E(B)

3 표류 모형 (drift method)

□ 훈련 데이터의 전체적인 상승·하락 기울기(drift)를 계산하고, 이렇게 계된한 기울기를 관측되지 않은 미래 구간까지 직선으로 연장, 즉 외삽(extrapolation)하여 예측하는 방법

□ 전체적인 상승·하락 기울기는 훈련 데이터의 첫 번째 값과 마지막 값 사이의 변화를 이용하여 계산

$$d(\text{기울기}) = \frac{\text{변화량}}{\text{구간수}} = \frac{y_T - y_0}{T-1}$$

o y_0: 시작값, y_T: 마지막 값, T: 관측치 개수

□ h 시점 후 예측값은 마지막 관측값 y_T에서 출발하여 기울기 d만큼 계속 증가하거나 감소

$$\hat{y}_{T+h} = y_T + hd$$

따라서

$$\hat{y}_{T+h} = y_T + h\left(\frac{y_T - y_1}{T-1}\right)$$

o 기울기를 계산할 때 T가 아니라 T-1을 사용하는 이유는, 변화는 관측치 개수가 아니라 관측치 사이의 구간(interval)에서 발생하기 때문

▸ 예: 데이터 개수 5개: y_0, y_1, y_2, y_3, y_4, 구간(간격) 4개(T-1): $y_0 \rightarrow y_1 \rightarrow y_2 \rightarrow y_3 \rightarrow y_4$

◆ 표류 모형 구현 코드

```
first_value = train["value"].iloc[0]
last_value = train["value"].iloc[-1]

T = len(train)

drift = (last_value - first_value) / (T - 1)

h = np.arange(1, len(test) + 1)

test["pred_drift"] = last_value + drift * h
```

(3) 세 가지 예측방법의 비교

```
fig, ax = plt.subplots(figsize=(10, 6))

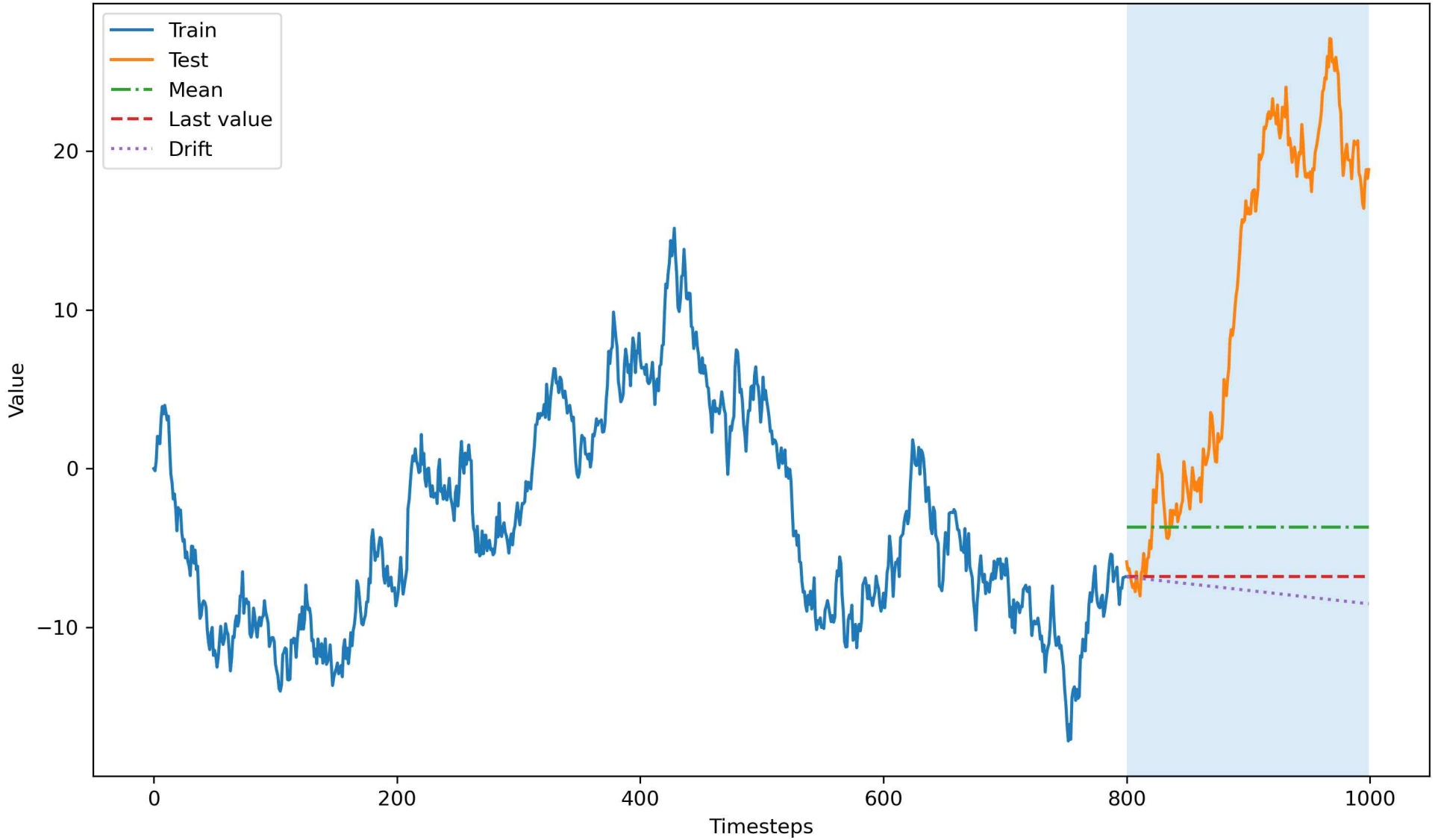
ax.plot(train["value"], label="Train")
ax.plot(test["value"], label="Test")

ax.plot(test["pred_mean"], linestyle="-.", label="Mean")
ax.plot(test["pred_last"], linestyle="--", label="Last value")
ax.plot(test["pred_drift"], linestyle=":", label="Drift")

ax.axvspan(800, 999, alpha=0.2)

ax.set_xlabel("Timesteps")
ax.set_ylabel("Value")
ax.legend()

plt.tight_layout()
plt.show()
```


Train
Test
Mean
Last value
Drift
20
10
0
−10
Value
0
200
400
600
800
1000
Timesteps

◇ 코드 설명

□ figsize=(10, 6): 그래프의 크기를 가로 10인치, 세로 6인치로 설정

□ ax.plot(test[“ “], linestyle=“ “, label=“ “)에서 linestyle은 선의 모양을 지정

코드	선 모양	표시하는 값
“-.“	일점쇄선	평균 예측
“--“	파선	마지막 값 예측
“:“	점선	표류 예측

□ label=“Mean“은 범례에 이 선의 이름을 Mean으로 표시하라는 의미

□ 이후에 ax.legend()를 실행하면 세 개의 label을 이용해 범례가 자동으로 생성됨

□ ax.legend()는 Matplotlib이 그래프와 덜 겹치는 범례 위치를 자동 선택

o 위치 지정: upper left, upper right, lower left, lower right, center 등.

▸ 예: ax.legend(loc=“upper left“)

□ axvspan()은 그래프에서 x축의 일정한 구간에 세로 방향으로 음영을 넣는 함수

◆ 그래프의 해석

□ 세 가지 방법 모두 테스트 구간에서 실제 확률보행의 움직임을 정확하게 따라가지 못함

□ 평균 예측과 마지막 값 예측은 일정한 값을 예측하므로 수평선으로 나타남

□ 표류 예측은 과거의 평균 변화 방향을 연장하므로 직선으로 나타남

□ 반면 실제 확률보행은 테스트 기간에 새롭게 발생하는 무작위 충격에 따라 움직임

⇒ 과거의 평균이나 추세만으로 미래의 무작위 충격을 예측할 수 없음

(4) 예측 오차 평가

□ 세 모형의 예측 성능을 평균제곱오차(mean squared error, MSE)로 비교

o 예측오차 = 실제값 - 예측값

o MSE는 실제값과 예측값의 차이를 제곱한 뒤 평균한 값으로, 값이 작을수록 예측성능이 우수함을 의미

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_t - \hat{y}_t)^2$$

◆ mean_squared_error() 함수를 사용하여 각 모형의 MSE를 계산

```
from sklearn.metrics import mean_squared_error

mse_mean = mean_squared_error( test["value"], test["pred_mean"] )

mse_last = mean_squared_error( test["value"], test["pred_last"] )

mse_drift = mean_squared_error( test["value"], test["pred_drift"] )

print("Mean :", mse_mean)
print("Last :", mse_last)
print("Drift:", mse_drift)
```

```
Mean : 326.50277395297474
Last : 425.1726033055617
Drift: 465.9076036601397
```

◆ 분석 결과 해석

□ 분석 결과, 평균 예측, 마지막 값 예측, 표류 예측의 MSE는 각각 327, 425, 466

□ 이번에 생성된 특정 확률보행 자료에서는 평균 예측의 MSE가 가장 작게 나타남

□ 따라서 이번 테스트 구간에서는 세 가지 방법 가운데 평균 예측이 실제값에 가장 가까운 예측 결과를 보임

□ 다만 이러한 결과는 이번에 무작위로 생성된 하나의 확률보행 경로에 대한 결과이므로, 다른 확률보행 자료를 생성하면 세 모형의 MSE 크기와 순위가 달라질 수 있음

(5) 예측기간이 길어질수록 왜 예측의 불확실성이 커지는가?

◆ 앞에서는 확률보행의 미래값을 예측하기 위해 평균 예측, 마지막 값 예측, 표류 예측의 세 가지 방법을 적용하고 예측 성과를 비교하였음

□ 그러나 어떤 방법으로 점예측값을 구하더라도, 예측기간이 길어질수록 실제 미래값에는 아직 발생하지 않은 무작위 충격이 계속 누적됨

□ 이를 확률보행의 식을 이용하여 살펴보면, 한 시점 후의 실제값은

$$y_{T+1} = y_T + \epsilon_{T+1}$$

□ 두 시점 후의 값은

$$y_{T+2} = y_T + \epsilon_{T+2}$$
$$= y_T + \epsilon_{T+1} + \epsilon_{T+2}$$

□ 같은 방법으로 h 시점 후의 값은 $y_{T+h} = y_T + \epsilon_{T+1} + \epsilon_{T+1} + \cdots + \epsilon_{T+h}$

□ 예측기간 h가 길어질수록 예측 시점에는 알 수 없는 미래의 충격이 더 많이 누적됨

□ 특히 확률보행의 이론적 점 예측인 마지막 값 예측을 기준으로 예측오차가 어떻게 증가하는지 살펴보면,

$\hat{y}_{T+h|T} = y_T$ 이므로

$$y_{T+h} - \hat{y}_{T+h|T} = (y_T + \epsilon_{T+1} + \cdots + \epsilon_{T+h}) - y_T$$
$$= \epsilon_{T+1} + \cdots + \epsilon_{T+h}$$

◆ **예측기간과 예측오차의 크기**

□ 위 식에서 보듯이 확률보행의 예측오차는 예측기간 동안 새롭게 발생한 무작위 충격의 합으로 나타남

□ 한 시점 후의 예측오차는 $y_{T+1} - \hat{y}_{T+1|T} = \epsilon_{T+1}$

□ 예측오차의 분산 $Var(y_{T+1} - \hat{y}_{T+1|T}) = Var(\epsilon_{T+1})$

앞에서 백색잡음에 대해 $\epsilon_t \sim WN(0, \sigma^2)$라고 가정했으므로 모든 시점의 충격은 다음과 같은 성질을 가짐

$E(\epsilon_t) = 0, Var(\epsilon_t) = \sigma^2$

따라서 T+1 시점의 충격도 $Var(\epsilon_{T+1}) = \sigma^2$

결국은 $Var(y_{T+1} - \hat{y}_{T+1|T}) = \sigma^2$

□ 두 시점 후의 예측오차는 $y_{T+2} - \hat{y}_{T+2|T} = \epsilon_{T+1} + \epsilon_{T+2}$

백색잡음의 각 충격은 서로 상관되지 않고 각각의 분산이 σ^2이므로,

$$\begin{aligned} Var(\epsilon_{T+1} + \epsilon_{T+2}) &= Var(\epsilon_{T+1}) + Var(\epsilon_{T+2}) \\ &= \sigma^2 + \sigma^2 \\ &= 2\sigma^2 \end{aligned}$$

□ 같은 방법으로 h 시점 후의 예측오차에는 h 개의 미래 충격이 포함되므로

$$Var(y_{T+h} - \hat{y}_{T+h}|T) = h\sigma^2$$

□ 따라서 예측기간 h가 길어질수록 예측오차의 분산이 커짐

◆ **예측 불확실성의 증가**

□ 예측오차의 표준편차는 분산의 제곱근이므로

$$SD(y_{T+h} - \hat{y}_{T+h}|T) = \sqrt{h\sigma^2}$$
$$= \sigma\sqrt{h}$$

□ 따라서 예측기간이 길어질수록 예측의 불확실성이 증가

◇ (결론) 확률보행에서는 미래의 무작위 충격을 미리 예측할 수 없으며, 예측기간이 길어질 수록 이러한 충격이 누적되므로 예측오차의 분산과 예측의 불확실성이 증가

8.3.2 단기 예측(One-step-ahead forecast)

◆ 단기 예측의 개념

□ 앞에서는 시점 T까지의 정보를 이용하여 이후의 여러 시점을 한꺼번에 예측하였음

□ 이번에는 매 시점 새로운 실제값을 관측한 후, 바로 다음 한 시점의 값만 예측하는 단기 예측(one-step-ahead forecast)을 살펴봄

□ 확률보행에서는 $y_{t+1} = y_t + \epsilon_{t+1}$이고, 미래 충격의 기대값이 0이므로 $E(y_{t+1}|y_t) = y_t$

따라서 한 시점 후의 예측값은 $\hat{y}_{t+1|t} = y_t$

즉, 현재 시점의 실제값을 다음 시점의 예측값으로 사용

◆ 예측 절차

□ 단기 예측은 다음 과정을 반복

① 시점 t의 실제값 y_t를 이용하여 t + 1 시점을 예측

$$\hat{y}_{t+1|t} = y_t$$

② t + 1 시점의 실제값 y_{t+1}이 새롭게 관측되면, 이를 이용해 다시 다음 시점을 예측

$$\hat{y}_{t+2|t+1} = y_{t+1}$$

③ 이러한 과정을 매 시점 반복

◆ shift() 함수를 이용한 구현

□ 파이썬 pandas의 shift(1)을 이용하면, 데이터를 아래로 한 칸 이동시켜, 각 시점에 바로 전 시점(t-1)의 값을 할당할 수 있음

df[“one_step_forecast“] = df[“value“].shift(1)

시점 t	실제값 () df[“value“]	한 시점 전 값 (y_{t-1}) df[“value“].shift(1)	비고
800	y_{800}	y_{799}	현재(800) 시점의 예측치로 이전(799) 값 사용
801	$y_{801}y_t$	y_{800}	현재(801) 시점의 예측치로 이전(800) 값 사용
802	y_{802}	y_{801}	
803	y_{803}	y_{802}	

□ 파이썬 pandas의 shift(-1)을 이용하면, 현재 시점에 한 시점 후(미래)의 값을 가져와 배치

◆ 실제값과 예측값 비교

```
fig, ax = plt.subplots(figsize=(10, 5))

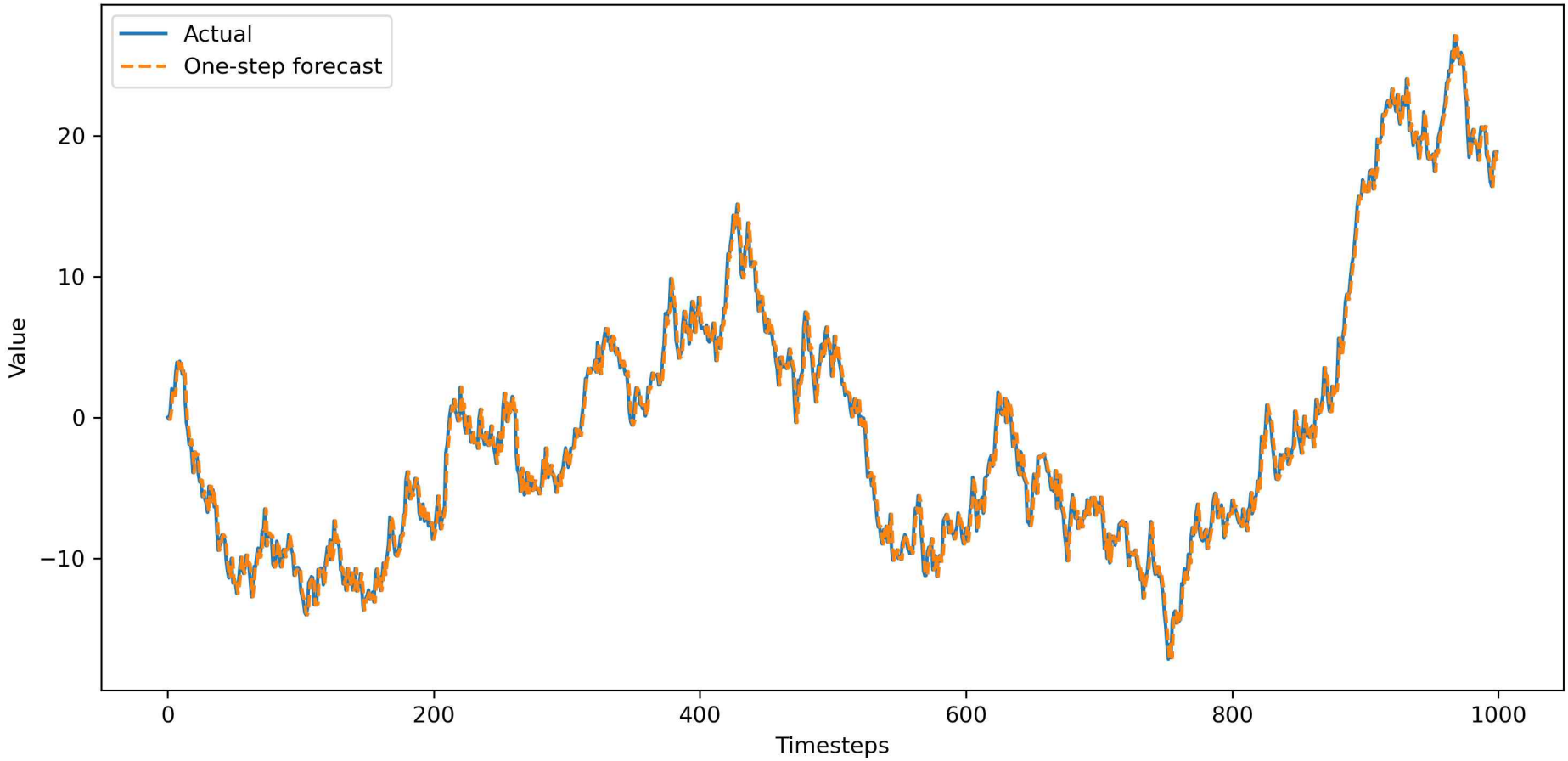
ax.plot(df["value"], label="Actual")
ax.plot(df["one_step_forecast"], linestyle="--", label="One-step forecast")

ax.set_xlabel("Timesteps")
ax.set_ylabel("Value")
ax.legend()

plt.tight_layout()
plt.show()
```

□ 예측값은 직전 시점의 실제값을 사용하므로 실제 시계열을 한 시점 늦게 따라가는 형태로 나타남

□ 확률보행의 시점 간 변화가 크지 않은 구간에서는 두 선이 거의 겹쳐 보일 수 있음


Actual
One-step forecast
20
10
0
−10
Value
0
200
400
600
800
1000
Timesteps

◆ 예측 정확도 평가

□ 테스트 구간에서 MSE를 계산하면,

```
from sklearn.metrics import mean_squared_error

mse_one_step = mean_squared_error(
    df.loc[800:, “value“], df.loc[800:, “one_step_forecast“]
)

mse_one_step
```

0.9256876651440581

□ 분석 결과, MSE는 약 0.93으로 나타남

□ 이 값이 작은 것은 복잡한 예측모형이 미래의 패턴을 학습했기 때문이 아니라, 확률보행에서는 현재값 자체가 다음 시점의 이론적인 최적 점예측값이기 때됨

◆ 장기 예측과 단기 예측의 차이

	장기 h-step 예측	One-step-ahead 예측
예측	T에서 T + h 까지 예측	바로 다음 한 시점만 예측
새로운 실제값	반영하지 않음	매 시점 반영
확률보행의 예측값	y_T	y_t
예측오차	여러 미래 충격의 합	다음 시점의 충격 하나
오차분산	$h\sigma^2$	σ^2

8.4 이동평균과정(MA)과 예측

◆ 개요 및 학습 내용

□ 앞에서는 확률보행과 정상성, 차분, 자기상관함수(ACF)의 개념을 살펴보았음

- o 확률보행을 1차 차분하면 백색잡음에 가까운 정상 시계열이 됨
- o 백색잡음에서는 lag 0을 제외하면 자기상관이 거의 나타나지 않음

□ 그러나 정상 시계열이라고 해서 항상 자기상관이 없는 것은 아님

- o 일부 정상 시계열에서는 과거의 충격이 일정 기간 동안 현재값에 영향을 미쳐 자기상관이 나타날 수 있음
- o 이러한 시계열을 설명하는 대표적인 모형 가운데 하나가 이동평균모형(Moving Average: MA)임

□ 본 절에서는

① 이동평균과정의 의미를 이해하고

② ACF를 이용하여 MA 모형의 차수를 식별한 후

③ 추정된 MA 모형을 이용하여 미래값을 예측하는 방법을 살펴봄

8.4.1 이동평균과정의 정의

◆ 이동평균과정

□ 이동평균과정(Moving Average Process)은 현재값이 현재의 무작위 충격과 일정 기간 동안의 과거 무작위 충격에 의해 결정되는 시계열 모형

□ 차수 q인 이동평균모형 MA(q)는 다음과 같이 표현됨

$$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2} + \cdots + \theta_q \epsilon_{t-q}$$

- o μ : 시계열의 평균
- o ϵ_t: 현재 시점의 무작위 충격
- o $\epsilon_{t-1}, \epsilon_{t-2}, \cdots, \epsilon_{t-q}$: 과거의 무작위 충격
- o $\theta_1, \theta_2, \cdots, \theta_q$: 과거 충격이 현재값에 주는 영향의 크기
- o q: 현재값에 영향을 주는 과거 충격의 수

□ 오차항 ϵ_t은 평균이 0이고 분산이 일정하며 시점 간 상관관계가 없는 백색잡음으로 가정

$$\epsilon_t \sim WN(0, \sigma^2)$$

◆ MA(1)과 MA(2)

□ MA(1) : 현재값은 현재 충격과 바로 직전 시점의 충격에 영향을 받음

$$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1}$$

□ MA(2) : 현재값은 현재 충격과 과거 2개 시점의 충격에 영향을 받음

$$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2}$$

□ 따라서 q는 현재값에 영향을 미치는 과거 충격이 몇 시점까지 포함되는가를 나타냄

8.4.2 이동평균과정의 차수 식별

◆ 왜 ACF를 이용하는가?

□ MA(q) 과정에서는 현재값 y_t가 현재 충격과 과거 q 시점까지의 충격에 영향을 받음

□ 따라서 q 시점보다 멀리 떨어진 관측값과는 공통으로 포함되는 충격이 없어짐

□ MA(q) 과정의 자기상관은 일반적으로 lag q까지 나타나고 그 이후에는 0이 되는 특징

o 이를 ACF의 절단(cutoff)이라고 함

□ 예를 들어 MA(2)라면, $y_t = \mu + \epsilon_t + \theta_1\epsilon_{t-1} + \theta_2\epsilon_{t-2}$ 이므로 ACF는 이론적으로 lag 2까지 나타나고 이후에는 0이 됨

◆ MA 차수 식별 절차

① 분석할 시계열의 정상성 확인

② 비정상 시계열이면 차분 등의 방법으로 정상성 확보

③ 정상 시계열의 ACF 확인

④ ACF가 lag q까지 유의하고 이후 급격히 사라지는 절단(cutoff) 형태를 보이는지 확인

⑤ 절단 시점을 바탕으로 MA(q)의 후보 차수를 결정

◇ 예:

□ “1) lag 1, 2에서 유의, 2) lag 3 이후 대부분 유의하지 않음” 이라면
MA(2) 모형을 후보로 고려

8.4.3 미국 제조업 내구재 신규주문액을 이용한 MA 모형 분석

❶ 데이터 불러오기

□ 분석 대상은 미국 제조업의 월별 내구재 신규주문액

- 자료: FRED DGORDER
- 기간: 2000년 1월 ~ 2025년 12월
- 단위: 백만 달러(계절조정 자료)

```
import pandas_datareader.data as web
import datetime

start = datetime.datetime(2000, 1, 1)
end = datetime.datetime(2025, 12, 31)
df = web.DataReader("DGORDER", "fred", start, end)
df.columns = ["orders"]
df.head()
```

◆ 코드 설명

□ web.DataReader(): 파이썬에서 금융 및 거시경제 데이터를 외부 웹에서 손쉽게 가져오기 위해 사용하는 pandas_datareader 라이브러리의 핵심 함수

□ web.DataReader()의 기본 구조와 매개변수

```
df = web.DataReader(name, data_source, start, end)
```

1. name: 조회하고자 하는 데이터의 고유 심볼이나 코드
   - ▸ “DGORDER“는 미국 세인트루이스 연은(FRED)에서 관리하는 내구재 주문(Durable Goods Orders) 지표의 고유 코드
2. data_source: 데이터를 제공하는 외부 플랫폼을 지정
   - ▸ “fred“: 미국 연방준비은행 경제 데이터
3. start와 end: 데이터를 가져올 시작일과 종료일(datetime 객체 또는 “YYYY-MM-DD“ 형태의 문자열)을 지정

□ df.columns = [“orders“]: 가져온 데이터의 열(Column) 이름을 알기 쉽게 “orders“로 바꿈

❷ 시계열 그래프 작성

```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

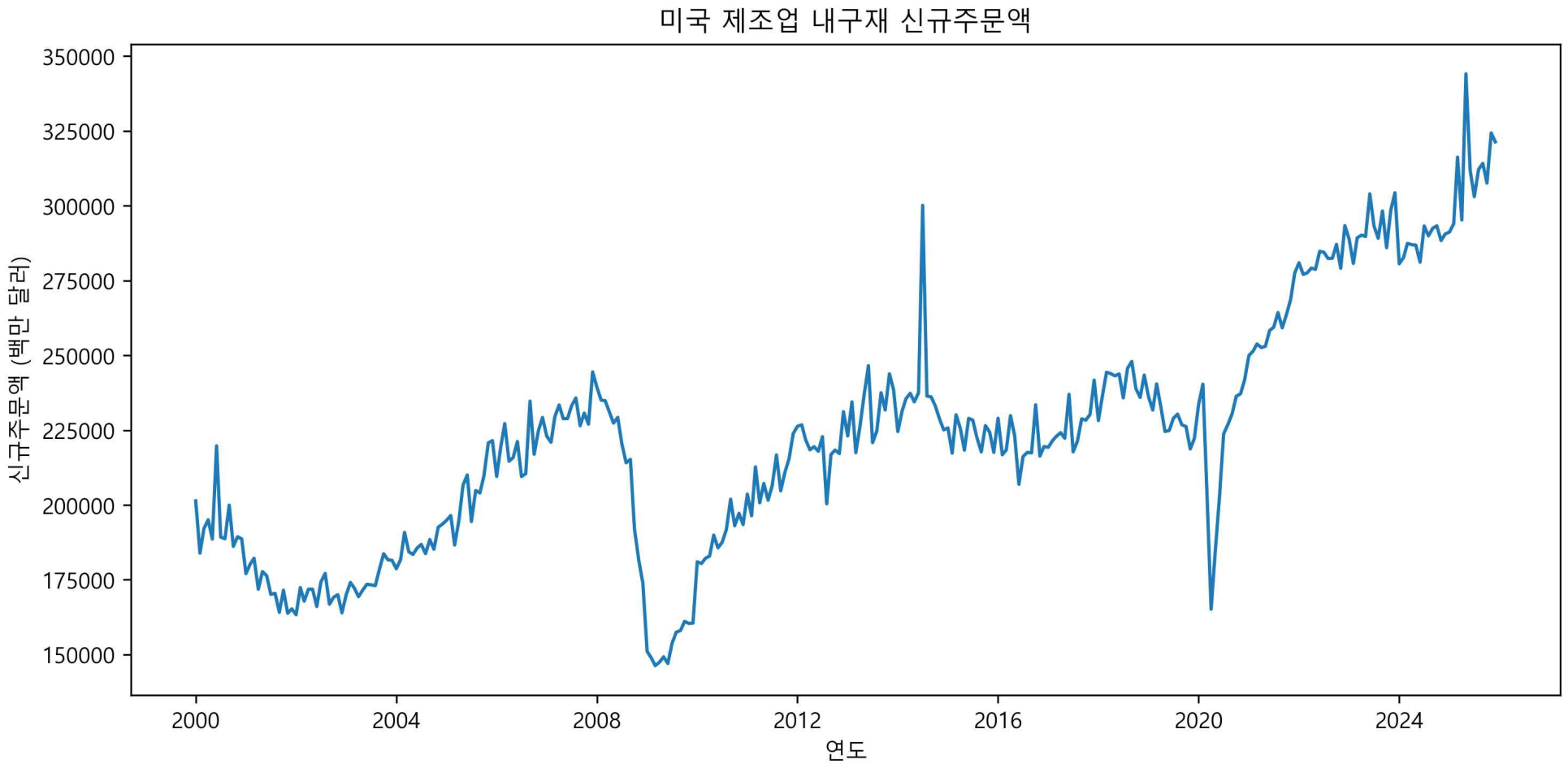
plt.plot(df.index, df["orders"])

plt.xlabel("연도")
plt.ylabel("신규주문액 (백만 달러)")
plt.title("미국 제조업 내구재 신규주문액")

plt.tight_layout()
plt.show()
```

□ 미국 제조업 내구재 신규주문액은 장기적으로 증가하는 추세를 보이지만, 금융위기와 코로나19 시기에는 큰 폭으로 감소

□ 따라서 원 시계열은 평균 수준이 시간에 따라 변하는 비정상적 시계열의 특징을 나타냄



❸ 정상성 검정

□ MA 모형을 적용하기 전에 ADF 검정을 이용하여 정상성을 확인

```
from statsmodels.tsa.stattools import adfuller

adf_result = adfuller(df[“orders“])

print(f“ADF Statistic: {adf_result[0]:.3f}“)
print(f“p-value: {adf_result[1]:.4f}“)
```

```
ADF Statistic: -0.408
p-value: 0.9087
```

□ ADF 통계량은 -0.408, p-값은 0.9087로 나타남

□ p-값이 0.05보다 크므로 단위근이 존재한다는 귀무가설을 기각할 수 없음

→ 원 시계열의 정상성이 확인되지 않아 1차 차분을 적용

❹ 1차 차분 후 ADF 검정

□ 차분 후 다시 ADF 검정을 실시하여 정상성이 확보되었는지 확인

```
orders_diff = df[“orders“].diff().dropna()

adf_diff = adfuller(orders_diff)

print(f“ADF Statistic: {adf_diff[0]:.3f}“)
print(f“p-value: {adf_diff[1]:.4f}“)
```

```
ADF Statistic: -17.521
p-value: 0.0000
```

□ ADF 통계량은 -17.521, p-값은 0.0000으로 나타남

□ p-값이 0.05보다 작으므로 단위근이 존재한다는 귀무가설을 기각

→ 1차 차분된 시계열은 정상성을 지지하는 결과를 보임

❺ 차분된 시계열 확인

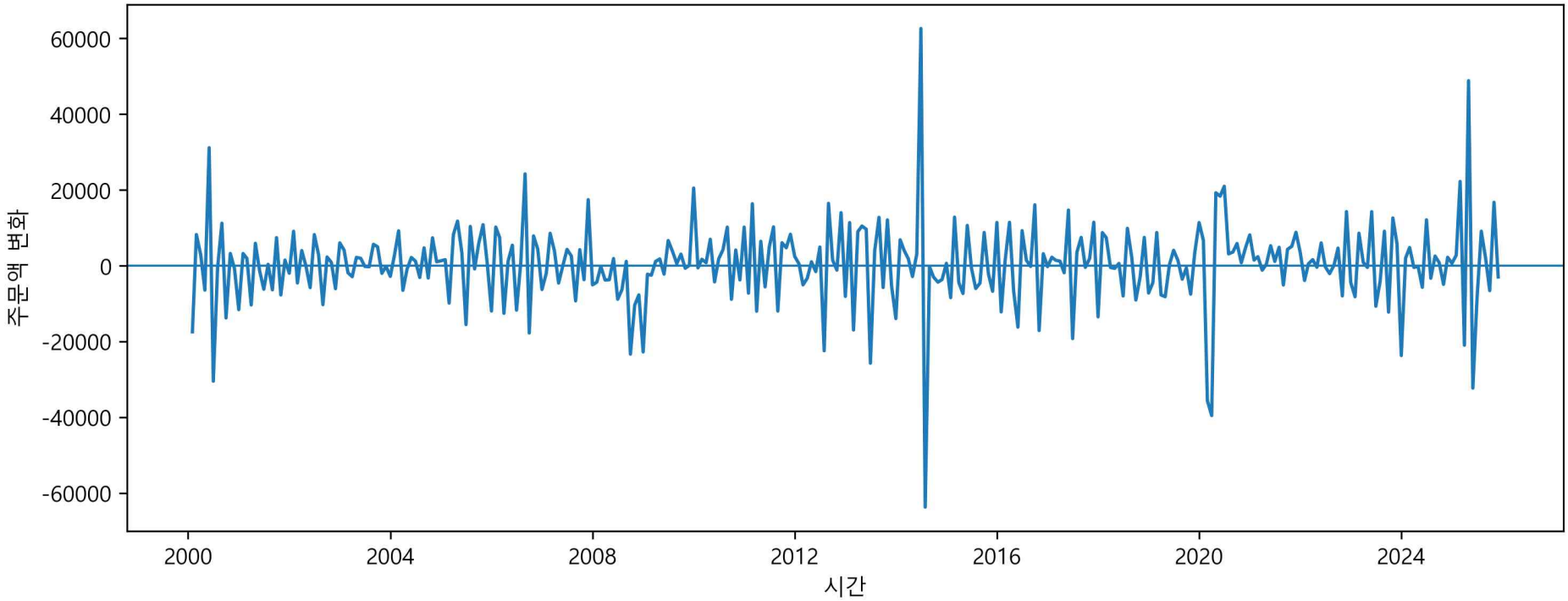
□ 차분 후 시계열이 일정한 수준을 중심으로 변동하는지 확인

```
fig, ax = plt.subplots(figsize=(10, 4))

ax.plot(orders_diff)
ax.axhline(0, linewidth=1)

ax.set_xlabel("시간")
ax.set_ylabel("주문액 변화")

plt.tight_layout()
plt.show()
```

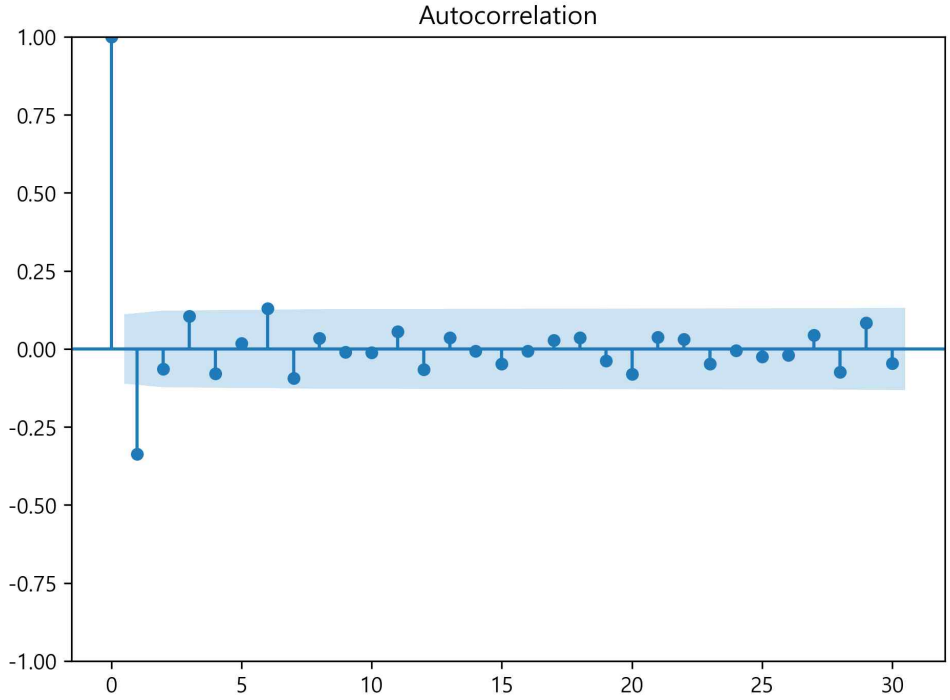


□ 차분 후 시계열은 대체로 0을 중심으로 변동하며, 원 시계열에서 관찰되던 장기적인 상승 추세가 제거됨

□ 다만 일부 시점에서 큰 폭의 변동이 나타나 충격의 크기가 일정하지 않을 가능성이 있음

❻ ACF를 이용한 MA 차수 식별

```
from statsmodels.graphics.tsaplots import plot_acf
plot_acf(orders_diff, lags=30)
plt.tight_layout()
plt.show()
```



☐ ACF 그래프에서 lag 1의 자기상관계수는 약 -0.34로 유의한 음(-)의 값을 나타냄

☐ lag 2 이후에는 대부분의 자기상관계수가 신뢰구간 안에 위치하며, lag 6에서 일부 유의한 자기상관이 관찰됨

☐ ACF는 lag 1 이후 대체로 절단되는 형태를 보이므로 MA(1) 모형을 우선적인 후보로 고려.

$\triangle y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1}.$ 여기서 MA(1) 모형은 1차 차분된 신규주문액에 적용

◆ 정리

□ MA(q) 과정은 현재값이 현재 및 과거 q시점까지 무작위 충격에 의해 결정되는 정상 시계열 모형

$$y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \cdots + \theta_q \epsilon_{t-q}$$

□ MA(q) 과정의 ACF는 일반적으로 lag q 이후 절단(cutoff)되는 특징을 가짐

MA(q) ⇒ ACF가 lag q 이후 절단

□ 따라서 정상성을 확인한 후 ACF의 절단 패턴을 이용하여 MA 모형의 후보 차수 q를 식별할 수 있음

8.4.3 이동평균모델(MA)을 사용하여 시계열 예측

(1) 이동평균모형의 예측 원리

◆ 앞에서는 미국 제조업 내구재 신규주문액을 1차 차분한 후 ACF를 분석하여 MA(1)을 후보 모형으로 선정하였음

□ 따라서 차분된 신규주문액에 MA(1) 모형을 적용하여 향후 신규주문액의 변화를 예측

$$\triangle y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1}$$

- Δy_t : 전월 대비 신규주문액 변화
- μ : 주문액 변화의 평균
- ϵ_t : 현재 시점의 무작위 충격
- θ_1: 직전 시점 충격의 영향

◆ MA(1) 모형의 예측 특성

□ 현재 시점이 T일 때 한 시점 후의 실제 변화량은

$$\Delta y_{T+1} = \mu + \epsilon_{T+1} + \theta_1 \epsilon_T$$

□ 미래 충격 ϵ_{T+1}은 미리 알 수 없지만, 현재까지 발생한 충격 ϵ_T은 모형을 통해 추정가능

$$\widehat{\Delta y}_{T+1|T} = \mu + \theta_1 \hat{\epsilon}_T$$

□ MA(1) 모형에서 두 시점 후의 신규주문액 변화는

$$\Delta y_{T+2} = \mu + \epsilon_{T+2} + \theta_1 \epsilon_{T+1}$$

□ 시점 T에서는 ϵ_{T+1}과 ϵ_{T+2}가 모두 미래의 무작위 충격이므로 그 값을 알 수 없음

□ 백색잡음 충격이 독립적이고 평균이 0이라는 가정에 따라

$$E(\epsilon_{T+1}|\Omega_T) = 0,\ E(\epsilon_{T+2}|\Omega_T) = 0$$

여기서 Ω_T는 시점 T까지 이용할 수 있는 정보를 의미함

□ 따라서 두 시점 후의 예측값은

$$\begin{aligned}\hat{\Delta} y_{T+2|T} &= E(\Delta y_{T+2}|\Omega_T) \\ &= E(\mu + \epsilon_{T+2} + \theta_1 \epsilon_{T+1}|\Omega_T) \\ &= \mu + E(\epsilon_{T+2}|\Omega_T) + E(\theta_1 \epsilon_{T+1}|\Omega_T) \\ &= \mu + 0 + \theta_1(0) \ = \mu\end{aligned}$$

□ 두 시점 후에는 현재 이용 가능한 과거 충격의 영향이 예측식에서 사라지므로, 신규주문액 변화의 예측값은 평균 μ와 같아짐

o 미래 충격이 실제 0이 아니고, 현재 정보로 예측한 미래 충격의 기댓값이 0이라는 의미

□ 따라서 MA(1) 모형은 한 시점 후의 예측에는 직전 충격 정보를 활용하지만, 두 시점 후부터는 평균값을 예측하는 특성을 가짐

(2) 데이터 분할

◆ 차분된 신규주문액 전체 관측치의 처음 90%를 훈련 데이터, 마지막 10%를 테스트 데이터로 분할하여 사용

```
# 차분된 주문액
df_diff = orders_diff.to_frame(name="orders_diff")

# 훈련 및 테스트 데이터 분할
train_len = int(len(df_diff) * 0.9)

train = df_diff.iloc[:train_len]
test = df_diff.iloc[train_len:].copy()

print("훈련 데이터:", len(train))
print("테스트 데이터:", len(test))
```

□ iloc을 이용하여 시간 순서대로 데이터를 분할

- o train: MA(1) 모형을 추정하는 데 사용
- o test: 추정된 모형의 예측 성능을 평가하는 데 사용

□ 시계열 자료에서는 미래 정보가 과거의 모형 추정에 포함되지 않도록 시간 순서를 유지하여 분할해야 함

(3) 예측 구간 시각화

□ 위쪽 그래프는 원래 신규주문액의 흐름을, 아래쪽 그래프는 전월 대비 변화량을 보여줌

□ 음영으로 표시된 기간은 모형의 예측값과 실제값을 비교하는 테스트 구간

□ sharex=True는 두 그래프가 동일한 시간 축을 사용하도록 설정

```
fig, (ax1, ax2) = plt.subplots( 2, 1, figsize=(10, 7), sharex=True )
```

```
ax1.plot(df.index, df["orders"])
ax1.set_ylabel("신규주문액 (백만 달러)")

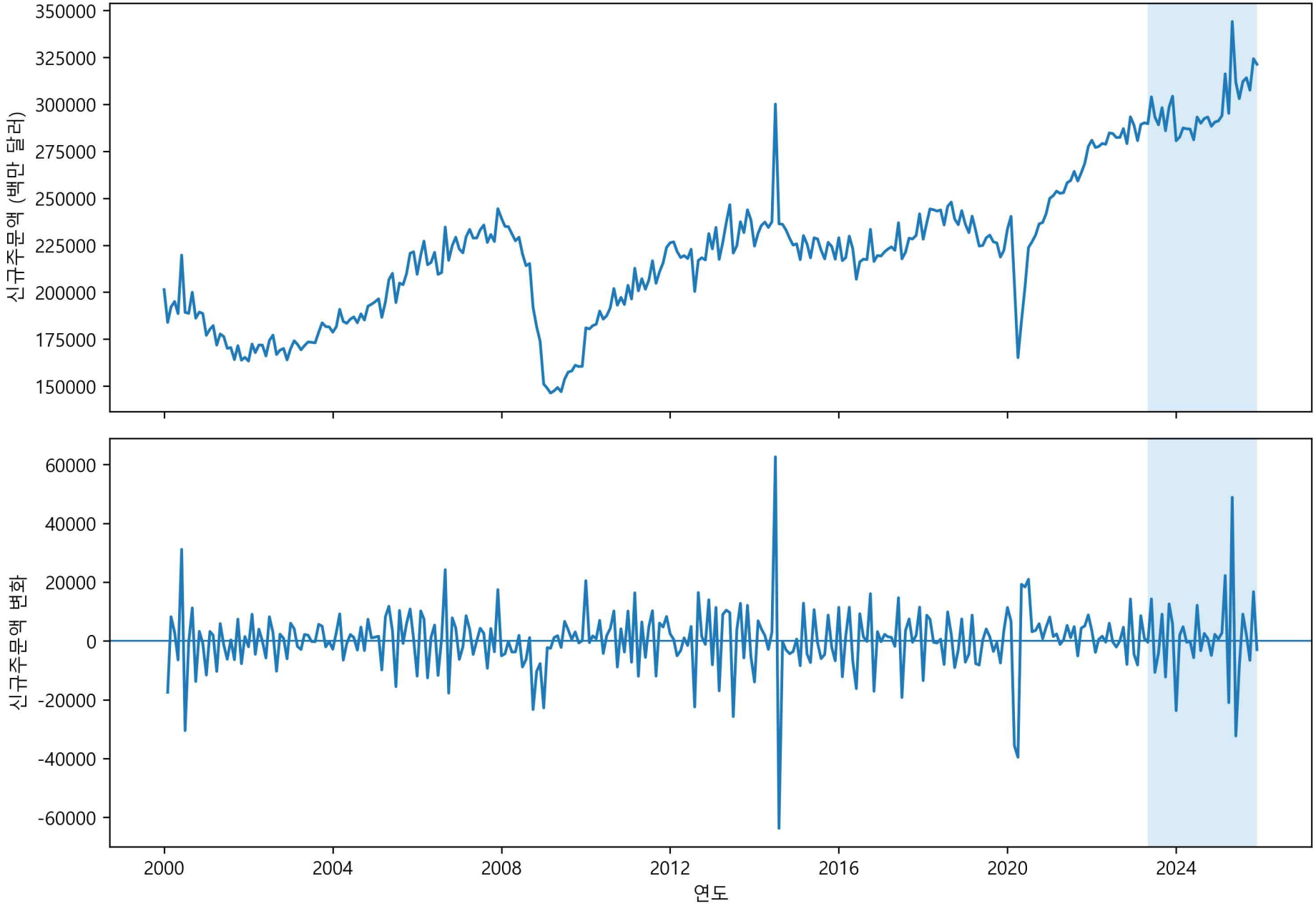
ax2.plot(df_diff.index, df_diff["orders_diff"])
ax2.axhline(0, linewidth=1)
ax2.set_ylabel("신규주문액 변화")

# 테스트 구간 표시
test_start = test.index[0]

ax1.axvspan(test_start, df.index[-1], alpha=0.2)
ax2.axvspan(test_start, df.index[-1], alpha=0.2)

ax2.set_xlabel("연도")

plt.tight_layout()
plt.show()
```


350000
325000
300000
275000
250000
225000
200000
175000
150000
신규주문액 (백만 달러)
60000
40000
20000
0
-20000
-40000
-60000
신규주문액 변화
2000
2004
2008
2012
2016
2020
2024
연도

(4) MA(1) 모형의 장기 예측 한계

◆ 예측 시계(forecast horizon)는 현재 시점에서 얼마나 먼 미래까지 예측하는지를 의미

□ MA(1) 모형에서는 예측기간이 길어질수록 현재 이용할 수 있는 충격 정보가 미래값의 예측에 반영되지 않게 됨

□ 한 시점 후에는 $\widehat{\Delta y}_{T+1|T} = \mu + \theta_1 \hat{\epsilon}_T$

□ 두 시점 후부터는 $\widehat{\Delta y}_{T+h|T} = \mu$

→ MA(1) 모형의 차분값 예측은 두 시점 후부터 평균과 같아짐

□ 따라서 MA(1)은 직전 충격을 이용한 단기 예측에 특징이 있으며, 먼 미래의 변화량을 예측할 때는 새로운 충격에 관한 정보를 활용할 수 없음

(5) 롤링 포캐스트(rolling forecast)

◆ 롤링 포캐스트는 미래 전체를 한 번에 예측하는 대신, 새로운 실제값이 관측될 때마다 정보를 갱신하여 단기 예측을 반복하는 방법

◆ **롤링 포캐스트의 절차:**

① 훈련 데이터를 이용하여 MA(1) 모형을 추정

② 다음 한 시점의 신규주문액 변화를 예측

③ 새로운 실제값이 관측되면 이를 훈련 데이터에 추가

④ 갱신된 데이터로 MA(1) 모형을 다시 추정

⑤ 테스트 기간이 끝날 때까지 반복

□ 이러한 방식은 매월 새로운 경제통계가 발표되면 이를 반영하여 다음 달을 예측하는 실제 경제분석 상황과 유사함

8.4.5 롤링 포캐스트 기반 모형 비교

(1) 분석 목적

◆ MA(1) 모형의 예측 성능을 평가하기 위해 세 가지 방법을 비교

예측 방법	예측값
평균 예측(Mean)	과거 주문액 변화의 평균
최근값 예측(Last Value)	직전 주문액 변화
MA(1) 예측	현재까지 추정된 충격 구조를 반영한 다음 시점의 주문액 변화

(2) 모형 구현 도구: SARIMAX

◆ statsmodels 라이브러리의 SARIMAX를 이용하여 MA(1) 모형을 추정

❶ 데이터 불러오기 및 차분

```
import pandas_datareader.data as web
import datetime
```

```
start = datetime.datetime(2000, 1, 1)
end = datetime.datetime(2025, 12, 31)

# 원자료 불러오기
df = web.DataReader("DGORDER", "fred", start, end)
df.columns = ["orders"]

# 1. 월별 빈도 지정
df = df.asfreq("MS")

# 2. 1차 차분
orders_diff = df["orders"].diff().dropna()

# 3. 차분된 데이터를 DataFrame으로 변환
df_diff = orders_diff.to_frame(name="orders_diff")
```

❷ 훈련 데이터와 테스트 데이터 분할

```
# 4. 훈련 및 테스트 데이터 분할
train_len = int(len(df_diff) * 0.9)
train = df_diff.iloc[:train_len]
test = df_diff.iloc[train_len:].copy()

# 5. 빈도 정보 확인
print("df:", df.index.freq)
print("df_diff:", df_diff.index.freq)
print("train:", train.index.freq)
```

❸ 롤링 포캐스트

◆ 테스트 기간의 각 시점에서 세 가지 예측 방법을 반복적으로 적용

```
import numpy as np
from statsmodels.tsa.statespace.sarimax import SARIMAX

# 예측 결과 저장
pred_mean = []
pred_last = []
pred_ma = []

# 테스트 구간에서 한 시점씩 예측
for i in range(len(train), len(df_diff)):
    # 현재까지 관측된 데이터
    history = df_diff["orders_diff"].iloc[:i]

    # 1. 평균 예측
    pred_mean.append(history.mean())

    # 2. 최근값 예측
```

```
pred_last.append(history.iloc[-1])

# 3. MA(1) 예측
model = SARIMAX( history, order=(0, 0, 1), trend="c" )

result = model.fit(disp=False)

forecast = result.forecast(steps=1)
pred_ma.append(forecast.iloc[0])
```

◇ 주요 코드 설명

□ from statsmodels.tsa.statespace.sarimax import SARIMAX

- statsmodels 라이브러리의 SARIMAX를 이용하여 MA(1) 모형을 추정
- SARIMAX는 AR, MA, 차분, 계절성 등을 포함할 수 있는 시계열 모형

□ for i in range(len(train), len(df_diff))

- 훈련 구간의 마지막 다음 시점부터 테스트 구간의 마지막까지 반복
- 반복할 때마다 예측 시점이 한 칸씩 이동

□ history = df_diff[“orders_diff“].iloc[:i]

- 예측 시점 이전까지 실제로 관측된 데이터만 선택
- 미래의 실제값은 현재 예측에 사용하지 않음

□ history.iloc[-1]: 가장 최근의 변화량을 예측값으로 사용

□ SARIMAX(history, order=(0, 0, 1), trend=“c“)

- order=(0, 0, 1) : MA(1) 모형 생성
  - ▸ 첫 번째 0: AR 차수, ▸ 두 번째 0: 추가 차분 차수, ▸ 세 번째 1: MA 차수
- 주문액 변화의 평균을 포함하기 위해 trend=“c“를 설정

□ result.fit(disp=False): 모형의 모수를 추정하고 추정 과정의 출력은 생략

□ result.forecast(steps=1): 다음 한 시점의 예측값 생성

❹ 예측 결과 저장

◆ 세 가지 방법으로 계산한 예측값을 테스트 데이터에 추가

□ pred_df에는 실제 신규주문액 변화와 세 가지 예측값이 함께 저장됨

```
pred_df = test.copy()

pred_df[“pred_mean“] = pred_mean
pred_df[“pred_last“] = pred_last
pred_df[“pred_ma“] = pred_ma

pred_df.head()
```

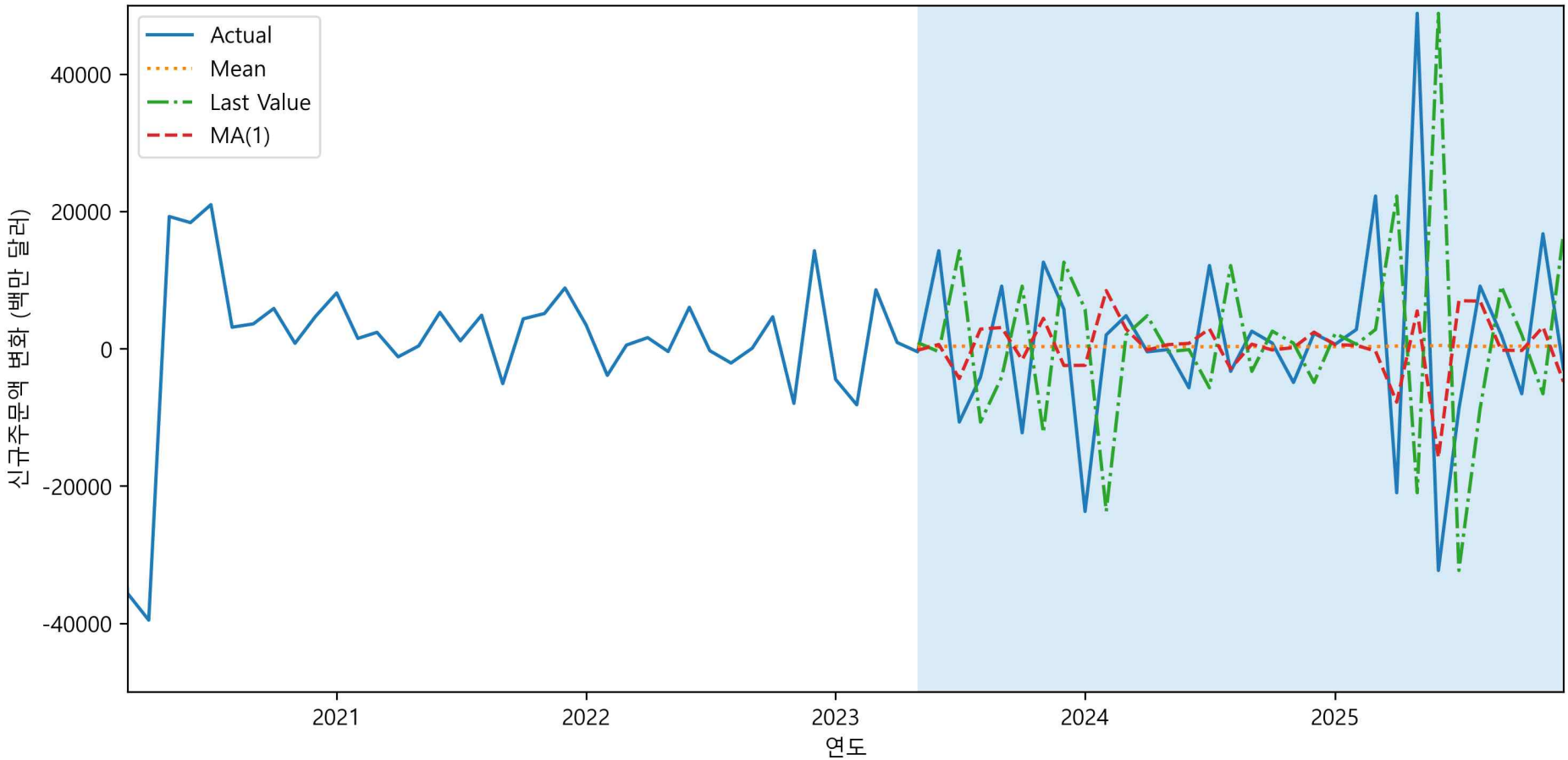
❺ 예측 결과 시각화

◆ 실제값과 세 가지 예측값을 비교

```
fig, ax = plt.subplots(figsize=(10, 5))

ax.plot( df_diff.index, df_diff["orders_diff"], label="Actual" )
ax.plot( pred_df.index, pred_df["pred_mean"], linestyle=":", label="Mean" )
ax.plot( pred_df.index, pred_df["pred_last"], linestyle="-.", label="Last Value" )
ax.plot( pred_df.index, pred_df["pred_ma"], linestyle="--", label="MA(1)" )

ax.axvspan( test.index[0], test.index[-1], alpha=0.2 )
ax.set_xlim(df_diff.index[-70], df_diff.index[-1])
ax.set_xlabel("연도")
ax.set_ylabel("신규주문액 변화 (백만 달러)")
ax.legend()
plt.tight_layout()
plt.show()
```


Actual
Mean
Last Value
MA(1)
40000
20000
0
-20000
-40000
신규주문액 변화 (백만 달러)
2021
2022
2023
2024
2025
연도

◆ 시각화 결과의 평가

□ MA(1) 모형(빨간 파선)은 최근값 예측에 비해 과도한 변동을 완화하면서 신규주문액의 단기적인 움직임을 일부 반영

o 최근값 예측(Last Value) : 초록 일점쇄선

▸ 직전 시점의 관측값을 이용하므로 신규주문액의 변동을 한 시점 늦게 따라가는 경향

□ 그러나 신규주문액의 급격한 변동을 충분히 설명하지 못하며, 평균 예측과 비교해서도 시각적으로 뚜렷한 우위를 확인하기 어려움

o 평균 예측(Mean) : 주황 점선

▸ 예측값이 0 부근에서 수평선으로 나타나 신규주문액의 변동을 제대로 반영하지 못함

□ 따라서 MA(1) 모형의 예측 성능 우월성은 MSE 등 정량적인 예측오차 지표를 통해 판단

❻ 예측 정확도 평가

◆ 평균제곱오차(MSE)를 이용하여 평균 예측, 최근값 예측, MA(1) 모형의 예측 성능을 비교

$$MSE = \frac{1}{n}\sum_{t=1}^{n}(y_t - \hat{y}_t)^2$$

```
from sklearn.metrics import mean_squared_error

mse_mean = mean_squared_error( pred_df[“orders_diff“], pred_df[“pred_mean“] )

mse_last = mean_squared_error( pred_df[“orders_diff“], pred_df[“pred_last“] )

mse_ma = mean_squared_error( pred_df[“orders_diff“], pred_df[“pred_ma“] )

print(f“Mean MSE: {mse_mean:.2f}“)
print(f“Last Value MSE: {mse_last:.2f}“)
print(f“MA(1) MSE: {mse_ma:.2f}“)
```

Mean MSE: 201447603.10

Last Value MSE: 627960646.03

MA(1) MSE: 141811498.60

□ 분석 결과, MA(1) 모형의 MSE가 가장 작게 나타나 세 가지 예측방법 중 가장 우수한 예측 성능을 보임

□ 최근값 예측은 MSE가 가장 크게 나타났는데, 이는 신규주문액의 증감이 급격하게 반전되는 경우 직전 관측값을 그대로 사용하는 방식이 큰 예측오차를 발생시킬 수 있기 때문

□ MA(1) 모형은 직전 충격의 영향을 반영하면서 예측값의 변동을 조정하므로 평균 예측이나 최근값 예측보다 작은 예측오차를 기록한 것으로 해석할 수 있음

□ 다만 이 결과만으로 MA(1)이 모든 기간에서 최적 모형이라고 일반화할 수는 없음

<참고> **MA(q)의 장기 예측 한계에 대한 수학적 표현**

◇ MA(2) 모형: $y_t = \mu + \epsilon_t + \theta_1 \epsilon_{t-1} + \theta_2 \epsilon_{t-2}$

◇ 현재 시점이 t라고 할 때 미래를 예측하면 다음과 같음:

1 1시점 앞 예측(one-step ahead forecast): $y_{t+1} = \mu + \epsilon_{t+1} + \theta_1 \epsilon_t + \theta_2 \epsilon_{t-1}$

o 미래 오차항 ϵ_{t+1}은 알 수 없으므로 $E(\epsilon_{t+1}) = 0$ ←

o 따라서 예측값은 $\hat{y}_{t+1|t} = \mu + \theta_1 \epsilon_t + \theta_2 \epsilon_{t-1}$ → 관측가능한 과거 오차항 정보 2개를 사용 가능

2 2시점 앞 예측(two-step ahead forecast): $y_{t+2} = \mu + \epsilon_{t+2} + \theta_1 \epsilon_{t+1} + \theta_2 \epsilon_t$

o ϵ_{t+2}, ϵ_{t+1}은 미래 오차항이므로 예측 불가능: $E(\epsilon_{t+2}) = E(\epsilon_{t+1}) = 0$

o 따라서 예측값은 $\hat{y}_{t+2|t} = \mu + \theta_2 \epsilon_t$ → 사용할 수 있는 과거 오차항 정보가 1개로 줄어듦

3 3시점 앞 예측(three-step ahead forecast): $y_{t+3} = \mu + \epsilon_{t+3} + \theta_1 \epsilon_{t+2} + \theta_2 \epsilon_{t+1}$

o 그런데 모든 오차항이 미래값이므로 $E(\epsilon_{t+3}) = E(\epsilon_{t+2}) = E(\epsilon_{t+1}) = 0$

o 따라서 예측값은 $\hat{y}_{t+3|t} = \mu$ → 사용할 수 있는 과거 오차항 정보가 없음

□ 이를 일반화하면, MA(2) 모형에서는 h > q 인 경우 $\hat{y}_{t+h|t} = \mu$

□ **예측시점이 q를 초과하면 예측값은 시계열 평균 μ로 수렴**⇒ MA(q) 모형이 장기예측에 부적합한 이유

8.4.6 예측 결과의 원래 데이터로 변환

(1) 역변환이 필요한 이유

□ 시계열 분석에서는 정상성 확보를 위해 원자료를 차분하여 모형을 추정하는 경우가 많음

□ 그러나 실제 분석에서는 차분값뿐만 아니라 원래 데이터 수준(level)에 대한 예측값에도 관심이 있음

□ 따라서 차분된 데이터로 추정한 예측값을 원래 데이터 수준으로 복원하는 과정이 필요하며, 이를 역차분(inverse differencing)이라고 함

(2) 1차 차분과 역차분의 원리

□ 1차 차분은 현재 시점의 값에서 이전 시점의 값을 뺀 변화량을 의미

$$\triangle y_t = y_t - y_{t-1}$$

□ 위 식을 정리하면 이전 시점의 값에 변화량을 더하여 원래 값을 복원할 수 있음

$$y_t = y_{t-1} + \triangle y_t$$

□ 따라서 한 시점 후의 원자료 예측값은 다음과 같이 계산

$$\hat{y}_{T+1|T} = y_T + \widehat{\Delta y}_{T+1|T}$$

- y_T : 예측 시점에 관측된 마지막 실제값
- $\widehat{\Delta y}_{T+1|T}$: MA(1) 모형으로 예측한 다음 시점의 변화량
- $\hat{y}_{T+1|T}$: 원래 데이터 수준으로 복원한 예측값

(3) 여러 시점 예측과 롤링 예측의 차이

□ 여러 시점을 한 번에 예측하는 경우 마지막 관측값을 기준으로 예측된 변화량을 누적하여 원래 수준을 복원

□ 1차 차분 : $\triangle y_t = y_t - y_{t-1}$

□ 따라서 여러 시점의 차분값을 누적하면

$$y_{T+1} = y_T + \Delta y_{T+1}$$

$$y_{T+2} = y_T + \Delta y_{T+1} + \Delta y_{T+2}$$

$$y_{T+3} = y_T + \Delta y_{T+1} + \Delta y_{T+2} + \Delta y_{T+3}$$

□ 일반적으로

$$\hat{y}_{T+1} = y_T + \sum_{j=1}^{h} \widehat{\Delta y}_{T+j|T}$$

□ 파이썬에서는 cumsum()을 이용해 역차분을 실행

o cumsum()은 마지막 훈련 관측값을 기준으로 예측 변화량을 누적하는 방식이므로, 기존의 롤링 예측값을 누적하면 새로운 예측 경로를 구성

(4) 파이썬 실습

□ 본 실습에서는 차분된 MA(1) 예측값을 원래 신규주문액 수준으로 복원하기 위해 cumsum() 함수를 이용

□ 훈련 구간의 마지막 실제값을 기준점으로 설정하고, 테스트 구간의 예측 변화량을 순차적으로 누적하여 원래 데이터 수준으로 변환

□ 다만 예측 변화량은 매 시점 실제 관측값을 반영한 롤링 1단계 예측에서 얻었으므로, 누적된 결과는 테스트 시작 시점에서 한 번에 예측한 다기간 미래 경로와 구분해야 함

□ 아래 코드 설명

- initial_value: 테스트 구간 직전의 실제 신규주문액으로, 역차분의 기준점
- pred_df[“pred_ma“].cumsum(): MA(1) 모형이 예측한 변화량을 순서대로 누적
- initial_value + ... : 누적된 변화량을 기준점에 더하여 원래 신규주문액 수준으로 복원

❶ 차분 예측값을 원래 데이터 수준으로 복원

```
# 훈련 구간 마지막 시점의 실제 신규주문액
initial_value = df["orders"].loc[:train.index[-1]].iloc[-1]

# # 차분 예측값의 누적합을 이용한 역차분
pred_df["pred_orders_cumsum"] = ( initial_value + pred_df["pred_ma"].cumsum() )

# 실제 신규주문액
pred_df["actual_orders"] = df["orders"].loc[pred_df.index]
```

❷ 실제값과 예측값 시각화

```
import matplotlib.pyplot as plt

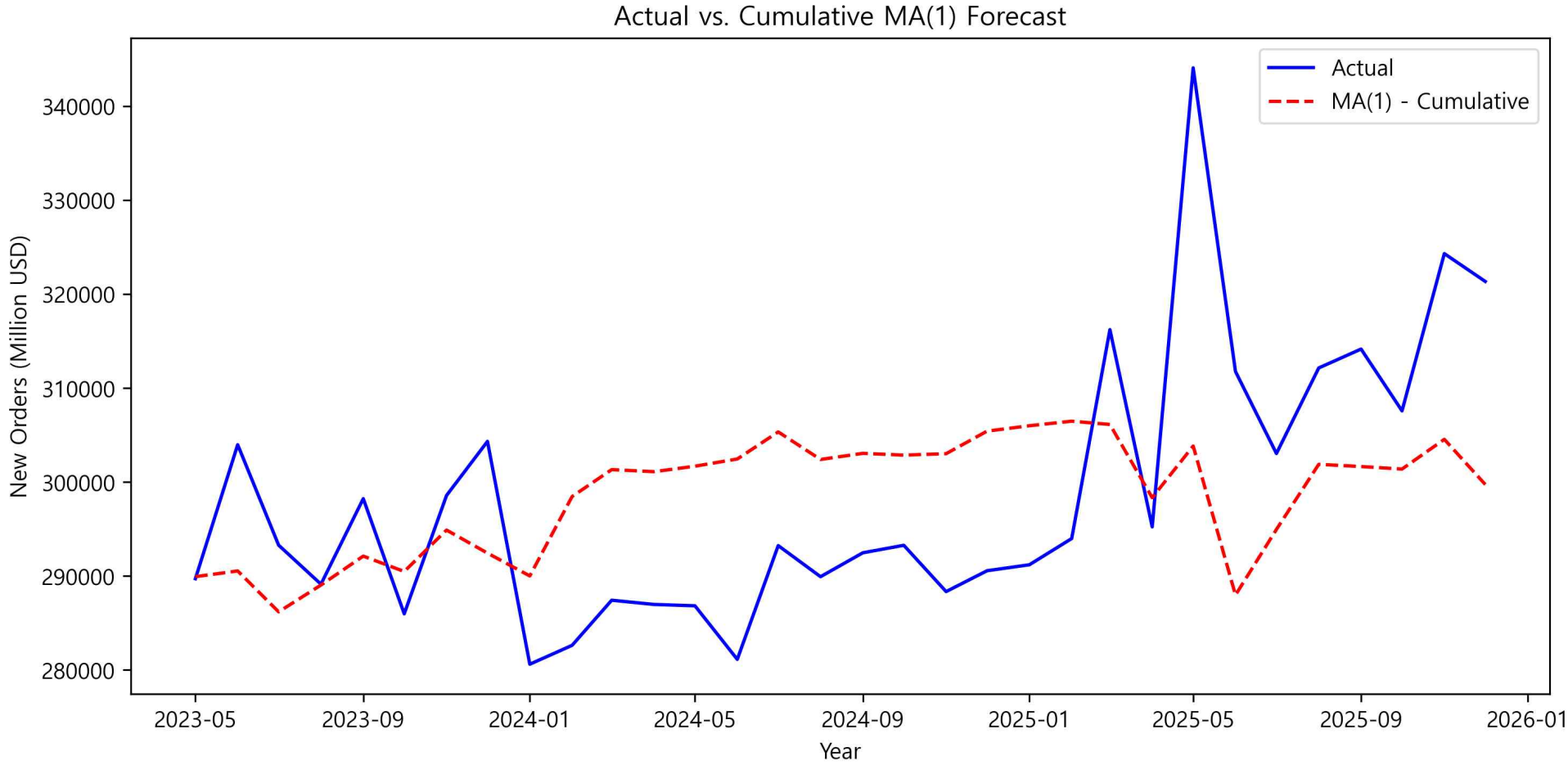
fig, ax = plt.subplots(figsize=(10, 5))

ax.plot( pred_df.index, pred_df["actual_orders"], color="blue", label="Actual" )

ax.plot( pred_df.index, pred_df["pred_orders_cumsum"], color="red", linestyle="--",
      label="MA(1) - Cumulative")

ax.set_xlabel("Year")
ax.set_ylabel("New Orders (Million USD)")
ax.set_title("Actual vs. Cumulative MA(1) Forecast")

ax.legend()
plt.tight_layout()
plt.show()
```



❸ 시각화 결과 해석

□ MA(1) 모형의 예측값은 실제 신규주문액의 전반적인 움직임을 비교적 잘 따라감

o 신규주문액 변동이 비교적 작은 구간(2024년 하반기)에서는 실제값과 예측값이 근접

□ 반면 2025년 4~6월경에는 실제 신규주문액이 급격하게 증가한 후 감소하면서 예측오차가 크게 발생

o MA(1) 모형은 급격한 증가를 제대로 예측하지 못하고, 이후 실제 신규주문액이 감소하는 시점에는 오히려 높은 예측값을 제시

o 이는 MA(1) 모형이 직전 충격을 반영하지만, 예측 시점에 발생하는 새로운 충격을 미리 알 수 없기 때문임

□ 따라서 MA(1) 모형은 단기적인 움직임을 예측하는 데 유용하지만, 급격한 변동이나 예기치 못한 충격을 정확하게 예측하는 데에는 한계가 있음

◇ 예측값이 실제 수준을 비교적 잘 따라가는 것은 MA(1) 모형의 예측력뿐 아니라, 최근 실제 수준을 기준으로 예측한다는 점에도 기인

→ 역차분 방식의 특성을 반영

→ 새로운 관측값이 들어올 때마다 갱신한 롤링 1단계 예측 결과

8.4.7 원래 데이터 수준에서 예측오차 평가

(1) 평균절대오차(MAE)

□ 차분된 데이터뿐만 아니라 원래 데이터 수준에서의 예측오차를 평가하기 위해 평균절대오차(mean absolute error, MAE)를 계산

$$MAE = \frac{1}{n}\sum_{t=1}^{n}|y_t - \hat{y}_{t|t-1}|$$

□ MAE는 실제값과 예측값 사이의 절대적인 차이를 평균한 값

□ MSE와 달리 예측오차를 제곱하지 않으므로 원자료와 동일한 단위로 해석할 수 있음

(2) 파이썬 실습

```
from sklearn.metrics import mean_absolute_error

mae_ma = mean_absolute_error( pred_df[“actual_orders“], pred_df[“pred_orders“] )

print(f“MA(1) MAE: {mae_ma:.2f}“)
```

```
MA(1) MAE: 7965.20
```

(3) 결과 해석

□ MA(1) 모형의 예측값을 역차분하여 실제 신규주문액과 비교한 결과, MAE는 7,965.20

□ 원자료의 단위가 백만 달러이므로, 이는 월별 신규주문액의 예측값과 실제값 사이에 평균적으로 약 79.65억 달러의 절대오차가 발생하였음을 의미

□ 다만 MAE는 예측오차의 평균적인 크기를 나타내는 지표이므로, 이 값만으로 MA(1) 모형의 예측 성능이 우수한지 판단하기는 어려움

◆ 8.4 이동평균모형(MA) 요약

1. 이동평균모형(MA)은 현재값이 평균과 현재 및 과거 오차항의 선형결합으로 결정되는 시계열 모형

2. 정상적인 MA(q) 과정의 자기상관함수(ACF)는 이론적으로 시차 q 이후 0이 되므로, ACF를 이용하여 모형의 차수를 식별할 수 있음

3. MA(q) 모형에서는 미래 충격의 조건부기댓값이 0이므로, q시점 이후의 예측값은 모형의 평균과 같아짐

4. 롤링 포캐스트는 새로운 관측값이 추가될 때마다 예측을 갱신하는 방법으로, 본 실습에서는 확장창을 이용한 1단계 예측을 수행

5. 미국 제조업 내구재 신규주문액의 차분자료를 이용한 예측 결과, MA(1) 모형의 MSE가 평균 예측 및 최근값 예측보다 작게 나타남

6. 차분된 데이터로 추정한 예측값을 원래 데이터 수준으로 복원할 때는 예측 방식에 맞는 역차분을 적용해야 함

8.5 자기회귀과정(AR)과 예측

◆ 개요

□ 앞에서는 이동평균과정(MA)을 통해 현재값이 현재 및 과거의 오차항에 의존하는 시계열 모형을 살펴봄

□ 그러나 경제 시계열에서는 현재의 관측값이 자신의 과거 관측값과 밀접하게 관련되는 경우가 많음

o 예: 고용, 생산, 소비 등은 과거의 움직임이 일정 기간 지속되는 경향이 있음

□ 자기회귀과정(autoregressive process, AR)은 현재값이 자신의 과거 관측값들에 선형적으로 의존한다고 가정하는 시계열 모형

o 과거의 자기 자신을 설명변수로 이용한다는 점에서 자기회귀모형이라고 부름

□ AR 과정은 과거 관측값 간 관계를 추정하여 미래의 값을 예측하는 데 활용

◆ MA 과정과 AR 과정의 차이

□ MA 과정과 AR 과정은 모두 시계열의 과거 정보를 이용하지만, 현재값을 설명하는 방식에 차이가 있음

□ MA 과정에서는 과거 충격의 영향이 현재값에 반영되는 반면, AR 과정에서는 과거 관측값이 현재값을 설명하는 데 직접 사용됨

□ 두 과정 모두 모형의 특성을 이해하고 적절한 차수를 선택하기 위해 자기상관구조를 분석할 필요가 있음

구분	MA(q) 과정	AR(p) 과정
현재값을 설명하는 요소	현재 및 과거 오차항	과거 관측값 및 현재 오차항
차수의 의미	과거 오차항의 개수	과거 관측값의 개수
차수 식별의 주요한 도구	ACF	PACF
이론적인 특징	ACF가 시차 q 이후 절단	PACF가 시차 p 이후 절단

◆ 학습 내용

□ 이번 절에서는 다음 내용을 살펴봄

❶ 자기회귀과정 AR(p)의 정의와 특징

❷ AR 과정의 정상성 조건

❸ ACF와 PACF를 이용한 AR 차수 식별

❹ 실제 경제자료를 이용한 AR 모형의 추정과 예측

8.5.2 자기회귀과정의 정의와 특징

(1) AR(1) 모형의 정의

□ 가장 단순한 자기회귀과정은 직전 시점의 관측값 하나를 이용하는 AR(1) 과정

o AR(1) 과정에서는 현재값이 직전 시점의 값과 현재의 오차항에 의해 결정됨

$$y_t = c + \phi_1 y_{t-1} + \epsilon_t$$

- y_t : 현재 시점의 관측값
- y_{t-1} : 직전 시점의 관측값
- c : 상수항
- ϕ_1: 자기회귀계수
- ϵ_t : 평균이 0인 자기상관이 없는 백색잡음 오차항

□ 자기회귀계수 ϕ_1은 직전 시점의 값이 현재값과 어떤 관계가 있는지를 나타냄

◆ 자기회귀계수의 부호와 크기

◇ AR(1) 과정에서 현재 발생한 충격은 자기회귀계수 ϕ_1을 통해 미래 시점으로 전달됨

◇ 현재 시점에 크기가 1인 충격이 발생하면, 이후 추가적인 충격이 없을 때 h 시점 후 충격의 영향은 ϕ_1^h로 나타남

◇ 따라서 자기회귀계수의 부호와 크기에 따라 충격의 전달방식이 달라짐

❶ $0 < \phi_1 < 1$

□ 충격의 영향이 같은 방향으로 지속되면서 점차 감소

❷ $-1 < \phi_1 < 0$

□ 충격의 영향이 양(+)과 음(-)의 방향을 번갈아 나타내면서 점차 감소

◇ 경우 모두 $|\phi_1| < 1$ 이므로 충격의 영향은 장기적으로 0에 수렴

$$\lim_{h\to\infty}\phi_1^h = 0$$

o 예: ϕ_1 = -0.5이면, 충격의 영향은 1 → -0.5 → 0.25 → -0.125 → … 로 나타남

❸ ϕ_1 = 0

□ 현재값은 직전 시점의 관측값에 의존하지 않음

o 이 경우 시계열은 상수항을 포함한 백색잡음 과정이 됨

$$y_t = c + \epsilon_t$$

<참고> ◆ AR(1) 모형에서 충격의 영향이 1, ϕ_1, ϕ_1^2, ϕ_1^3, …로 나타나는 이유

❶ AR(1) 모형에서 평균을 차감

□ AR(1) 모형은 $y_t = c + \phi_1 y_{t-1} + \epsilon_t$ (1)

□ 정상적인 AR(1) 과정의 평균을 μ 라고 하면, $\mu = c + \phi_1 \mu$ (2)

□ 식 (1)에서 식 (2)를 빼면, $y_t - \mu = c + \phi_1 (y_{t-1} - \mu) + \epsilon_t$ (3)

▸ 현재값이 평균에서 벗어나는 정도가 직전 시점의 평균과의 편차와 충격에 의해 결정

□ 편의상 $x_t = \phi_1 x_{t-1} + \epsilon_t$라고 정의하면 $x_t = \phi_1 x_{t-1} + \epsilon_t$ (4)

❷ 현재 시점에 충격이 1만큼 발생하는 경우

◇ 가정: 1. 현재 시점 t에 충격이 1만큼 발생하고, 이후 추가적인 충격은 없다.

- 시점 t에 크기가 1인 충격이 발생한다고 가정하면, $\epsilon_t = 1$

2. 충격이 발생하기 직전에는 시계열이 평균에 있다.

- 시점 t-1에서 시계열이 평균 μ에 있다고 가정하면, $y_{t-1} = \mu$
- 따라서 평균으로부터의 편차는 $x_{t-1} = y_{t-1} - \mu = 0$

□ 따라서 현재 시점의 평균으로부터 편차는 $x_t = \phi_1 x_{t-1} + \epsilon_t$ = $= \phi_1(0) + 1$ = 1

❸ 이후 추가적인 충격이 없다는 가정

□ 현재 발생한 충격이 미래에 어떻게 전달되는지만 살펴보기 위해 이후에는 새로운 충격이 발생하지 않는다고 가정

$\epsilon_{t+1} = \epsilon_{t+2} = \cdots = 0$

□ 그러면 다음 시점의 편차는 $x_{t+1} = \phi_1 x_t + 0 = \phi_1$

□ 두 시점 후에는 $x_{t+2} = \phi_1 x_{t+1} + 0 = \phi_1^2$

□ 따라서 일반적으로 h 시점 후 충격의 영향은 ϕ_1^h

◆ AR(2) 과정의 정상성

□ AR(2) 과정의 정상성은 두 자기회귀계수의 조합에 의해 결정됨

□ 따라서 AR(1)과 같이 각 계수의 절댓값이 1보다 작은 것만 확인해서는 정상성을 판단할 수 없음

□ AR(2) 과정이 정상적이기 위한 조건은 다음과 같음

$$\phi_1 + \phi_2 < 1$$

$$\phi_1 - \phi_2 < 1$$

$$-1 < \phi_1 < 1$$

□ 이러한 조건을 충족하면 충격의 영향이 장기적으로 소멸하는 정상적인 AR(2) 과정이 됨

□ AR(2)에서는 계수의 조합에 따라 자기상관이 점진적으로 감소하거나 진동하면서 감소하는 패턴이 나타날 수 있음

(3) 일반적인 AR(p) 모형

◆ AR(p) 모형의 정의

□ AR(1)과 AR(2)를 일반화하면, 과거 p개 시점의 관측값을 사용하는 AR(p) 과정을 얻게 됨

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \cdots + \phi_p y_{t-p} + \epsilon_t$$

- p : 자기회귀모형의 차수(order)
- AR(1): 과거 1개 시점 사용, AR(2): 과거 2개 시점 사용, AR(3): 과거 3개 시점 사용

□ 차수가 증가하면 더 많은 과거 관측값을 이용할 수 있지만 추정해야 할 계수의 수도 증가

□ 따라서 자료의 자기상관구조와 모형 적합도를 고려해 적절한 차수를 선택

◆ AR(p) 과정의 정상성

□ 일반적인 AR(p) 과정에서는 모든 자기회귀계수가 개별적으로 -1과 1 사이에 있어야 하는 것은 아님

□ 정상성은 자기회귀계수들의 조합에 의해 결정되며, 자기회귀 다항식

$1-\phi_1 z-\phi_2 z^2-\cdots-\phi_p z^p=0$ 의 모든 근이 단위원 밖에 존재해야 함 : $|z|>1$

□ 이는 AR(p) 과정에서 과거 충격의 영향이 시간이 지남에 따라 소멸하기 위한 조건

□ 실제 분석에서는 통계 패키지를 이용하여 추정된 모형의 정상성 여부를 확인할 수 있음

8.5.3 ACF와 PACF를 이용한 자기회귀모형의 차수 식별

(1) 자기상관함수(ACF)의 특징

◆ 자기상관함수(Autocorrelation Function, ACF)

□ 자기상관함수는 현재 관측값과 과거 관측값 사이의 상관관계를 측정하는 함수

□ 시차 k에서의 자기상관계수는 다음과 같이 정의됨

$$\rho_k = Corr(y_t, y_{t-k})$$

□ 정상적인 AR 과정에서는 과거 관측값의 영향이 여러 시점에 걸쳐 전달되므로, ACF가 점차 감소하는 특징을 보임

◆ AR(1) 과정의 ACF

□ 정상적인 AR(1) 모형을 고려 : $y_t = c + \phi_1 y_{t-1} + \epsilon_t$

□ AR(1) 과정의 시차 k에서의 자기상관계수는 다음과 같음 : $\rho_k = \phi_1^k$

□ 따라서 자기회귀계수의 부호와 크기에 따라 ACF의 형태가 달라짐

❶ 0<ϕ_1<1 : 자기상관계수가 양(+)의 값을 유지하면서 점진적으로 감소

❷ -1<ϕ_1<0 : 자기상관계수의 부호가 양(+)과 음(-)으로 번갈아 나타나면서 절댓값이 감소

(2) 부분자기상관함수(PACF)의 개념

◆ 부분자기상관함수(Partial Autocorrelation Function, PACF)

□ ACF는 현재값과 과거값 사이의 전체 상관관계를 측정하므로, 중간 시차를 통해 발생하는 간접적인 상관관계까지 포함할 수 있음

□ 예: AR(1) 모형에서는 현재값 y_t가 직전 시점의 값 y_{t-1}에 의존하고, y_{t-1}은 다시 y_{t-2}에 의존

□ 따라서 y_t와 y_{t-2}사이에도 상관관계가 나타날 수 있지만, 이는 y_{t-1}을 통한 간접적인 관계일 수 있음

□ 부분자기상관함수(PACF)는 중간 시차의 영향을 통제한 후 현재값과 특정 과거값 사이에 남아 있는 상관관계를 측정

◆ ACF와 PACF의 차이

□ 시차 2에서 두 함수의 차이를 비교하면 다음과 같음

구분	측정하는 관계
ACF(2)	y_t와 y_{t-2} 사이의 전체 상관관계
PACF(2)	y_{t-1}의 영향을 통제한 후 y_t와 y_{t-2} 사이에 남아 있는 상관관계

□ PACF(3)은 y_{t-1}와 y_{t-2}의 영향을 통제한 후 y_t와 y_{t-3} 사이에 남아 있는 상관관계를 의미

□ 따라서 PACF를 이용하면 특정 과거 시점의 관측값이 현재값을 설명하는 데 추가적인 정보를 제공하는지 확인할 수 있음

(3) PACF를 이용한 AR 차수 식별

◆ AR(p) 과정의 PACF

□ 정상적인 AR(p) 과정에서는 현재값이 과거 p개 시점의 관측값에 직접적으로 의존

□ 따라서 이론적으로 PACF는 시차 p 이후 0이 되는 특징을 가지며, 이를 PACF의 절단(cutoff)이라고 부름

$$PACF(k) = 0, \quad k > p$$

□ 예를 들어 다음과 같이 해석할 수 있음

모형	PACF의 이론적 특징
AR(1)	시차 1 이후 0
AR(2)	시차 2 이후 0
AR(3)	시차 3 이후 0

◆ AR(2) 과정의 예

□ 다음과 같은 AR(2) 모형을 고려

$$y_t = c + \phi_1 y_{t-1} + \phi_2 y_{t-2} + \epsilon_t$$

□ 현재값은 1시점 전과 2시점 전의 관측값에 직접적으로 의존

□ 그러나 3시점 전의 관측값은 이미 y_{t-1}과 y_{t-2}를 통해 전달되는 정보이므로, 이들을 통제하면 추가적인 선형관계가 남지 않음

□ 따라서 이론적으로 AR(2) 과정에서는 PACF(1), PACF(2)가 0이 아닐 수 있지만 PACF(3) 이후는 0임

(4) ACF와 PACF를 이용한 모형 식별

◆ AR 과정과 MA 과정의 비교

구분	ACF	PACF
AR(p)	점진적으로 감소	시차 p 이후 절단
MA(q)	시차 q 이후 절단	점진적으로 감소

□ AR 과정에서는 ACF가 점진적으로 감소하는 반면, PACF가 특정 시차 이후 절단되는 특징을 보임

□ MA 과정에서는 반대로 ACF가 특정 시차 이후 절단되는 특징을 보임

□ 따라서 ACF와 PACF를 함께 분석하면 시계열의 자기상관구조를 파악하고 후보 모형을 선정하는 데 도움이 됨

◆ 실제 데이터 분석 시 유의사항

□ 앞서 설명한 절단 패턴은 이론적인 자기상관함수와 부분자기상관함수의 특징

□ 실제 표본에서 계산한 ACF와 PACF는 표본오차의 영향을 받으므로 특정 시차 이후 정확히 0이 되지 않을 수 있음

□ 일반적으로 ACF·PACF 그래프의 신뢰구간을 이용하여 각 시차의 상관계수가 통계적으로 유의한지 판단

□ 다만 일부 시차에서 유의한 값이 나타난다는 사실만으로 해당 차수를 확정할 수는 없음

□ 따라서 ACF와 PACF를 이용하여 후보 차수를 선정한 후 AIC·BIC, 잔차 진단 및 예측 성능을 함께 검토할 필요가 있음

◆ 정리

❶ ACF는 현재값과 과거값 사이의 전체 상관관계를 측정

❷ PACF는 중간 시차의 영향을 통제한 후 현재값과 특정 과거값 사이에 남아 있는 상관관계를 측정

❸ 정상적인 AR(p) 과정의 ACF는 일반적으로 점차 감소하고, PACF는 시차 p 이후 절단

❹ 따라서 PACF는 AR 모형의 차수를 결정하는 데 중요한 정보를 제공

8.5.4 Python 실습: 미국 비농업부문 취업자 수의 정상성 검정과 차분

8.5.4.1 분석 목적과 자료

◆ 분석 목적

□ 미국 비농업부문 취업자 수를 이용하여 실제 경제 시계열의 정상성을 검정하고, 자기회귀 모형을 적용하기 위한 자료를 준비

□ 분석은 다음 순서로 진행

❶ FRED에서 월별 취업자 수 자료 수집

❷ 시계열 그래프를 이용한 추세 및 변동 특성 확인

❸ ADF 검정을 이용한 정상성 검정

❹ 1차 차분을 통한 자료 변환

❺ 차분자료의 정상성 재검정

◆ 사용 자료

□ 자료: 미국 비농업부문 취업자 수(Total Nonfarm Payroll Employment)

□ 분석기간: 2000년 1월~2019년 12월

□ 자료 빈도 및 단위: 월별(계절조정), 천명. 관측값 240개

□ 자료 출처: FRED

□ 데이터 코드: PAYEMS

□ 비농업부문 취업자 수는 미국의 고용 상황을 파악하는 대표적인 경제지표

□ 이번 실습에서는 코로나19에 따른 급격한 고용 변동을 제외하기 위해 분석기간을 2019년 12월까지로 제한

8.5.4.2 데이터 불러오기

```
import pandas_datareader.data as web
import pandas as pd
import matplotlib.pyplot as plt

# FRED에서 비농업부문 취업자 수 불러오기
df = web.DataReader( "PAYEMS", "fred", "2000-01-01", "2019-12-31" )

# 변수명 변경
df.columns = ["employment"]

# 월별 날짜 인덱스 설정
df = df.asfreq("MS")

# 데이터 확인
print(df.head())
print(df.shape)
print(df.isna().sum()))
```

```
            employment
DATE
2000-01-01      131011
 ⋮
2000-05-01      132105
(240, 1)
employment      0
dtype: int64
```

◇ 코드 설명

□ PAYEMS: 미국 비농업부문 취업자 수를 나타내는 FRED 데이터 코드

□ df.columns = [“employment“]: 분석의 편의를 위해 변수명을 employment로 변경

□ asfreq(“MS“): 월별 시계열의 빈도를 매월 시작일(Month Start)로 지정

□ df.isna().sum(): 결측값의 개수를 확인

❷ CSV 파일로 저장

```
df.to_csv("US_Nonfarm_Employment.csv")
```

❸ 저장한 파일을 불러오기

```
df = pd.read_csv( "US_Nonfarm_Employment.csv", index_col="DATE", parse_dates=True )
df = df.asfreq("MS")
df.head()
```

◇ 코드 설명

□ to_csv(): 데이터프레임을 CSV 파일로 저장

□ index_col="DATE": 날짜 열을 데이터프레임의 인덱스로 지정

□ parse_dates=True: 날짜 정보를 문자열이 아닌 날짜 형식으로 변환

□ asfreq("MS"): 월별 시계열의 빈도를 지정

8.5.4.3 취업자 수의 시계열 그래프

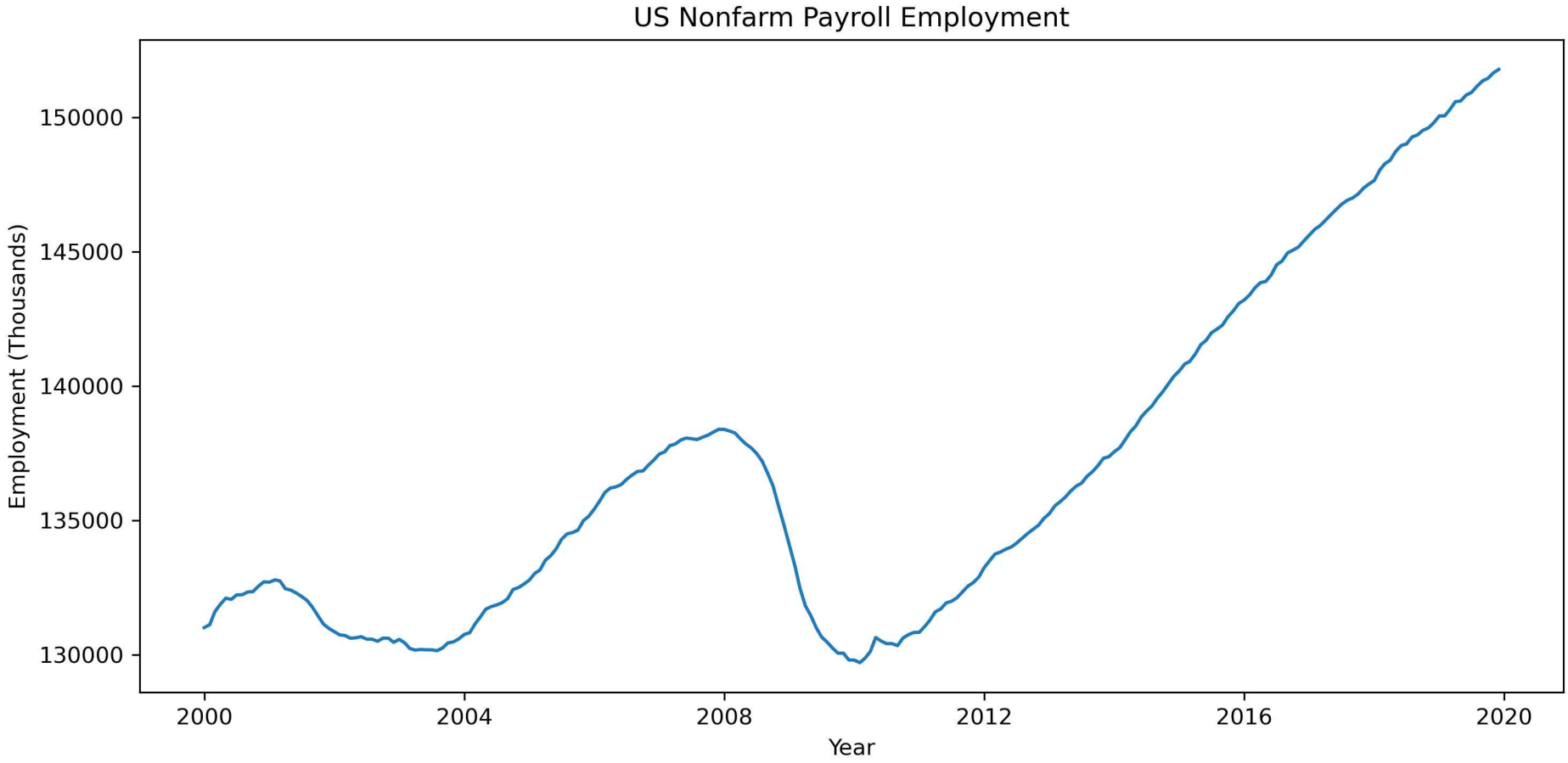
```
import matplotlib.pyplot as plt

fig, ax = plt.subplots(figsize=(10, 5))

ax.plot(df.index, df["employment"])

ax.set_xlabel("Year")
ax.set_ylabel("Employment (Thousands)")
ax.set_title("US Nonfarm Payroll Employment")

plt.tight_layout()
plt.show()
```



◆ 시계열 그래프 분석 결과

□ 비농업부문 취업자 수는 경기에 따라 증가 또는 하락하면서 전반적으로는 증가하는 추세

□ 따라서 취업자 수 수준자료는 시간에 따라 평균이 변화할 가능성이 있으며, 정상성을 만족하지 않을 수 있으므로, 이를 확인하기 위해 ADF 검정을 실시

8.5.4.4 ADF 검정을 이용한 정상성 검정

◆ ADF 검정(Augmented Dickey-Fuller Test)

□ ADF 검정은 시계열에 단위근이 존재하는지 검정하는 대표적인 방법

□ 귀무가설과 대립가설은 다음과 같음

H_0 : 단위근이 존재함

H_1 : 단위근이 존재하지 않음

□ 일반적으로 p-값이 0.05보다 작으면 5% 유의수준에서 단위근 귀무가설을 기각

□ 반대로 p-값이 0.05보다 크면 단위근 귀무가설을 기각할 수 없음

```
from statsmodels.tsa.stattools import adfuller

ADF_result = adfuller(df[“employment“])

print(f“ADF Statistic: {ADF_result[0]:.4f}“)
print(f“p-value: {ADF_result[1]:.4f}“)
```

◇ 결과 해석: 수준자료의 정상성 검정

- ADF Statistic: -0.0974, p-value: 0.9497
- ADF 검정의 p-값이 5% 유의수준보다 훨씬 크므로 귀무가설을 기각할 수 없음
- 이는 수준자료가 정상적이라는 통계적 근거를 확보하지 못했음을 의미
- 따라서 정상적인 AR 모형을 적용하기 위해 1차 차분을 고려

8.5.4.5 1차 차분의 개념과 적용

◆ 1차 차분(First Difference)

□ 1차 차분은 현재 시점의 관측값에서 직전 시점의 관측값을 차감하는 방법

□ 현재 월의 취업자 수에서 전월의 취업자 수를 차감하여 월별 취업자 수 변화량을 계산

```
# 1차 차분
employment_diff = df["employment"].diff().dropna()
print(employment_diff.head())
print(len(employment_diff))
```

◆ 코드 설명

□ diff(): 현재 관측값에서 직전 관측값을 차감하여 1차 차분값 계산

□ dropna(): 차분 과정에서 발생하는 첫 번째 결측값 제거

□ 원래 관측값은 240개이지만, 1차 차분 후에는 239개가 남음

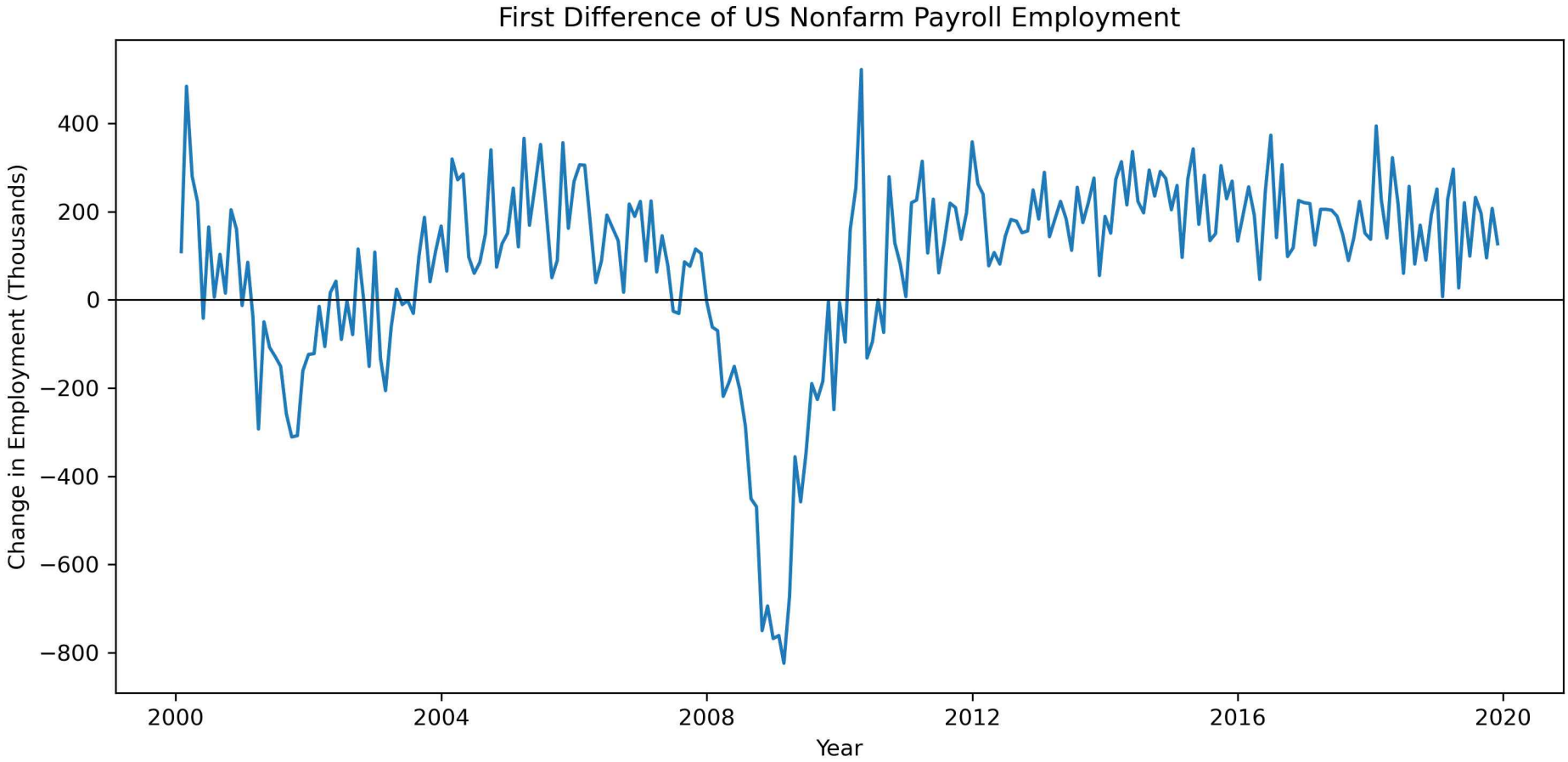
8.5.4.6 차분자료의 시계열 그래프

```
ax.plot(employment_diff.index, employment_diff)

ax.axhline(0, color="black", linewidth=0.8)

ax.set_xlabel("Year")
ax.set_ylabel("Change in Employment (Thousands)")
ax.set_title("First Difference of US Nonfarm Payroll Employment")

plt.tight_layout()
plt.show()
```


First Difference of US Nonfarm Payroll Employment
Change in Employment (Thousands)
400
200
0
−200
−400
−600
−800
2000
2004
2008
2012
2016
2020
Year

◆ 그래프 분석

□ 차분 후에는 수준자료에서 관찰되었던 장기적인 증가 추세가 제거되고, 월별 취업자 수의 증감이 나타남

o 글로벌 금융위기 기간에는 취업자 수가 큰 폭으로 감소하면서 음(-)의 변화량이 지속

o 금융위기 이후에는 고용이 회복되면서 양(+)의 변화량이 지속

→ 이는 취업자 수 변화량에도 과거의 움직임이 일정 기간 지속되는 자기상관이 존재할 가능성을 시사

□ 차분자료의 정상성 여부는 그래프만으로 판단할 수 없으므로 ADF 검정을 다시 실시

8.5.4.7 차분자료의 정상성 재검정

```
ADF_result = adfuller(employment_diff)

print(f"ADF Statistic: {ADF_result[0]:.4f}")
print(f"p-value: {ADF_result[1]:.4f}")
```

◇ 결과 해석: 1차 차분 후 정상성 검정

- ADF Statistic: -2.9057, p-value: 0.0447
- p-값이 0.05보다 작으므로 5% 유의수준에서 단위근 귀무가설을 기각
- 따라서 수준자료와 달리 차분자료는 단위근이 존재하지 않는다는 통계적 근거를 확보
- 다만 p-값이 0.05에 가까우므로 검정 결과가 강력하다고 보기는 어려우며, 다른 검정 설정이나 표본기간에서는 결론이 달라질 가능성도 있음

8.5.5 ACF와 PACF를 이용한 AR 모형의 차수 식별

8.5.5.1 분석 목적

□ 앞에서는 미국 비농업부문 취업자 수의 1차 차분자료에 대해 정상성 검정을 실시함

□ 이제 차분된 취업자 수의 자기상관구조를 분석하여 적절한 자기회귀모형의 차수를 결정

□ 이를 위해 자기상관함수(ACF)와 부분자기상관함수(PACF)를 이용

- o ACF: 현재의 취업자 수 변화량과 과거 변화량 사이의 전체 상관관계를 확인
- o PACF: 중간 시차의 영향을 통제한 후 특정 과거 변화량이 현재 변화량과 갖는 추가적인 상관관계를 확인

□ 분석 결과를 바탕으로 AR 모형의 후보 차수를 선정

8.5.5.2 ACF와 PACF 그래프 작성

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

fig, axes = plt.subplots(1, 2, figsize=(12, 4))

# 자기상관함수

plot_acf( employment_diff, lags=24, ax=axes[0] )

# 부분자기상관함수

plot_pacf( employment_diff, lags=24, method="ywm", ax=axes[1] )

axes[0].set_title("ACF of Differenced Employment")

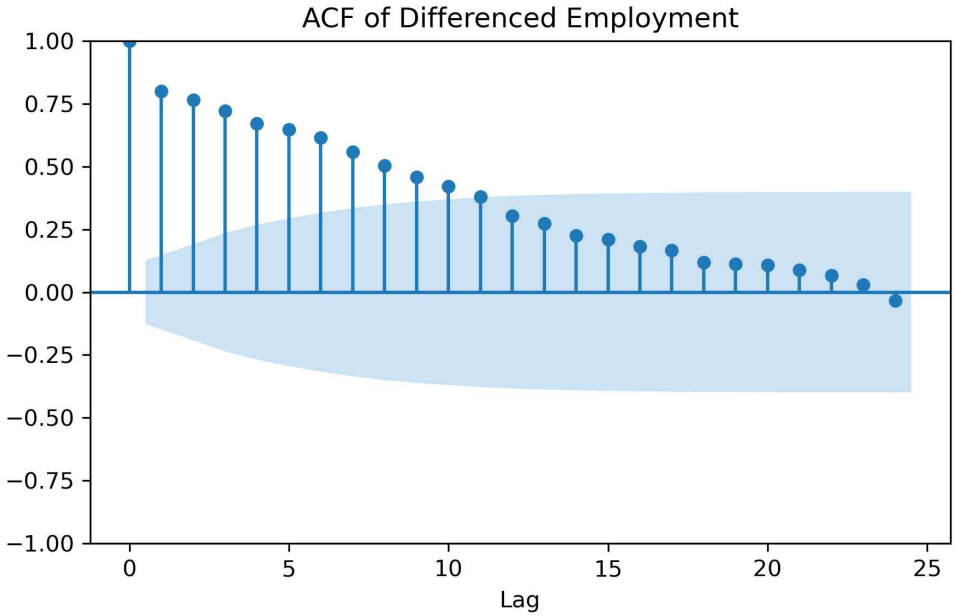
axes[1].set_title("PACF of Differenced Employment")

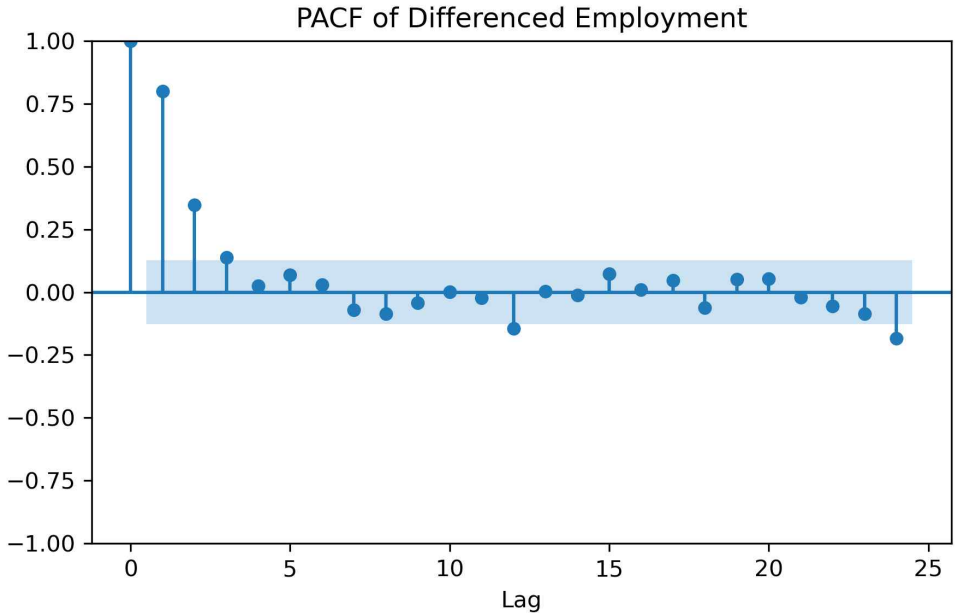
axes[0].set_xlabel("Lag")

axes[1].set_xlabel("Lag")

plt.tight_layout()

plt.show()
```


ACF of Differenced Employment
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
Lag


PACF of Differenced Employment
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
0
5
10
15
20
25
Lag

◆ 코드 설명

□ plot_acf(): 시차별 자기상관계수를 계산하여 그래프로 나타내는 함수

□ plot_pacf(): 시차별 부분자기상관계수를 계산하여 그래프로 나타내는 함수

□ lags=24: 최대 24개월 전까지의 자기상관과 부분자기상관을 분석

□ method="ywm": 자기상관계수를 이용하는 Yule-Walker 방법*으로 PACF를 추정

* 자기공분산을 계산할 때 전체 관측값의 개수 N으로 나누는 방식을 사용

□ ACF와 PACF 그래프의 음영 영역은 약 95% 신뢰구간을 나타냄

o 막대가 신뢰구간을 벗어나면 해당 시차의 상관계수가 0과 통계적으로 다르다는 근거임

o 그러나 여러 시차를 동시에 분석하면, 일부 막대는 우연히 신뢰구간을 벗어날 수 있음

□ 따라서 신뢰구간을 벗어나는 막대의 개수만으로 AR 모형의 차수를 결정하지 않으며, 유의한 시차가 나타나는 위치와 전체적인 절단 패턴을 함께 고려

□ 최종적으로는 모형 추정 후 Ljung-Box 검정 등을 이용하여 잔차의 자기상관 여부를 확인

8.5.5.3 ACF 분석 결과

◆ 그래프 해석

□ 차분된 취업자 수의 ACF에서 시차 1의 자기상관계수는 약 0.8로 높은 양(+)의 값을 보임

□ 이후 시차가 증가하면서 자기상관계수가 점진적으로 감소하는 패턴을 나타냄

□ 이는 현재 월의 취업자 수 변화량이 직전 월의 변화량과 밀접하게 관련되어 있으며, 그 관계가 여러 달에 걸쳐 지속될 수 있음을 의미

□ 특히 ACF가 특정 시차에서 갑자기 0에 가까워지는 것이 아니라 서서히 감소하는 모습은 AR 과정에서 나타나는 전형적인 특징과 부합

□ 따라서 차분된 취업자 수에 자기회귀모형을 적용하는 것이 적절한 후보가 될 수 있음

8.5.5.4 PACF 분석 결과

◆ 그래프 해석

□ PACF에서는 시차 1, 2, 3의 부분자기상관계수가 신뢰구간을 벗어나는 것으로 나타남

□ 시차 4 이후에는 대부분의 부분자기상관계수가 신뢰구간 안에 위치

□ 이는 과거 1~3개월의 취업자 수 변화량이 현재의 변화량을 설명하는 데 추가적인 정보를 제공할 가능성을 시사

□ 반면 시차 4 이후의 과거 변화량은 앞선 시차들을 통제하면 추가적인 설명력이 상대적으로 작을 수 있음

□ 따라서 PACF가 시차 3 이후 대체로 절단되는 패턴을 보이므로 AR(3)을 우선적인 후보 모형으로 선정

◆ ACF와 PACF 분석 결과 종합

분석 도구	주요 결과	해석
ACF	시차가 증가하면서 점진적으로 감소	AR 과정의 가능성을 시사
PACF	시차 1~3에서 유의한 부분자기상관	AR(3)을 후보 모형으로 고려
종합	ACF는 점진적 감소, PACF는 시차 3 이후 대체로 절단	AR(3) 모형을 우선 추정

□ 다만 ACF와 PACF는 후보 모형을 식별하기 위한 도구이므로, AR(3)이 최적 모형이라는 결론을 의미하지는 않음

□ 실제 추정 후에는 다른 차수의 모형과 정보기준 및 예측 성능을 비교할 필요가 있음

8.5.5.5 AR(3) 모형의 설정

◆ 분석 대상의 정의

□ y_t는 t월의 미국 비농업부문 취업자 수를 의미

□ 앞에서 수준자료의 정상성이 확인되지 않았으므로, 이번 분석에서는 1차 차분한 취업자 수 변화량을 사용

$$\Delta y_t = y_t - y_{t-1}$$

□ 따라서 Δy_t는 t월의 취업자 수가 전월보다 얼마나 증가하거나 감소하였는지를 나타냄

◆ AR(3) 모형

□ PACF 분석 결과를 바탕으로 과거 3개월의 취업자 수 변화량을 이용하여 현재의 변화량을 설명하는 AR(3) 모형을 설정

$$\Delta y_t = c + \phi_1 \Delta y_{t-1} + \phi_2 \Delta y_{t-2} + \phi_3 \Delta y_{t-3} + \epsilon_t$$

- Δy_t : 현재 월의 취업자 수 변화량, Δy_{t-1} : 1개월 전 취업자 수 변화량
- Δy_{t-2} : 2개월 전 취업자 수 변화량, Δy_{t-3} : 3개월 전 취업자 수 변화량
- c : 상수항
- $\phi_{1,}\phi_{2,}\phi_3$: 자기회귀계수
- ϵ_t : 과거 변화량으로 설명되지 않는 현재의 새로운 충격

◆ 모형의 경제적 의미

□ 기업의 고용 조정이 여러 달에 걸쳐 이루어질 수 있으므로, 과거의 취업자 수 변화량은 현재의 변화량을 예측하는 데 유용한 정보를 제공할 수 있음

□ 그러나 자기회귀계수는 과거와 현재의 고용 변화 사이의 통계적 관계를 나타내며, 인과효과를 의미하지는 않음

◆ AR(3) 모형을 이용한 예측

□ AR(3) 모형을 추정하여 계수 $\hat{c}, \hat{\phi}_1, \hat{\phi}_2, \hat{\phi}_3$를 얻으면 다음 달의 취업자 수 변화량을 예측할 수 있음

$$\widehat{\Delta y}_{t+1|t} = \hat{c} + \hat{\phi}_1 \Delta y_t + \hat{\phi}_2 \Delta y_{t-1} + \hat{\phi}_3 \Delta y_{t-2}$$

o $\widehat{\Delta y}_{t+1|t}$는 t월까지 정보를 이용하여 예측한 t + 1월의 취업자 수 변화량

□ 예측에는 가장 최근 3개월의 실제 취업자 수 변화량이 사용됨

□ 이 예측값은 취업자 수의 수준이 아니라 변화량이므로, 취업자 수 수준을 예측하려면 현재의 취업자 수에 예측 변화량을 더해야 함

$$\hat{y}_{t+1|t} = y_t + \widehat{\Delta y}_{t+1|t}$$

◆ 정리

❶ 차분된 미국 비농업부문 취업자 수의 ACF는 시차가 증가함에 따라 점진적으로 감소

❷ PACF에서는 시차 1~3의 부분자기상관이 유의하게 나타나 AR(3)을 후보 모형으로 선정

❸ AR(3) 모형은 과거 3개월의 취업자 수 변화량을 이용하여 현재 또는 미래의 변화량을 설명하는 모형

❹ 추정된 AR(3) 모형으로 다음 달 취업자 수 변화량을 예측하고, 이를 현재 취업자 수에 더하면 다음 달 취업자 수 수준의 예측값을 얻을 수 있음

❺ AR(3)의 적절성은 실제 모형 추정 후 다른 차수의 모형과 비교하여 평가해야 함

◆ MA 과정과 AR 과정의 차이

□ MA(q) 과정 → ACF가 lag q 이후 급격히 0에 가까워짐

□ AR(p) 과정 → ACF가 천천히 감소(decay)

<참고> **ACF 그래프의 시차 0**

□ 시차 0(lag 0)은 제외하고 시차 1부터 봄

o 자기상관은 현재의 나(y_t)와 과거의 나(y_{t-1}) 사이의 상관관계인데 시차 0은 현재의 나(y_t)와 현재의 나(y_t)를 비교하는 것이므로 시차 0(lag 0), 즉 자기 자신과의 상관관계는 언제나 1

→ 시차 0은 데이터의 특성 정보가 아니므로 상수로 간주하여 분석에서 제외

□ 모형의 성격(AR, MA)을 알려주는 데이터는 시차 1(lag 1)부터 시작

8.5.6 AR(3) 모형의 추정과 예측

□ 이번 절에서는 Python의 AutoReg()를 이용하여 AR(3) 모형을 추정하고, AIC와 BIC를 통해 적절한 모형의 차수를 검토

□ 이어서 과거 평균값(Mean), 최근 값(Naive), AR(3) 모형의 세 가지 방법으로 미국 비농업부문 취업자 수 변화량을 예측하고, MAE와 RMSE를 이용하여 예측 성능을 비교·평가

8.5.6.1 AR(3) 모형의 추정

◆ AR(3) 모형의 설정

□ 앞 절의 ACF와 PACF 분석 결과를 바탕으로 취업자 수 변화량 Δy_t에 AR(3) 모형을 적용

$$\Delta y_t = c + \phi_1 \Delta y_{t-1} + \phi_2 \Delta y_{t-2} + \phi_3 \Delta y_{t-3} + \epsilon_t$$

□ AutoReg() 함수를 이용하여 자기회귀계수를 추정하고, 과거 취업자 수 변화량과 현재 변화량 사이의 관계를 분석

◆ Python 코드

```
from statsmodels.tsa.ar_model import AutoReg

# AR(3) 모형 추정
model_ar3 = AutoReg( employment_diff, lags=3, trend="c" )

result_ar3 = model_ar3.fit()

# 추정 결과 출력
print(result_ar3.summary())
```

◆ 코드 설명

□ 'AutoReg()': 자기회귀모형(AR)을 설정하는 함수

- o 'employment_diff': 분석 대상인 취업자 수 변화량
- o 'lags=3': 과거 3개월의 변화량을 설명변수로 사용
- o 'trend="c"': 상수항을 포함

AutoReg Model Results

Dep. Variable:	employment	No. Observations:	239
Model:	AutoReg(3)	Log Likelihood	-1459.973
Method:	Conditional MLE	S.D. of innovations	117.612
Date:	Wed, 14 Oct 2026	AIC	2929.947
Time:	16:16:33	BIC	2947.266
Sample:	05-01-2000 - 12-01-2019	HQIC	2936.928

	coef	std err	z	P>|z|	[0.025	0.975]
const	8.3973	8.331	1.008	0.314	-7.932	24.727
employment.L1	0.4707	0.064	7.305	0.000	0.344	0.597
employment.L2	0.2772	0.068	4.075	0.000	0.144	0.411
employment.L3	0.1398	0.063	2.207	0.027	0.016	0.264

Roots

	Real	Imaginary	Modulus	Frequency
AR.1	1.0750	-0.0000j	1.0750	-0.0000
AR.2	-1.5288	-2.0774j	2.5793	-0.3510
AR.3	-1.5288	+2.0774j	2.5793	0.3510

◆ AR(3) 모형의 추정 결과

□ 추정된 AR(3) 모형은 $\widehat{\Delta y}_t = 8.397 + 0.471\Delta y_{t-1} + 0.277\Delta y_{t-2} + 0.140\Delta y_{t-3}$

□ 세 자기회귀계수 모두 양(+)이며 각 계수의 유의확률이 0.05보다 작아 해당 계수가 0이라는 귀무가설을 5% 유의수준에서 기각

o 취업자 수 변화량에는 양(+)의 자기상관이 존재하며, 특히 직전 월의 영향이 가장 크고 시차가 길어질수록 영향이 작아짐

□ AR 모형의 특성근은 특성근(Roots)을 통해 확인할 수 있으며, 모든 특성근의 절댓값이 1보다 커야 함

o 추정된 AR(3) 모형의 특성근 절댓값은 1.075, 2.579, 2.579로 모두 1보다 크게 나타남

□ 따라서 추정된 AR(3) 모형은 정상성 조건을 충족하며, 충격의 영향은 장기적으로 소멸

⇒ 종합하면, 추정 결과는 앞 절에서 PACF를 이용하여 AR(3)을 후보 모형으로 선정한 결과와 부합

8.5.6.2 AR 모형의 차수 비교

◆ 정보기준을 이용한 모형 비교

□ PACF에서는 AR(3)을 후보 모형으로 선정하였지만, AR(1)이나 AR(2)가 더 적절할 수도 있음

□ 따라서 AR(1), AR(2), AR(3) 모형을 각각 추정하고 정보기준을 비교

o 정보기준: AIC(Akaike Information Criterion)와 BIC(Bayesian Information Criterion)

$$AIC = -2\ln L + 2k, \qquad BIC = -2\ln L + k\ln T$$

o L은 최대화된 우도, k는 추정 모수의 수, T는 추정에 사용된 관측값의 수를 의미

□ AIC와 BIC는 모형의 적합도와 복잡성을 함께 고려하는 정보기준

o 모형의 적합도가 높을수록 정보기준이 작아지는 경향

o 추정 모수의 수가 많아질수록 벌점(penalty)을 부과하여 복잡한 모형의 선택을 억제

□ 일반적으로 AIC 또는 BIC가 작은 모형을 선호

◆ Python 코드

```
results = []

for p in [1, 2, 3]:
    model = AutoReg( employment_diff, lags=p, trend="c", hold_back=3 )

    fitted = model.fit()

    results.append({ "Model": f"AR({p})", "AIC": fitted.aic, "BIC": fitted.bic })

comparison = pd.DataFrame(results)

print(comparison)
```

	Model	AIC	BIC
0	AR(1)	2961.914071	2972.305566
1	AR(2)	2932.769024	2946.624352
2	AR(3)	2929.946712	2947.265871

◆ 코드 설명

◇ AR(1)~AR(3) 가운데 상대적으로 적합한 모형을 선택하는 과정

□ ‘for p in [1, 2, 3]‘: AR(1), AR(2), AR(3) 모형을 차례로 추정

□ ‘hold_back=3‘: 모든 모형에서 처음 3개 관측값을 제외하여 동일한 표본을 사용

o AR(1)은 첫 번째 관측값만 제외하지만, AR(3)은 처음 3개 관측값을 제외해야 함

o 따라서 AR 모형의 차수에 따라 필요한 과거 관측값의 개수가 다르므로, 정보기준을 비교할 때 표본을 통일

□ ‘fitted.aic‘, ‘fitted.bic‘: 각 모형의 정보기준을 추출

□ AIC와 BIC가 가장 작은 모형을 확인하여 AR(3) 모형의 상대적인 적합성을 평가

8.5.6.3 세 가지 예측방법의 설정

◆ 예측 방법의 비교

□ AR(3) 모형의 예측 성능을 평가하기 위해 두 가지 단순 예측방법을 비교 대상으로 설정

❶ 과거 평균값(Mean)을 이용한 예측

□ 예측 시점까지 관측된 취업자 수 변화량의 평균을 다음 달의 예측값으로 사용

□ 가정: 과거의 평균적인 고용 증가 수준이 다음 달에도 유지된다.

❷ 최근 값(Naive)을 이용한 예측

□ 직전 월의 취업자 수 변화량을 다음 달의 예측값으로 사용

□ 가정: 가장 최근에 관측된 변화량이 다음 달에도 그대로 나타난다.

❸ AR(3) 모형을 이용한 예측

□ 과거 3개월의 취업자 수 변화량과 추정된 자기회귀계수를 이용하여 다음 달의 변화량을 예측

$$\widehat{\Delta y}_{t+1|t} = \hat{c} + \hat{\phi}_1 \Delta y_t + \hat{\phi}_2 \Delta y_{t-1} + \hat{\phi}_3 \Delta y_{t-2}$$

□ 과거 변화량 사이의 자기상관구조를 반영한다는 점이 특징

8.5.6.4 세 가지 예측방법의 실증분석

□ 모형의 예측 성능을 평가하기 위해 전체 자료를 훈련자료와 검증자료로 구분

- o 훈련자료: 모형 추정에 사용. 2000.2월~2017.12월
- o 검증자료: 모형의 예측 성능을 평가하는 데 사용. 2018.1월~2019.12월(24개월)

◆ Python 코드: 순차적 1개월 예측

```
# 훈련자료와 검증자료 구분
train = employment_diff.iloc[:-24]
test = employment_diff.iloc[-24:]

pred_mean = []
pred_naive = []
pred_ar3 = []

for i in range(len(train), len(employment_diff)):

    # 예측 시점 이전까지의 실제 자료
    history = employment_diff.iloc[:i]

    # ① 과거 평균값
    pred_mean.append(history.mean())
```

```
    # ② 최근 값
    pred_naive.append(history.iloc[-1])

    # ③ AR(3) 모형
    model = AutoReg(history, lags=3, trend="c")
    fitted = model.fit()
    pred_ar3.append( fitted.forecast(steps=1).iloc[0] )

# 실제값과 예측값 정리
forecast_df = pd.DataFrame({
    "Actual": test, "Mean": pred_mean, "Naive": pred_naive, "AR(3)": pred_ar3, index=test.index)

print(forecast_df.head())
```

```
              Actual        Mean   Naive        AR(3)
DATE
2018-01-01     137.0   76.800000   151.0   160.741804
 ⋮
2018-05-01     322.0   79.493151   140.0   180.562403
```

◆ 코드 설명

□ history = employment_diff.iloc[:i]: 예측 시점 이전까지의 실제 자료만 사용

□ history.mean(): 과거 평균값을 다음 달 예측값으로 사용

□ history.iloc[-1]: 가장 최근의 실제 변화량을 다음 달 예측값으로 사용

□ AutoReg(history, lags=3): 과거 3개월의 변화량을 이용하는 AR(3) 모형을 추정

□ forecast(steps=1): 다음 1개월의 취업자 수 변화량을 예측

□ 매월 새로운 실제 관측값이 추가되면 이를 반영하여 예측을 반복하는 방식을 순차적 1단계 예측(rolling one-step-ahead forecast)이고 함

8.5.6.5 세 가지 예측방법의 시각화

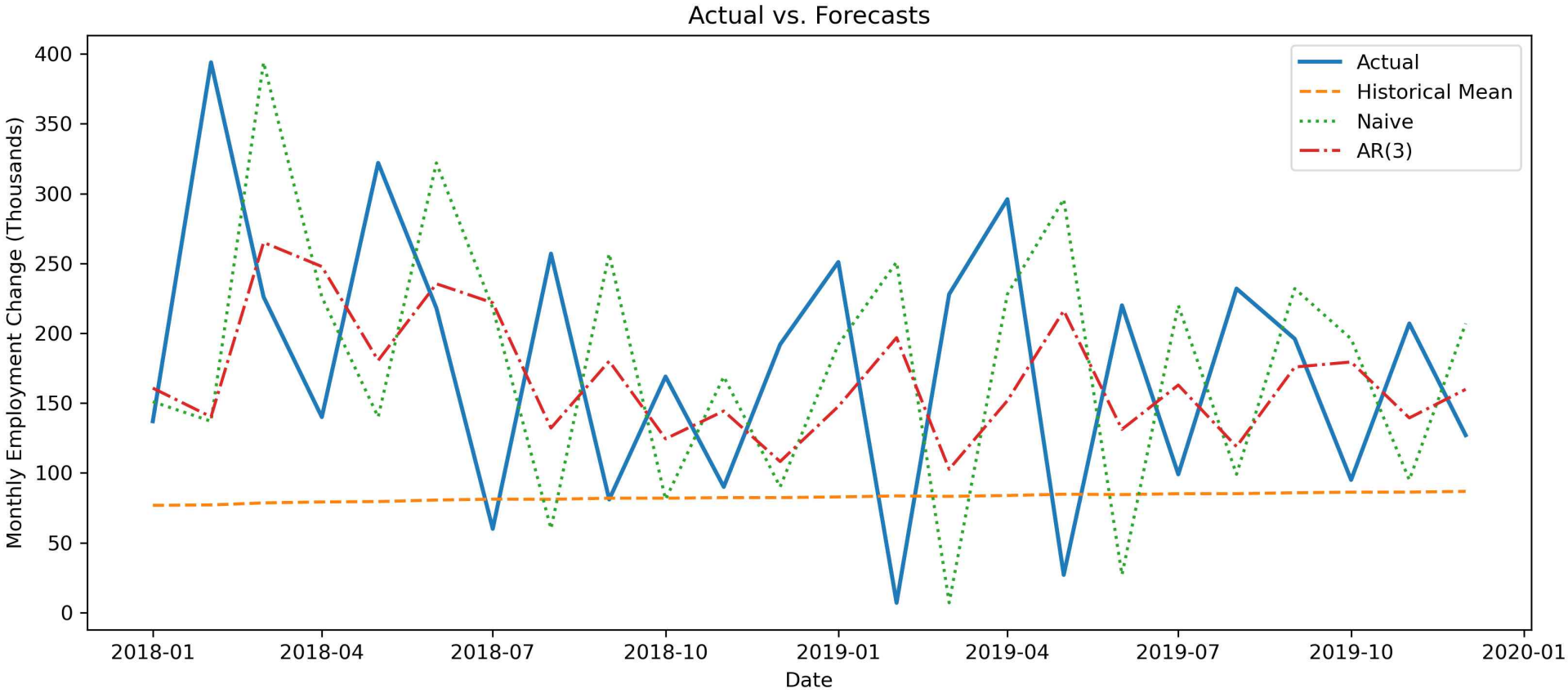
```
import matplotlib.pyplot as plt

plt.figure(figsize=(11, 5))

plt.plot( forecast_df.index, forecast_df["Actual"], label="Actual", linewidth=2 )
plt.plot( forecast_df.index, forecast_df["Mean"], label="Historical Mean", linestyle="--" )
plt.plot( forecast_df.index, forecast_df["Naive"], label="Naive", linestyle=":" )
plt.plot( forecast_df.index, forecast_df["AR(3)"], label="AR(3)", linestyle="-." )

plt.title("Actual vs. Forecasts")
plt.xlabel("Date")
plt.ylabel("Monthly Employment Change (Thousands)")

plt.legend()
plt.tight_layout()
plt.show()
```


Actual vs. Forecasts
Actual
Historical Mean
Naive
AR(3)
Monthly Employment Change (Thousands)
400
350
300
250
200
150
100
50
0
2018-01
2018-04
2018-07
2018-10
2019-01
2019-04
2019-07
2019-10
2020-01
Date

◆ 예측 결과 그래프의 해석

□ 과거 평균값(Historical Mean)은 약 8만 명 수준에서 거의 일정하게 유지되어 실제 취업자 수 증가폭을 전반적으로 과소예측

□ 최근 값(Naive)은 실제값의 움직임을 한 달 늦게 따라가는 경향을 보이며, 취업자 수 증가폭이 급격히 변하는 시점에서 실제값과 큰 차이를 나타냄

□ AR(3) 모형은 최근 값 예측보다 완만하게 움직이며, 실제 취업자 수 변화량의 일부 변동을 반영하지만 급격한 상승과 하락은 충분히 예측하지 못함

□ 종합하면, AR(3)은 극단적인 변동을 완화한 예측값을 제시하는 반면, 과거 평균값은 지나치게 평탄하고 최근 값 예측은 실제 변동에 한 달 늦게 반응하는 특징을 보임.

□ 다만 그래프만으로 세 방법의 예측 정확성을 판단하기 어려우므로 예측오차를 계산하여 비교할 필요가 있음

8.5.6.6 MAE와 RMSE를 이용한 예측 성능 비교

◆ 예측오차의 평가

□ 실제 취업자 수 변화량과 예측값의 차이를 예측오차라고 정의

$$e_t = \Delta y_t - \widehat{\Delta y}_{t|t-1}$$

□ 대표적인 예측오차 지표는 평균절대오차(MAE)와 평균제곱근오차(RMSE)

$$MAE = \frac{1}{n}\sum_{t=1}^{n}|e_t|$$

$$\mathrm{SE} = \sqrt{\frac{1}{\mathrm{n}}\sum_{\mathrm{t}=1}^{\mathrm{n}}\mathrm{e}_{\mathrm{t}}^{2}}$$

□ 두 지표 모두 값이 작을수록 예측오차가 작음을 의미

□ RMSE는 오차를 제곱하므로 큰 예측오차에 상대적으로 더 민감하게 반응

◆ Python 코드

```
import numpy as np
from sklearn.metrics import mean_absolute_error, mean_squared_error

results = []

for method in ["Mean", "Naive", "AR(3)"]:

    mae = mean_absolute_error( forecast_df["Actual"], forecast_df[method] )
    rmse = np.sqrt( mean_squared_error(forecast_df["Actual"], forecast_df[method]) )

    results.append({ "Method": method, "MAE": mae, "RMSE": rmse })

evaluation = pd.DataFrame(results)

# MAE가 작은 순서로 정렬
evaluation = evaluation.sort_values("MAE")

print(evaluation.round(2).to_string(index=False))
```

◆ 예측 성능 비교 결과

Method	MAE	RMSE
AR(3)	98.95	115.29
Mean	108.47	133.89
Naive	135.33	151.91

◆ 분석 결과의 해석

□ AR(3) 모형의 MAE와 RMSE가 세 가지 예측방법 중 가장 작게 나타남

o 과거 평균값 대비 MAE는 약 8.8%, RMSE는 약 13.9% 감소

o 최근 값 대비 MAE는 약 26.9%, RMSE는 약 24.1% 감소

□ 과거 평균값 예측은 실제 취업자 수 증가폭을 전반적으로 과소예측하지만, 최근 값 예측보다는 작은 예측오차를 보임

□ 최근 값 예측은 직전 월의 변화량을 그대로 반영하기 때문에 월별 변동이 심한 경우 예측오차가 크게 나타남

□ AR(3) 모형은 과거 3개월의 자기상관 정보를 활용함으로써 단순 예측방법보다 높은 예측 정확도를 보인 것으로 평가

□ 다만 AR(3) 모형도 월별 취업자 수 변화량의 급격한 변동을 충분히 예측하지 못하며, 이번 결과가 다른 기간에서도 동일하게 나타난다고 보장할 수는 없음

◆ 정리

❶ AutoReg()를 이용하면 실제 시계열 자료로부터 AR 모형의 자기회귀계수를 추정할 수 있음

❷ AIC와 BIC는 서로 다른 차수의 AR 모형을 비교하는 데 사용되며, 정보기준이 작은 모형을 선호

❸ AR(3) 모형의 예측 성능을 평가하기 위해 과거 평균값 및 최근 값 예측과 비교

❹ 2018~2019년 미국 비농업부문 취업자 수 변화량에 대한 예측 결과, AR(3) 모형이 MAE와 RMSE 모두 가장 작은 것으로 나타남

❺ 이는 과거의 자기상관구조를 활용하는 것이 예측 정확도를 개선할 수 있음을 보여주지만, 예상하지 못한 단기 충격까지 정확하게 예측할 수 있다는 의미는 아님

◆ AR 과정과 PACF의 특징

□ 정상적인 AR(p) 과정에서는 PACF 계수들이 lag p까지만 유의하고, 그 이후에는 갑자기 유의하지 않게 됨

□ 따라서 MA(q) 과정에서는 ACF를 이용하여 차수 q를 결정하고,

AR(p) 과정에서는 PACF를 이용하여 차수 p를 결정

<정리> **AR 구조와 MA 구조의 구별**

패턴	의미
ACF가 천천히 감소	AR(p) 가능성
ACF가 특정 lag 이후 급격히 사라짐(cutoff)	MA(q) 가능성
ACF와 PACF가 모두 천천히 감소	ARMA 가능성

8.5.7 AR(3) 모형의 잔차 진단

□ 이번 절에서는 잔차의 자기상관을 분석하여 AR(3) 모형이 시계열 자료의 자기상관구조를 충분히 설명하고 있는지 검토

□ 이를 위해 잔차의 ACF를 확인하고, Ljung-Box 검정을 실시

8.5.7.1 잔차의 자기상관 분석

◆ 잔차의 개념

□ AR(3) 모형은 다음과 같이 표현됨

$$\Delta y_t = c + \phi_1 \Delta y_{t-1} + \phi_2 \Delta y_{t-2} + \phi_3 \Delta y_{t-3} + \epsilon_t$$

□ 모형을 추정한 후 실제값과 모형의 적합값(fitted value) 사이의 차이를 잔차(residual)라 함

$$\hat{\epsilon}_t = \Delta y_t - \widehat{\Delta y}_t$$

□ AR(3) 모형이 자료의 자기상관구조를 충분히 설명하였다면, 잔차에는 뚜렷한 자기상관이 남아 있지 않아야 함

o 잔차에 자기상관이 남아 있다면 모형이 시계열의 일부 정보를 충분히 반영하지 못했을 가능성이 있음

◆ Python 코드: 잔차의 ACF. 앞 절에서 추정한 'result_ar3'를 그대로 사용

```
import matplotlib.pyplot as plt
from statsmodels.graphics.tsaplots import plot_acf

# AR(3) 모형의 잔차
residuals = result_ar3.resid

# 잔차의 ACF
fig, ax = plt.subplots(figsize=(9, 4))

plot_acf(residuals, lags=24, ax=ax)

ax.set_title("ACF of AR(3) Residuals")
ax.set_xlabel("Lag")
ax.set_ylabel("Autocorrelation")
plt.tight_layout()
plt.show()
```

◆ 코드 설명

□ result_ar3.resid : AR(3) 모형에서 추정된 잔차를 추출

□ plot_acf(residuals, lags=24) : 잔차의 자기상관계수를 24개월 시차까지 계산하여 시각화

□ 그래프의 음영 영역은 자기상관계수가 0이라는 가설에 대한 대략적인 95% 신뢰구간을 나타냄

◆ AR(3) 모형의 잔차 ACF 해석

◇ AR(3) 모형의 잔차에는 뚜렷한 자기상관 패턴이 나타나지 않아 모형이 자료의 자기상관구조를 비교적 잘 설명하고 있는 것으로 평가할 수 있음

□ 시차 0을 제외한 대부분의 잔차 자기상관계수가 0에 가까우며, 95% 신뢰구간 안에 위치

□ 특히 시차 1~3에서 자기상관계수가 거의 0으로 나타나, AR(3) 모형이 단기적인 자기상관구조를 상당 부분 설명한 것으로 판단됨

□ 시차 5~24에서도 자기상관계수가 대체로 작은 값을 보이며, 특정한 증가·감소 패턴이나 뚜렷한 주기성이 관찰되지 않음

□ 다만 잔차 ACF만으로 자기상관이 완전히 제거되었다고 단정할 수는 없으므로, Ljung-Box 검정을 통해 여러 시차의 자기상관이 공동으로 0인지 확인할 필요가 있음.

8.5.7.2 Ljung-Box 검정

◆ Ljung-Box 검정의 개념

□ Ljung-Box 검정은 잔차의 여러 시차에 걸친 자기상관계수가 공동으로 0인지 검정

□ 귀무가설과 대립가설은 다음과 같음

$$H_0 : \rho_1 = \rho_2 = \cdots = \rho_h = 0$$

$$H_1 : \text{적어도 하나의 } \rho_k \neq 0$$

ρ_k는 잔차의 k 시차 자기상관계수, h는 검정에 포함되는 최대 시차를 의미

□ Ljung-Box 검정통계량은 다음과 같음

$$Q = n(n+2)\sum_{k=1}^{h} \frac{\hat{\rho}_k^2}{n-k}$$

n은 잔차의 관측값 수, $\hat{\rho}_k$는 잔차의 표본 자기상관계수

□ 유의확률(p-value)이 0.05보다 작으면 귀무가설을 기각하여 잔차에 자기상관이 남아 있다는 증거가 있다고 판단

□ 반대로 유의확률이 0.05 이상이면 귀무가설을 기각하지 못하며, 잔차에 자기상관이 남아 있다는 충분한 증거가 없는 것으로 해석

◆ Python 코드: Ljung-Box 검정

```
from statsmodels.stats.diagnostic import acorr_ljungbox

# Ljung-Box 검정
lb_test = acorr_ljungbox( residuals, lags=[6, 12, 24], model_df=3, return_df=True )

print(lb_test.round(4))
```

◆ 코드 설명

□ acorr_ljungbox() : Ljung-Box 검정을 수행하는 함수

□ lags=[6, 12, 24] : 잔차의 자기상관을 각각 6개월, 12개월, 24개월 시차까지 검정

□ model_df=3 : 추정한 AR(3) 모형의 자기회귀계수 3개를 고려하여 검정의 자유도를 조정

□ return_df=True : 검정 결과를 DataFrame 형태로 반환

◆ 출력 결과의 해석

	lb_stat	lb_pvalue
6	4.4074	0.2207
12	9.9821	0.3519
24	26.2107	0.1985

□ 모든 시차에서 유의확률이 0.05보다 크게 나타나, 잔차의 자기상관계수가 공동으로 0이라는 귀무가설을 기각하지 못함

o 시차 6: p=0.2207

o 시차 12: p=0.3519

o 시차 24: p=0.1985

⇒ AR(3) 모형의 잔차에 통계적으로 유의한 자기상관이 남아 있다는 증거가 없음을 의미

□ 앞서 살펴본 잔차 ACF에서도 대부분의 자기상관계수가 95% 신뢰구간 안에 위치하여 Ljung-Box 검정 결과와 대체로 일치

◆ 정리

❶ AR 모형의 잔차는 실제값과 적합값의 차이로, 모형이 설명하지 못한 변동을 의미

❷ 잔차 ACF는 각 시차에서 자기상관이 남아 있는지 시각적으로 확인하는 방법

❸ Ljung-Box 검정은 여러 시차의 잔차 자기상관이 공동으로 0인지 검정하는 방법

❹ 잔차에 유의한 자기상관이 남아 있지 않다면 AR(3) 모형이 자료의 자기상관구조를 비교적 잘 설명하고 있다고 평가할 수 있음

❺ 반대로 잔차에 자기상관이 남아 있다면 AR 모형의 차수를 변경하거나 다른 시계열 모형을 고려할 필요가 있음